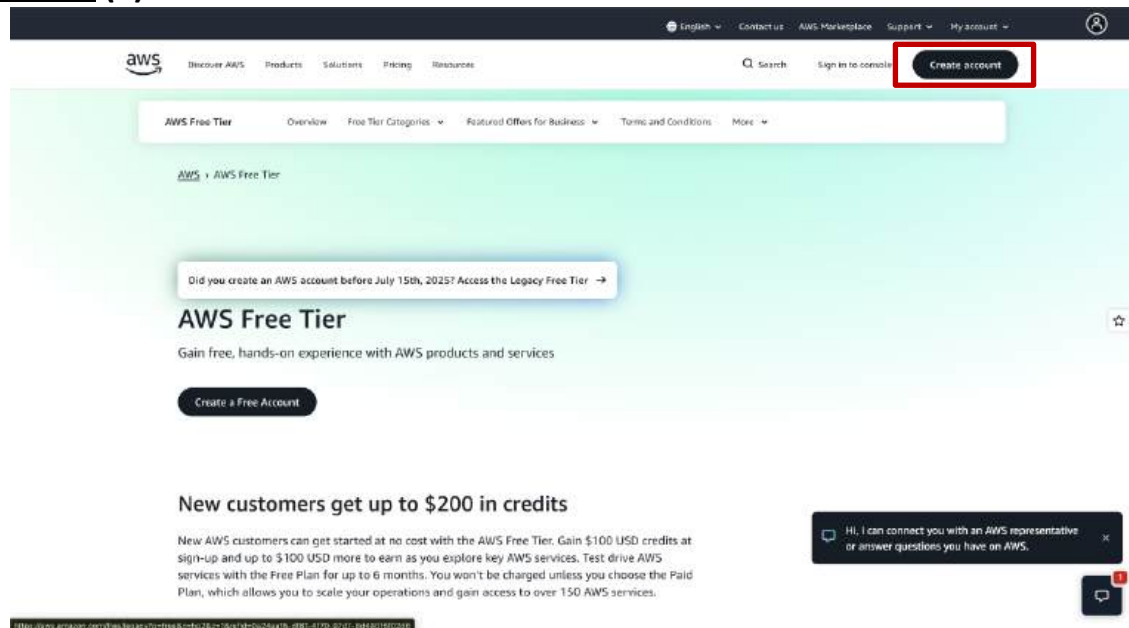


ASSIGNMENT 1

Problem Statement: Create an account in AWS and configure a budget.

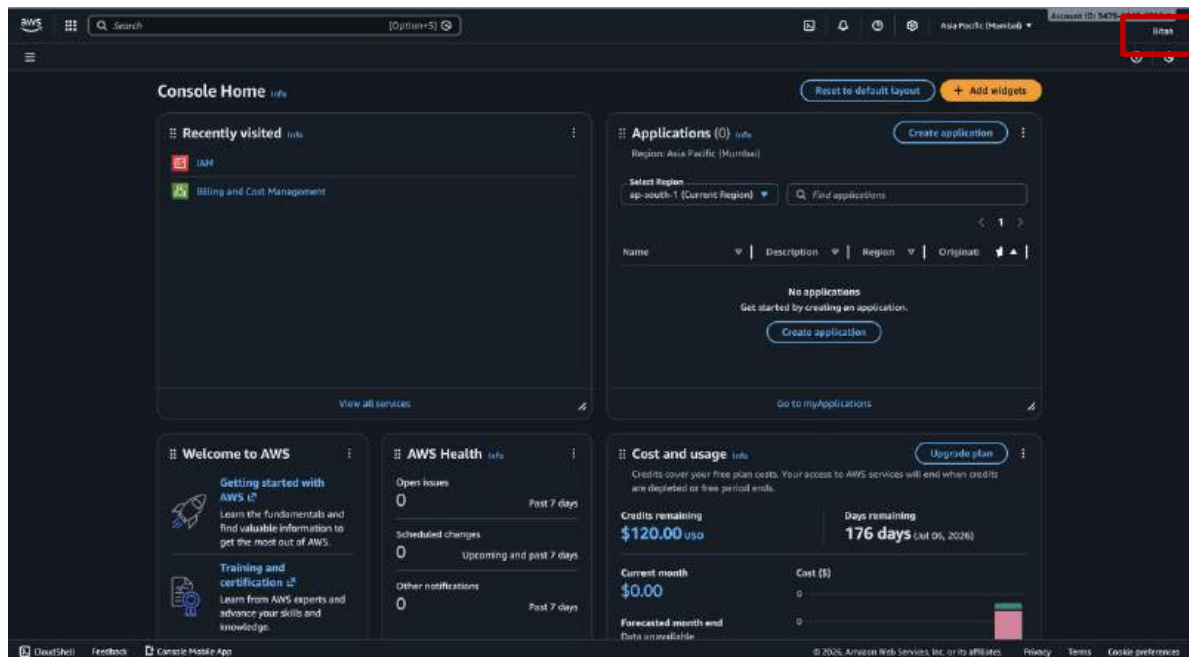
Procedure: (a) Create an account in AWS



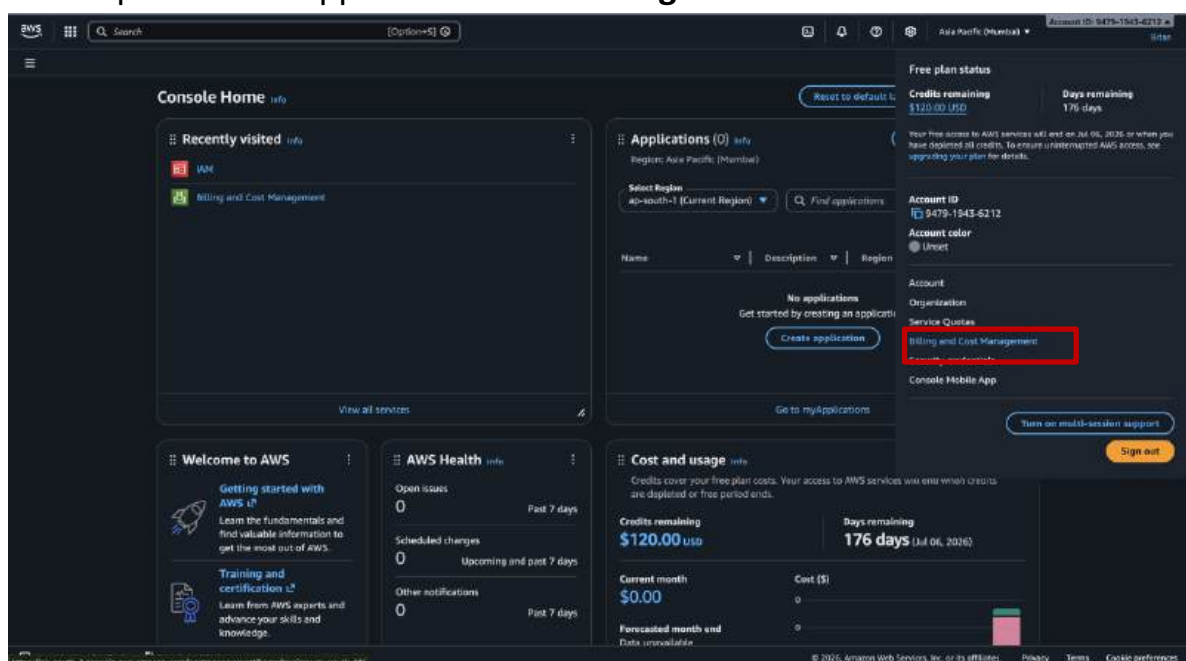
1. Open the [Amazon Web Services home page](#) .
2. Choose **Create an AWS account**. Make sure to create the account as a **Root user** to unlock full functionality of the newly created AWS account.
3. Enter our account information, and then choose Verify Email . Be sure that we enter our account information correctly, especially email address.
4. Choose **personal** account.
5. Enter our personal information.
6. Read and accept the **AWS Customer Agreement**.
7. Choose Create Account and Continue.
8. On the Payment Information page we can use card payment or upi , and then choose **Verify and Add**. We can't proceed with the sign-up process until we do valid payment method.
9. Next, we must **verify our phone number**. Choose our country or region code from the list, and enter a phone number .
10. Enter the code displayed in the CAPTCHA, and then submit.
11. On the **Select a Support Plan** page, choose the free tier.
12. Finally, wait for your new account to be **activated**. This usually takes a few minutes but can take up to **24 hours**.
13. When our account is **fully activated**, we receive a **confirmation email message**. Check email and spam folder for the confirmation message. **After receive this email message, we have full access to all AWS services.**

(b)Configure a Budget

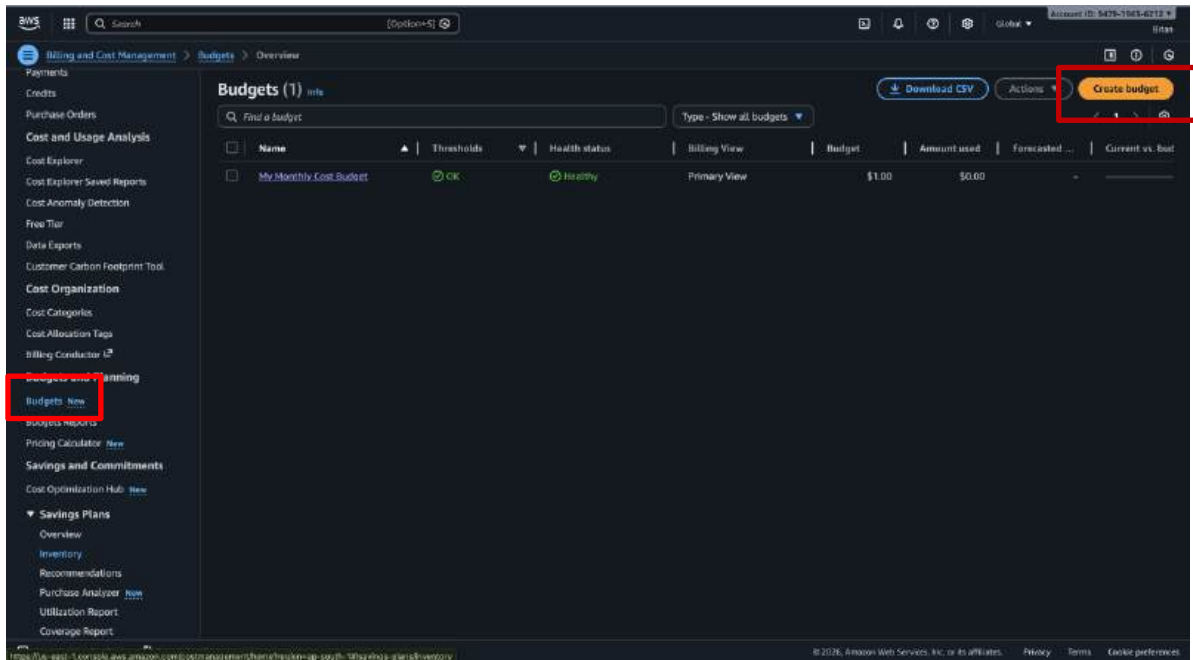
1. Sign in to the console using root email for newly created account.
2. Now after successfully signing in we will arrive at the home page. Then click on down arrow beside your account name on the top right corner of the home page.



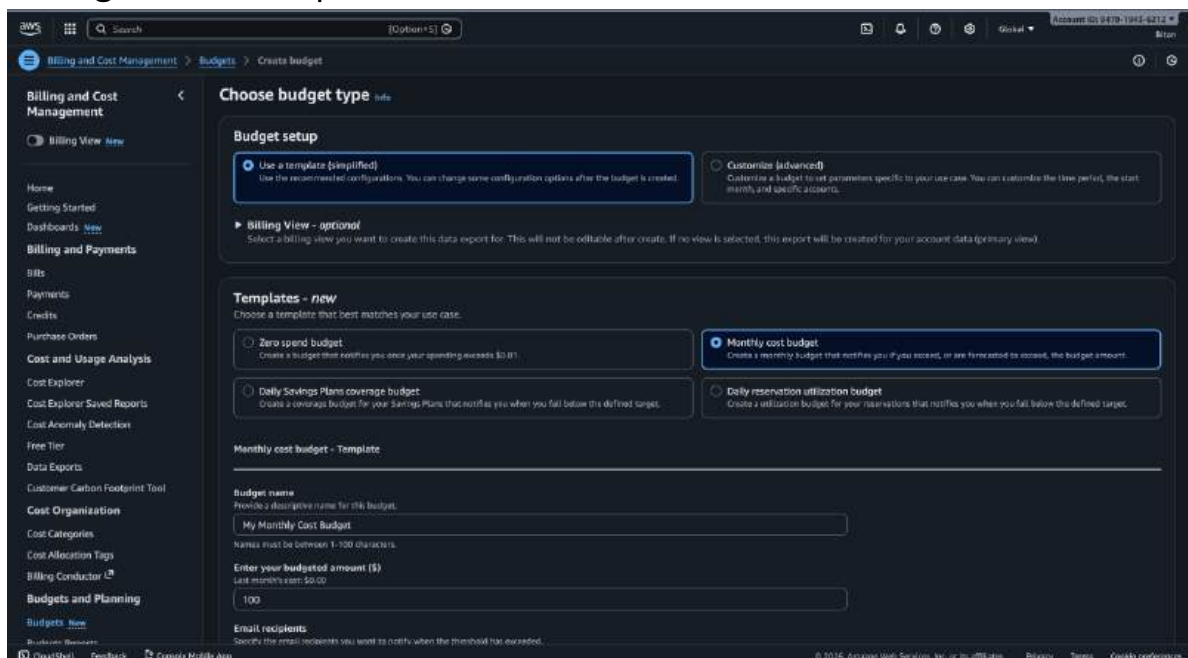
3. A drop down will appear and select **Billing Dashboard** from the list.



4. After arriving in Billing Dashboard, go to the **Budgets** section on the left side panel under cost management. Click on the **Create Budget** button in the **overview page** to start creating a new budget for your AWS account.



5. Select Use a template(simplified) in Budget Setup section and Monthly **Cost Budget** in the Templates-new section.



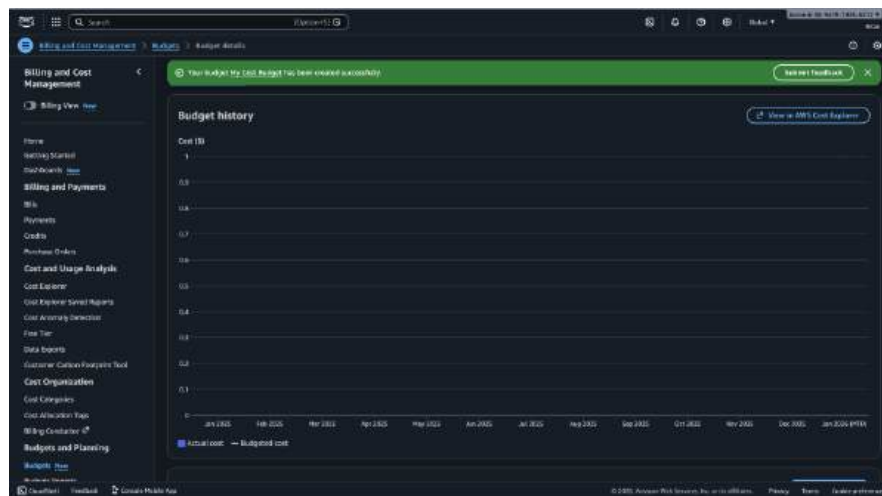
6. Enter **Budget Name** and Enter your Budgeted amount. For best free usage enter 1 in the box. (All amounts are automatically considered in dollars "\$") No need to change any other options in the page. After that enter the Email recipients to receive email about my budget. So, now we click on Create budget.

The screenshot shows the AWS Billing and Cost Management console. The left sidebar contains navigation links for Billing and Cost Management, Billing View, Home, Getting Started, Dashboards, Billing and Payments, Bills, Payments, Credits, Purchase Orders, Cost and Usage Analysis, Cost Explorer, Cost Explorer Saved Reports, Cost Anomaly Detection, Free Tier, Data Exports, Customer Carbon Footprint Tool, Cost Organization, Cost Categories, Cost Allocation Tags, Billing Conductor, Budgets and Planning, and Budgets. The main content area is titled 'Create budget' and shows two options: 'Daily Savings Plans coverage budget' and 'Daily reservation utilization budget'. The 'Monthly cost budget - Template' is selected. The 'Budget name' field is 'My Cost Budget'. The 'Enter your budgeted amount (\$)' field is '1.00'. The 'Email recipients' field is 'bhanikamukherjee@gmail.com'. The 'Scope' section shows 'You will be notified when 1) your actual spend reaches 85% 2) your actual spend reaches 100% 3) if your forecasted spend is expected to reach 100%'. The 'Create budget' button is highlighted in orange.

7. Next, again we click the My Cost Budget. Then go to Alerts section and select Actual cost > 85% and Actual cost > 100%. We can also select the Forecasted cost > 100%. Then click the Budgets to exit from that.

The screenshot shows the AWS Billing and Cost Management console. The left sidebar contains navigation links for Billing and Cost Management, Billing View, Home, Getting Started, Dashboards, Billing and Payments, Bills, Payments, Credits, Purchase Orders, Cost and Usage Analysis, Cost Explorer, Cost Explorer Saved Reports, Cost Anomaly Detection, Free Tier, Data Exports, Customer Carbon Footprint Tool, Cost Organization, Cost Categories, Cost Allocation Tags, Billing Conductor, Budgets and Planning, and Budgets. The main content area is titled 'Budget details' and shows a green banner stating 'Your budget My Cost Budget has been created successfully.' Below this, there are tabs for 'Budget history', 'Alerts', 'Tags', and 'Billing View'. The 'Alerts' tab is selected, showing three alerts: 'Actual cost > 85%', 'Actual cost > 100%', and 'Forecasted cost > 100%'. The 'Actual cost > 85%' and 'Actual cost > 100%' alerts are marked as 'Not exceeded'. The 'Forecasted cost > 100%' alert is marked as 'Not exceeded'. The 'Budgets' link in the left sidebar is highlighted with a red box.

8. Again, click on My Cost Budget and enter into it .
9. Now, we are finally at the review or summary page of the whole budget . We can see the Budget history and Monthly cost history



Date	Actual	Budgeted	Budget variance (\$USD)	Budget variance (%)
January 2020 (\$M)	\$0.00	\$1.00	\$1.00	100.00%
December 2019	\$0.00	-	-	-
November 2019	\$0.00	-	-	-
October 2019	\$0.00	-	-	-
September 2019	\$0.00	-	-	-
August 2019	\$0.00	-	-	-
July 2019	\$0.00	-	-	-
June 2019	\$0.00	-	-	-
May 2019	\$0.00	-	-	-
April 2019	\$0.00	-	-	-
March 2019	\$0.00	-	-	-
February 2019	\$0.00	-	-	-
January 2019	\$0.00	-	-	-

10. After this step we will be redirected to the **overview** page where all budgets will be shown. We can see your newly created budget in the table format with various information related to it. Our Budget creation is complete!

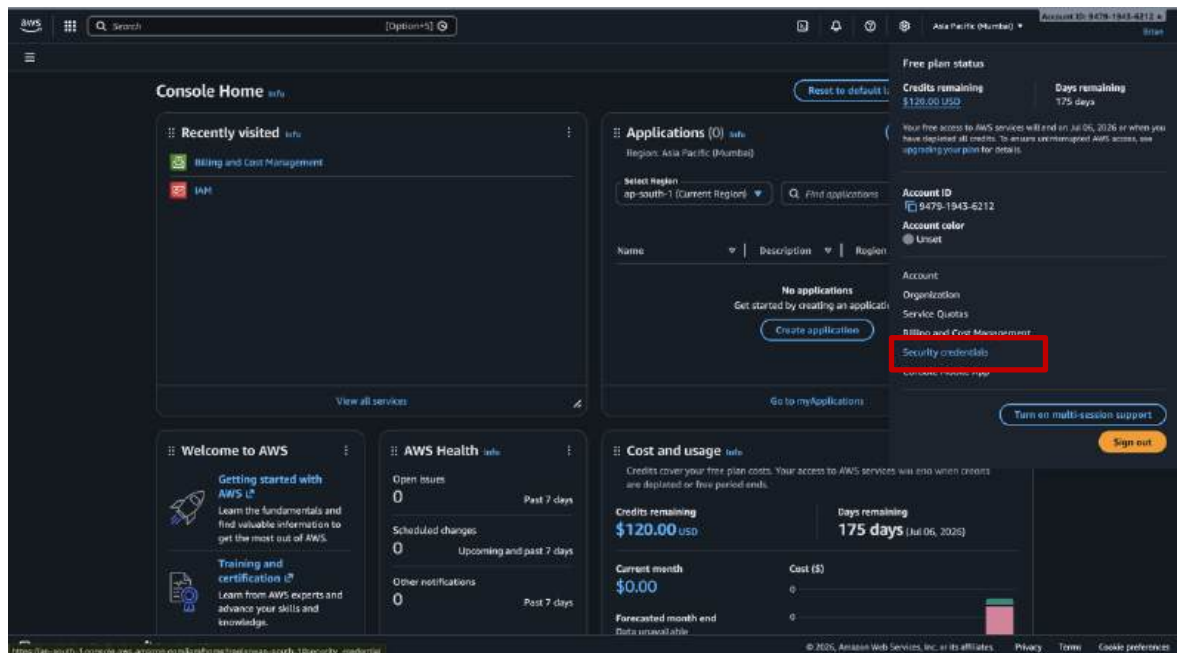
Budget Name	Status	Type	Primary View	Budgeted Cost (\$USD)	Actual Cost (\$USD)
My Cost Budget	OK	Monthly	Primary View	\$1.00	\$0.00
My Monthly Cost Budget	OK	Monthly	Primary View	\$1.00	\$0.00

ASSIGNMENT 2

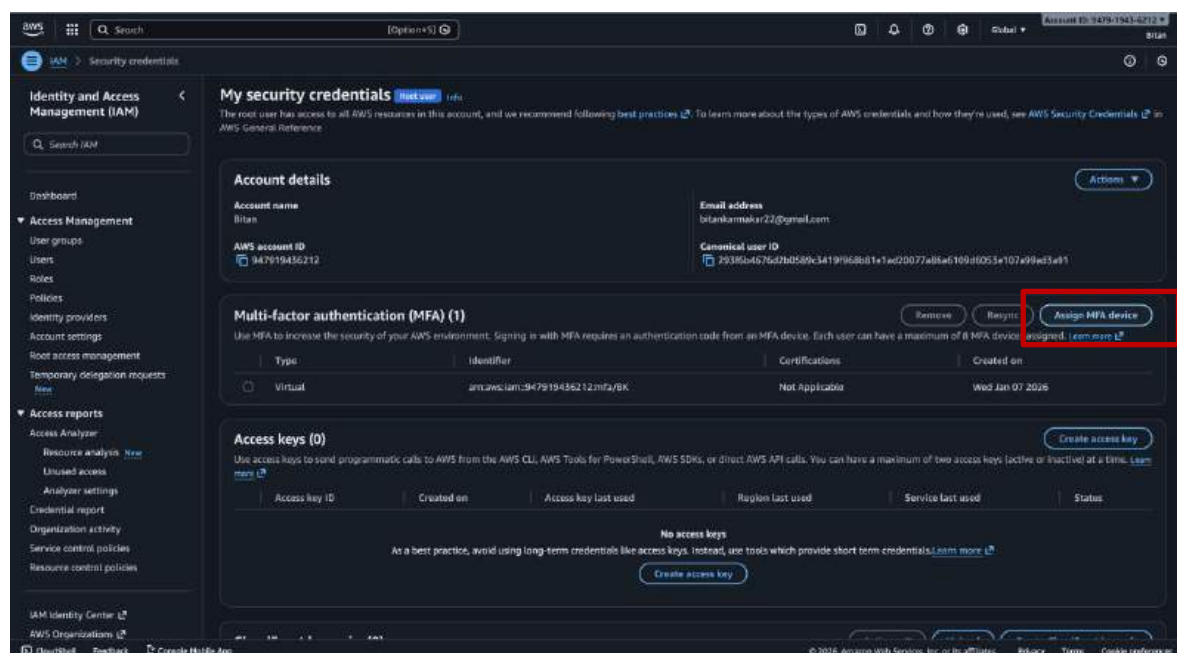
Problem Statement: Create Multi Factor Authentication(MFA) for account authentication.

Procedure:

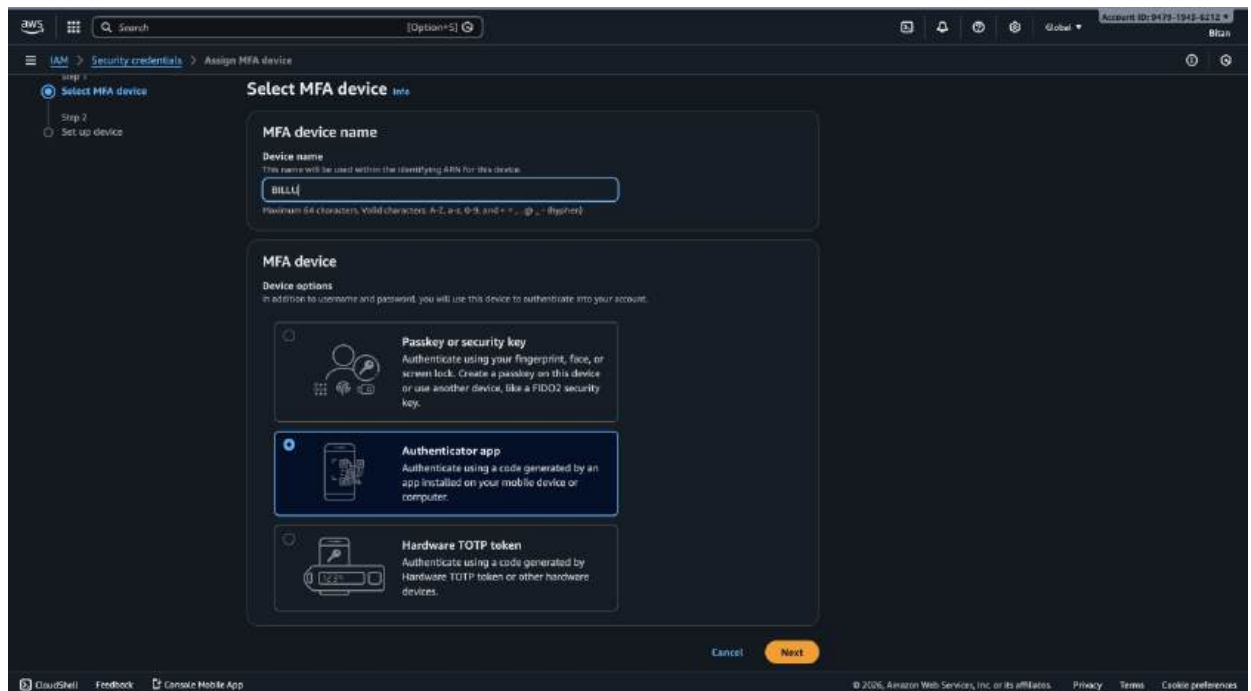
1. Sign-in to AWS console. Then select your account name in the top right of the page. Now select the Security credentials option in the drop-down menu .



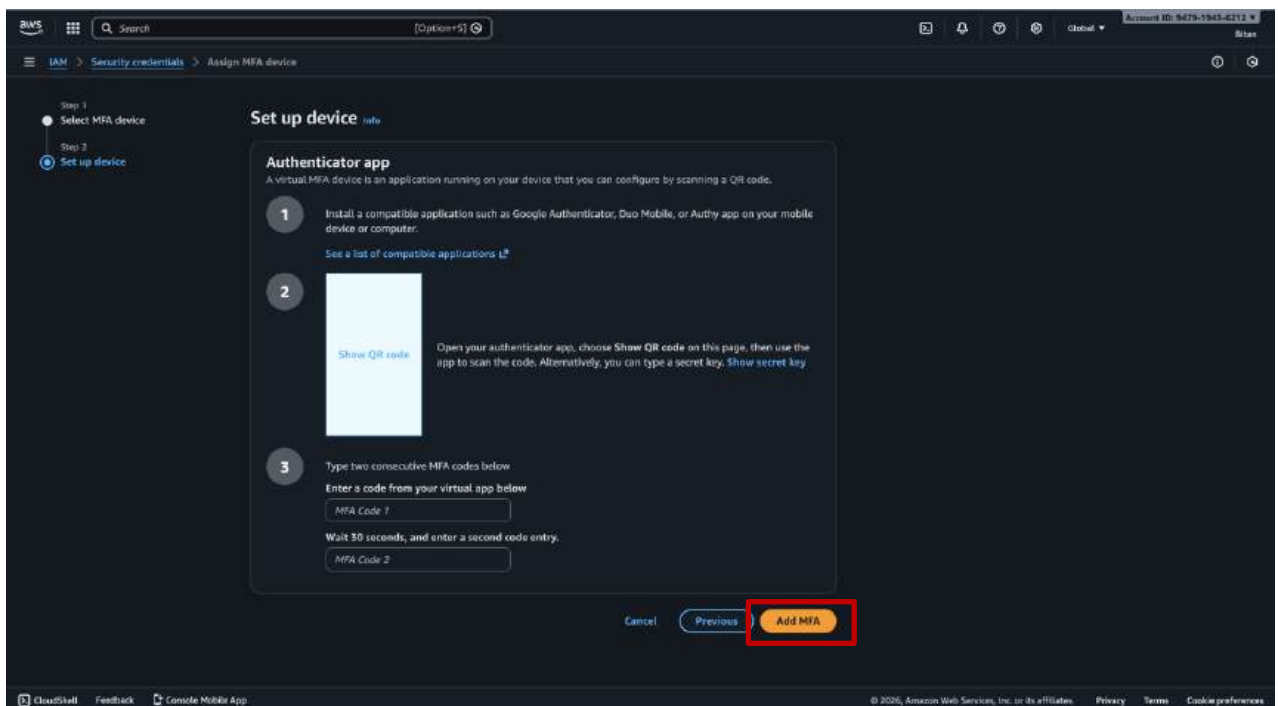
2. Now after arriving in My Security Credentials page, now scroll down to **Multi-Factor Authentication (MFA)** section. Click on the **Assign MFA device** button.



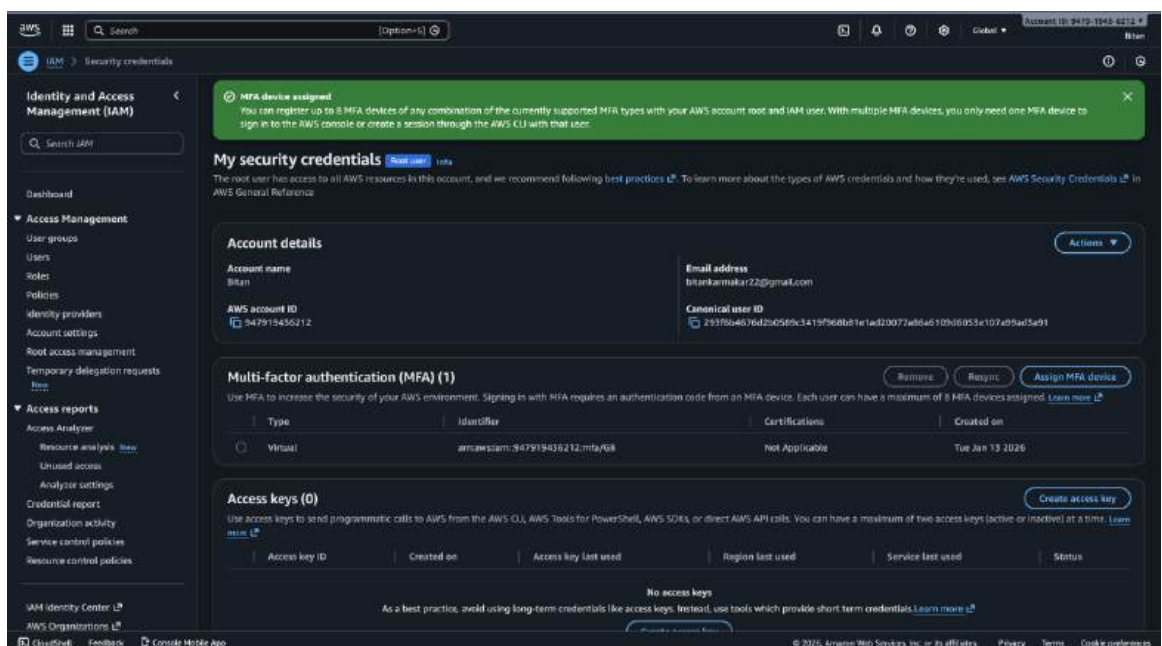
3. Next Assign a **unique device name** (very important and needs to be unique). This name is actually the name of the account that shows up in your authenticator app with the security codes and hence is essential for identification later. Select **Authenticator app** in **Select MFA device** section.



4. Click on **Next**.
5. After that you have to make sure you download an authenticator app from in your android/iOS device. Google/Microsoft authenticator is preferred. After installation, click on the **Show QR code box** here in the website. Scan the QR code with your authenticator app. Your device name given will show up in your authenticator main page. We will see a certain unique combination appearing against our given device name for our AWS account and stays only for 30-60 seconds. Enter 2 consecutive codes appearing in the given box in the website to authenticate your MFA. After successfully verifying your MFA, click on the Add MFA button.



6. You will be redirected to the security credentials page and see your newly added MFA method with your given device name for your account.



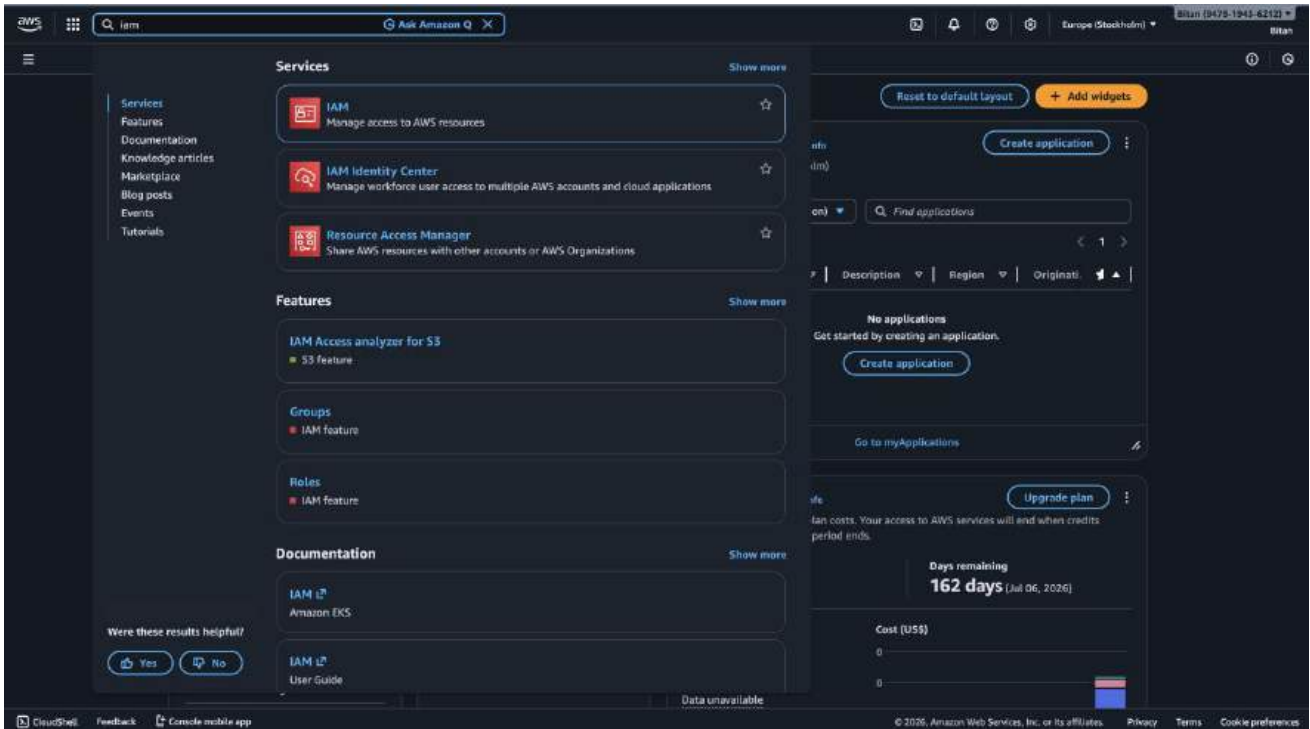
7. Hence, we have successfully added an MFA device. Now sign-out and re-login to the console using Root user.
8. Now after providing user email and password, from now on you have to enter the MFA code. The code changes every 30-60 seconds and we have to enter the current or existing one which has not expired.

ASSIGNMENT 3

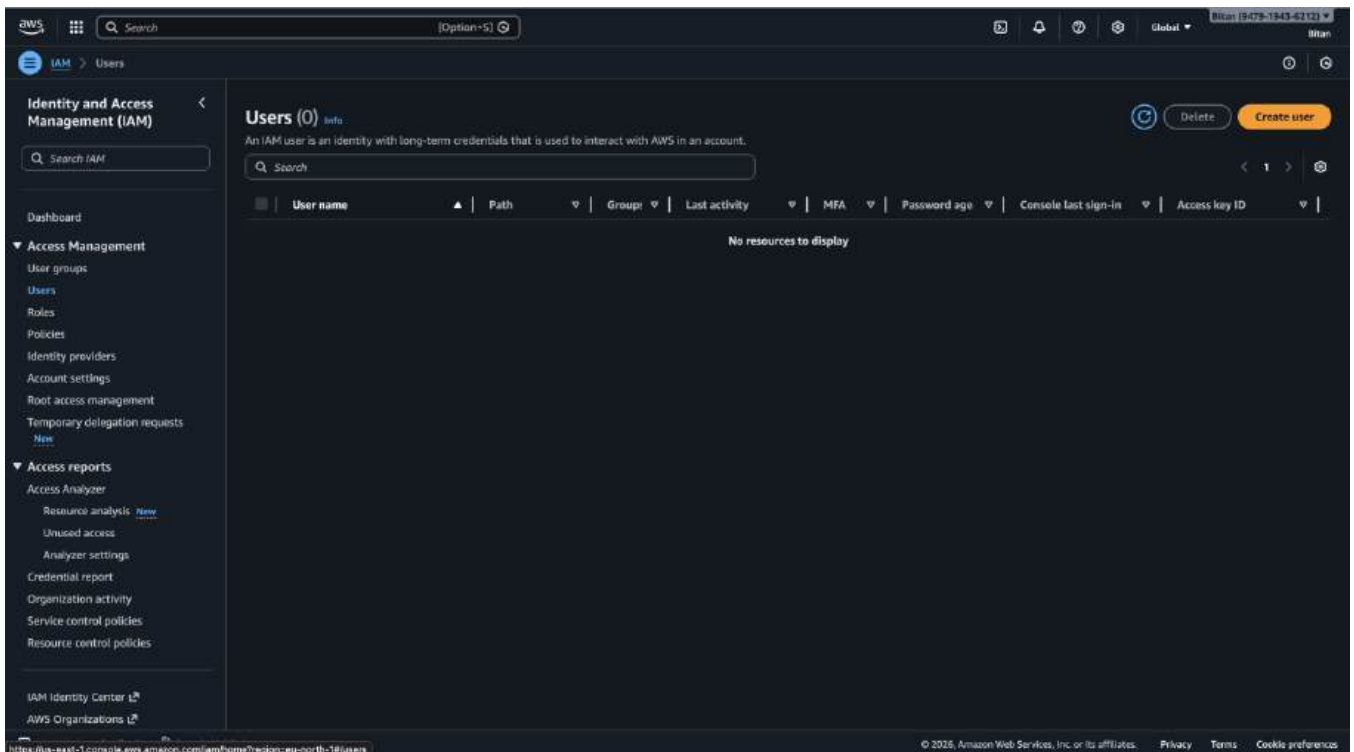
Problem Statement: Creating an IAM User and give full S3 access.

Procedure:

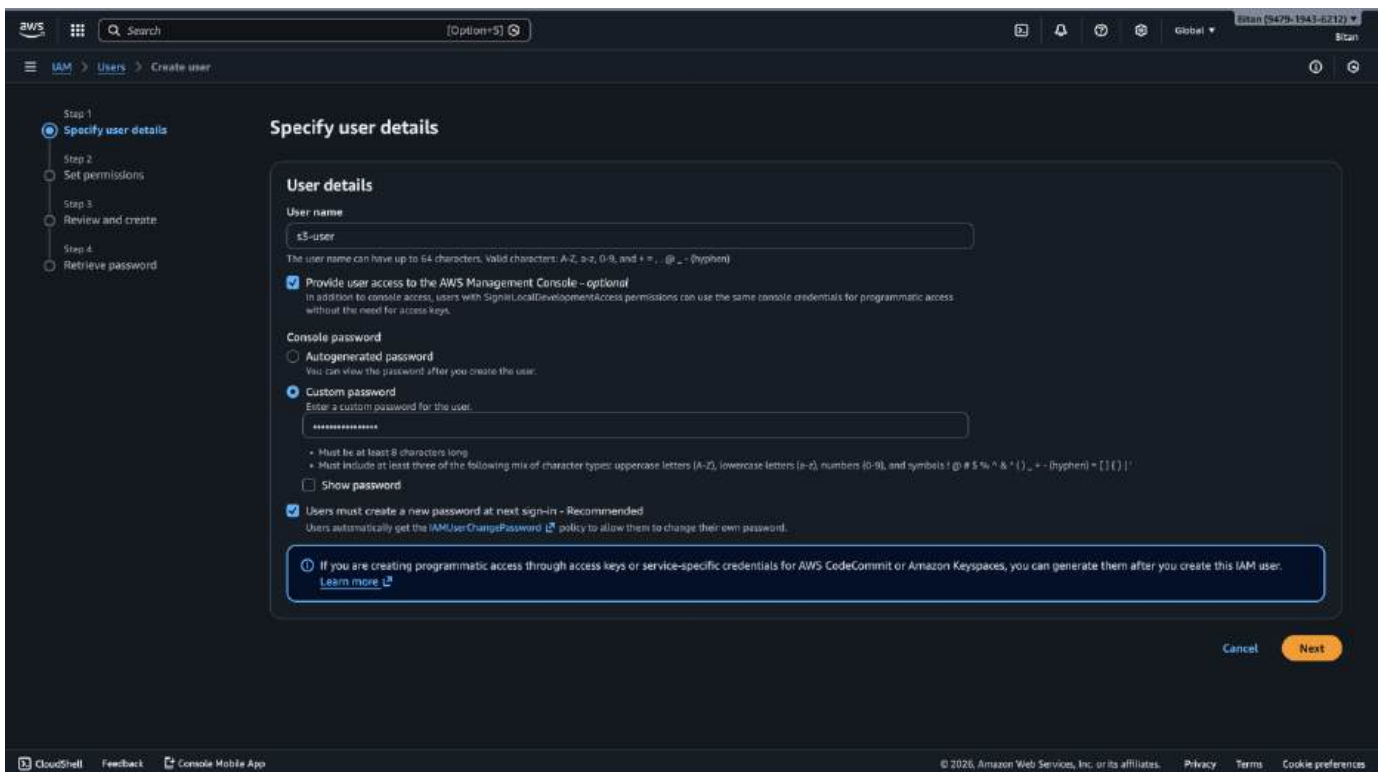
Step 1: Log in to the **AWS Management Console** as the Root user (or an Admin). Search for and select **IAM**



Step 2: In the left sidebar, click **Users**. Click the orange **Create user** button.



Step 3: Enter a **User name** . Check **Provide user access to the AWS Management Console**. Select **I want to create an IAM user**. Set a custom password and uncheck "Users must create a new password" Click **Next**.



Specify user details

User details

User name
s3-user
The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and +, =, ., @, _ - (hyphen)

☒ **Provide user access to the AWS Management Console - optional**
In addition to console access, users with `SignInLocalDevelopmentAccess` permissions can use the same console credentials for programmatic access without the need for access keys.

Console password

☐ **Autogenerated password**
You can view the password after you create the user.

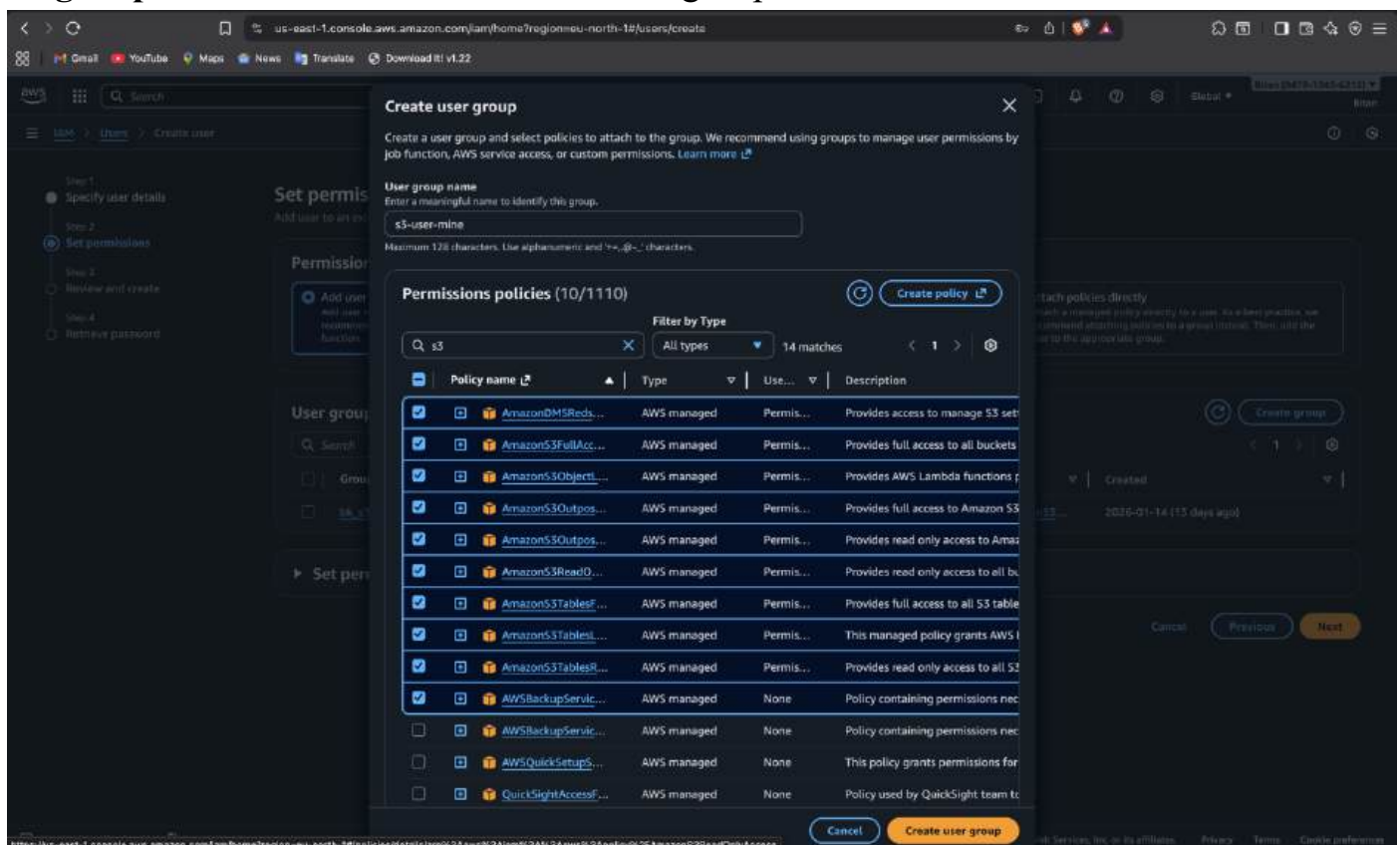
☒ **Custom password**
Enter a custom password for the user:
[password field]
Must be at least 8 characters long.
Must include at least three of the following mix of character types: uppercase letters (A-Z), lowercase letters (a-z), numbers (0-9), and symbols ! @ # \$ % ^ & * () _ + - (hyphen) = [] { } | ' ~

☐ **Show password**

☒ **Users must create a new password at next sign-in - Recommended**
Users automatically get the `IAMUserChangePassword` policy to allow them to change their own password.

Next

Step 4: Under **Permission options**, select **Add user to group**. Click **Create group**. Enter a group name . Search for and check **AmazonS3FullAccess**. Click **Create user group**. Back on the list, ensure the new group is **checked**. Click **Next**.



Create user group

Create a user group and select policies to attach to the group. We recommend using groups to manage user permissions by job function, AWS service access, or custom permissions. [Learn more](#)

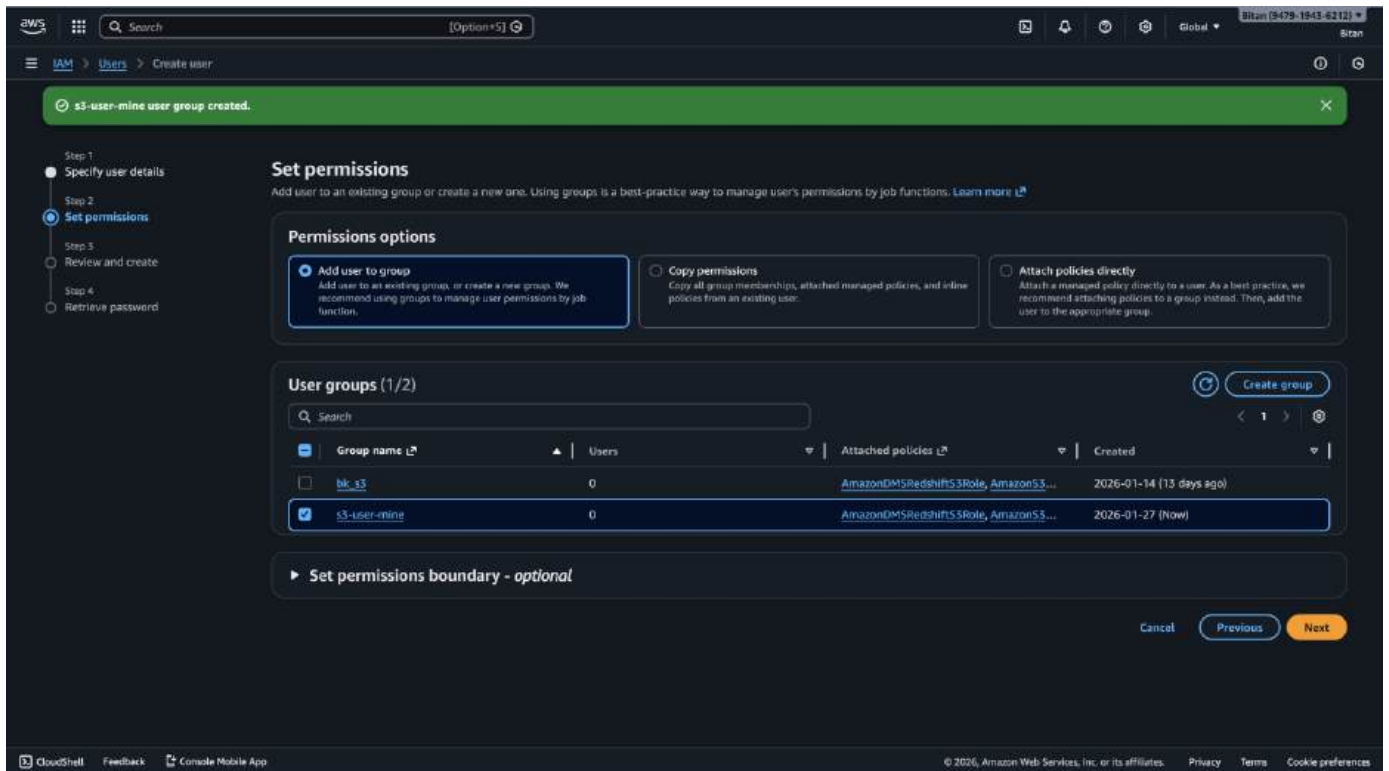
User group name
Enter a meaningful name to identify this group.
s3-user-mine
Maximum 128 characters. Use alphanumeric and +, =, @, _ - characters.

Permissions policies (10/1110)

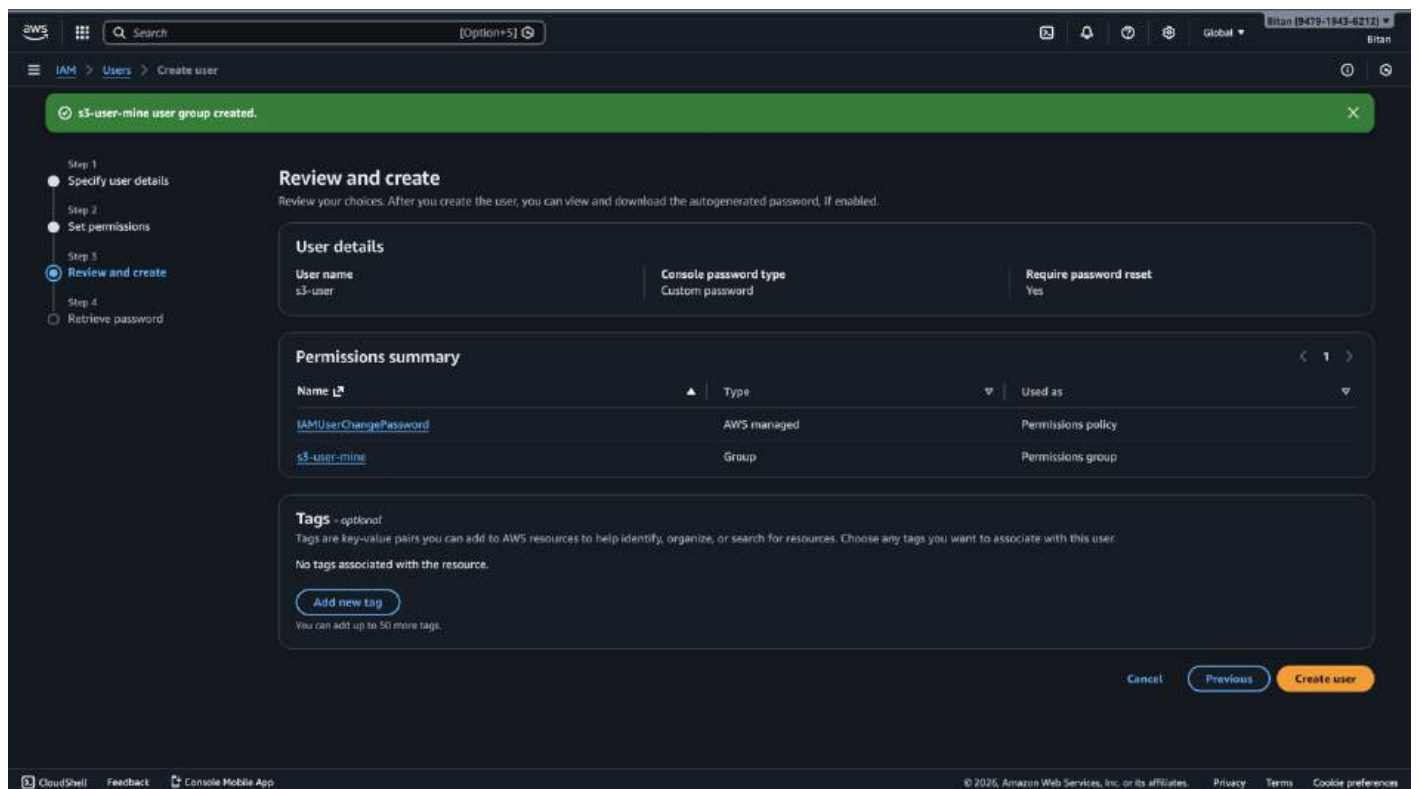
Filter by Type: All types 14 matches

Policy name	Type	Use...	Description
☒ AmazonS3FullAccess	AWS managed	Permis...	Provides full access to all buckets
☒ AmazonS3Outposts...	AWS managed	Permis...	Provides full access to Amazon S3 Outposts
☒ AmazonS3Outposts...	AWS managed	Permis...	Provides read only access to Amazon S3 Outposts
☒ AmazonS3ReadOnly...	AWS managed	Permis...	Provides read only access to all buckets
☒ AmazonS3Tables...	AWS managed	Permis...	Provides full access to all S3 tables
☒ AmazonS3Tables...	AWS managed	Permis...	This managed policy grants AWS IAM permissions for S3 tables
☒ AmazonS3Tables...	AWS managed	Permis...	Provides read only access to all S3 tables
☒ AWSBackupService...	AWS managed	None	Policy containing permissions nec...
☐ AWSBackupService...	AWS managed	None	Policy containing permissions nec...
☐ AWSQuickSetupS...	AWS managed	None	This policy grants permissions for...
☐ QuickSightAccess...	AWS managed	None	Policy used by QuickSight team to...

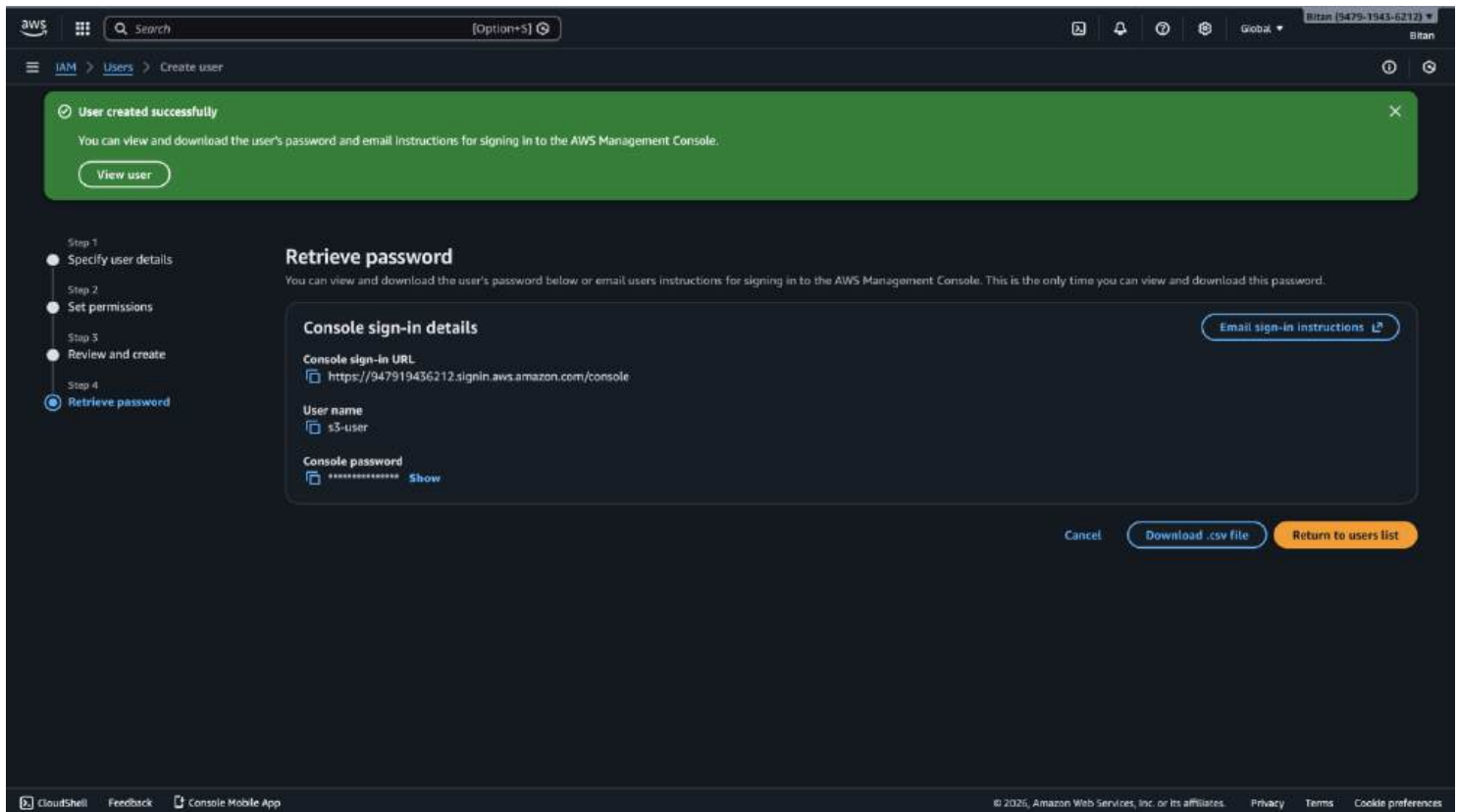
Create user group



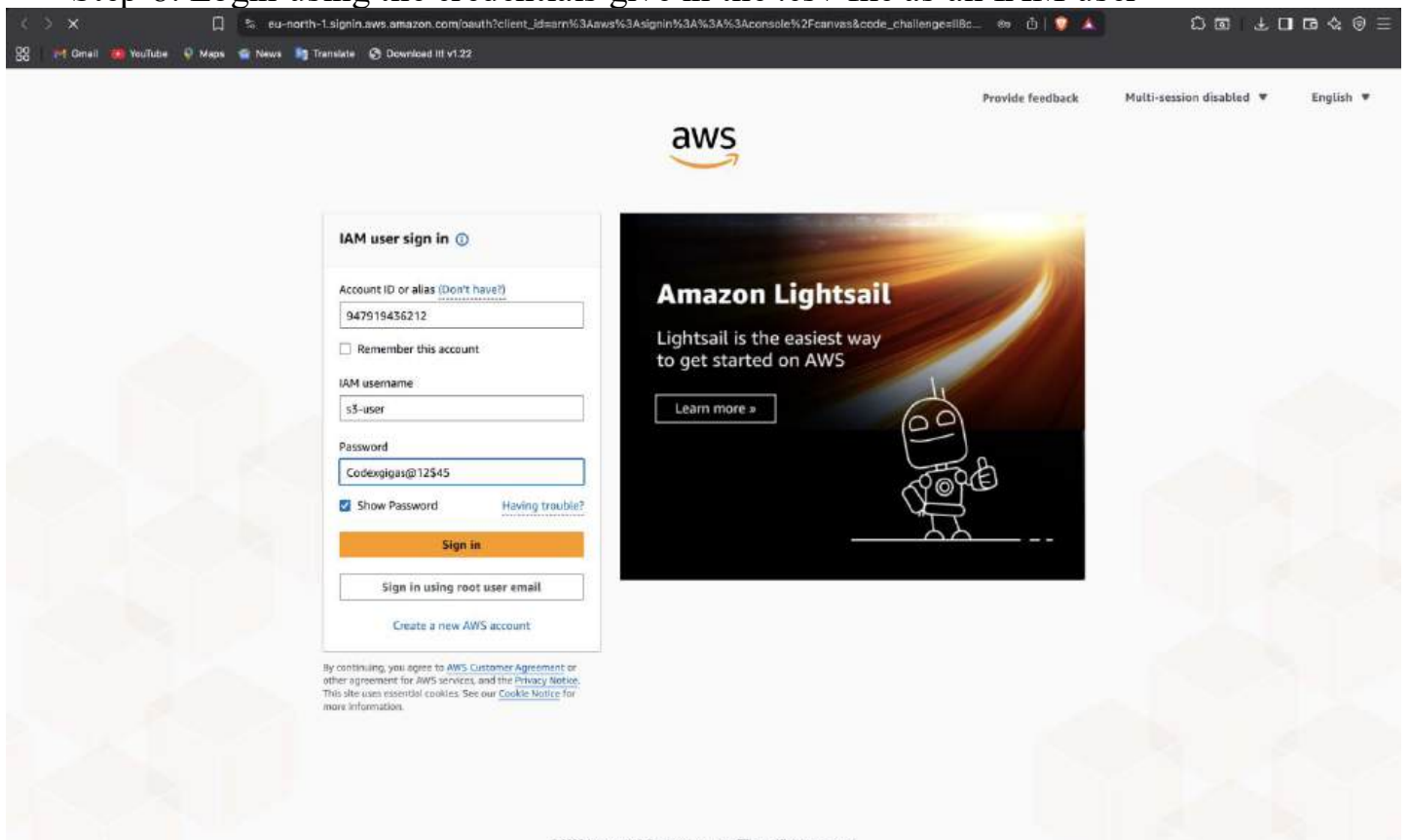
Step 5: On the **Review** page, confirm the entered details. Click **Create user**.



Step 6: Click Download the .csv to save the login details. Click **Return to users list**
Step 7: Newly created user will show up here



Step 8: Login using the credentials give in the .csv file as an IAM user

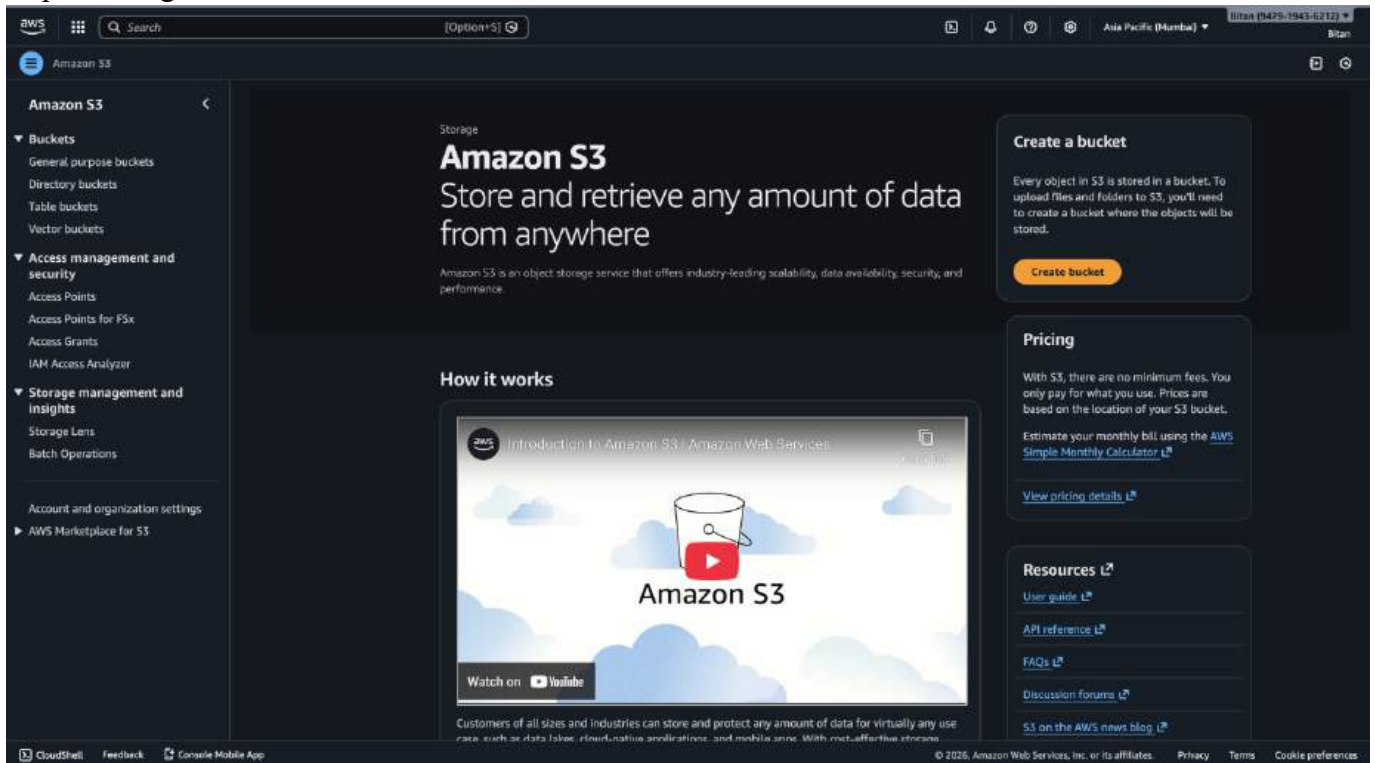


ASSIGNMENT 4

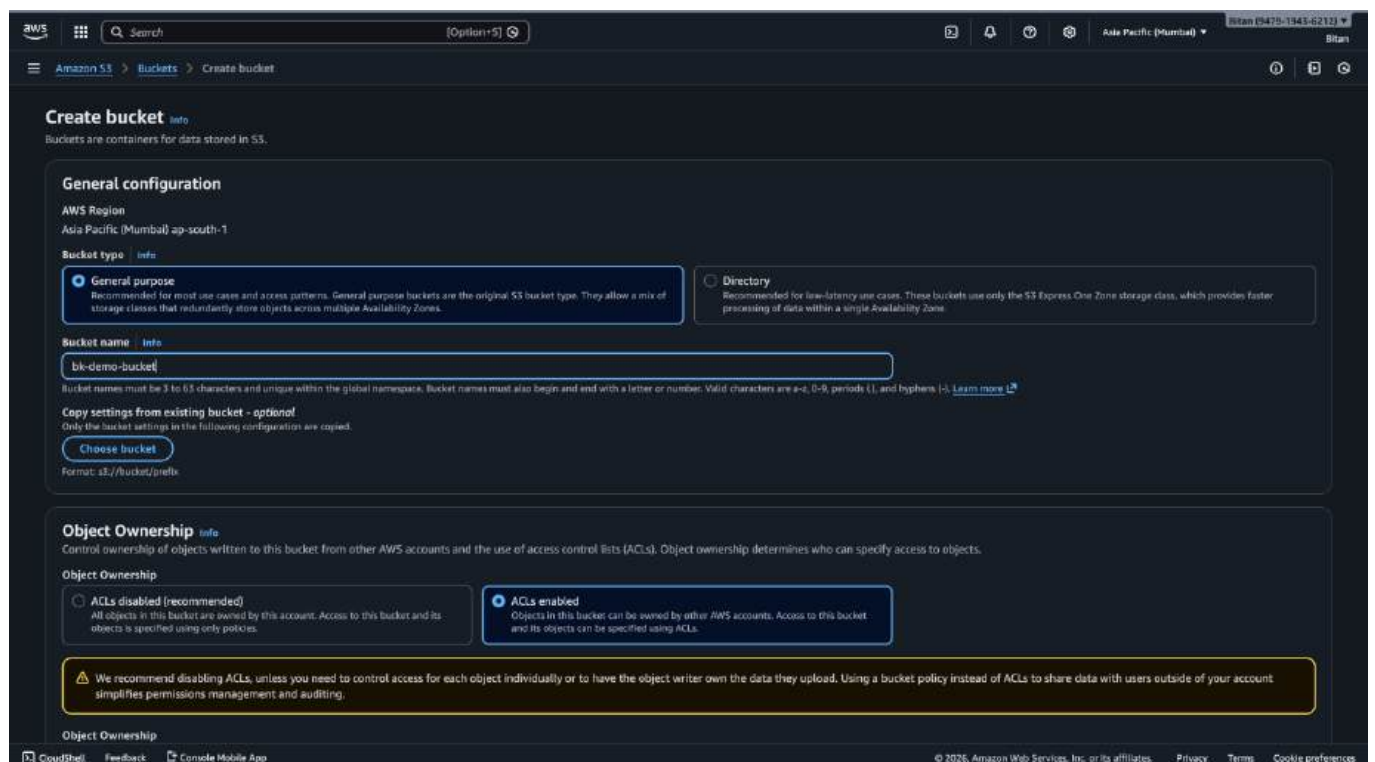
Problem Statement: Create a public bucket in AWS. Upload a file and give the necessary permission to check the file URL is working or not.

Procedure:

Step 1: Navigate to **Buckets** in **Amazon S3 Console** then select **Create bucket**



Step 2: Enter a unique bucket name. **Object Ownership**: Select ACLs enabled (and "Bucket owner preferred"). Uncheck **Block all Public Access**. Check the acknowledgment box. Lastly **Create bucket**.



[Options]
Asia Pacific (Mumbai)
Bitan (9479-1543-6212)

[Amazon S3](#) > [Buckets](#) > Create bucket

Object Ownership

☒ **Bucket owner preferred**
If new objects written to this bucket specify the bucket-owner-full-control canned ACL, they are owned by the bucket owner. Otherwise, they are owned by the object writer.

☐ **Object writer**
The object writer remains the object owner.

! If you want to enforce object ownership for new objects only, your bucket policy must specify that the bucket-owner-full-control canned ACL is required for object uploads. [Learn more](#)

Block Public Access settings for this bucket

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

☐ **Block all public access**
Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

- ☐ **Block public access to buckets and objects granted through new access control lists (ACLs)**
S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.
- ☐ **Block public access to buckets and objects granted through any access control lists (ACLs)**
S3 will ignore all ACLs that grant public access to buckets and objects.
- ☐ **Block public access to buckets and objects granted through new public bucket or access point policies**
S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.
- ☐ **Block public and cross-account access to buckets and objects through any public bucket or access point policies**
S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

! Turning off Block all public access might result in this bucket and the objects within becoming public. AWS recommends that you turn on block all public access, unless public access is required for specific and verified use cases such as static website hosting.

☒ I acknowledge that the current settings might result in this bucket and the objects within becoming public.

Bucket Versioning

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. [Learn more](#)

[Bucket Metadata](#)

[CloudShell](#) [Feedback](#) [Console Mobile App](#)

© 2025, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

[Options]
Asia Pacific (Mumbai)
Bitan (9479-1543-6212)

[Amazon S3](#) > [Buckets](#) > Create bucket

Tags - optional

You can use bucket tags to analyze, manage and specify permissions for a bucket. [Learn more](#)

! You can use s3:ListTagsForResource, s3:TagResource, and s3:UntagResource APIs to manage tags on S3 general purpose buckets for access control in addition to cost allocation and resource organization. To ensure a seamless transition, please provide permissions to s3:ListTagsForResource, s3:TagResource, and s3:UntagResource actions. [Learn more](#)

No tags associated with this bucket.

[Add new tag](#)

You can add up to 50 tags.

Default encryption

Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption type [info](#)

Secure your objects with two separate layers of encryption. For details on pricing, see [DSSE-KMS pricing on the Storage tab of the Amazon S3 pricing page](#).

- ☒ **Server-side encryption with Amazon S3 managed keys (SSE-S3)**
- ☐ Server-side encryption with AWS Key Management Service keys (SSE-KMS)
- ☐ Dual-layer server-side encryption with AWS Key Management Service keys (DSSE-KMS)

Bucket Key

Using an S3 Bucket Key for SSE-KMS reduces encryption costs by lowering calls to AWS KMS. S3 Bucket Keys aren't supported for DSSE-KMS. [Learn more](#)

- ☐ Disable
- ☒ Enable

▶ Advanced settings

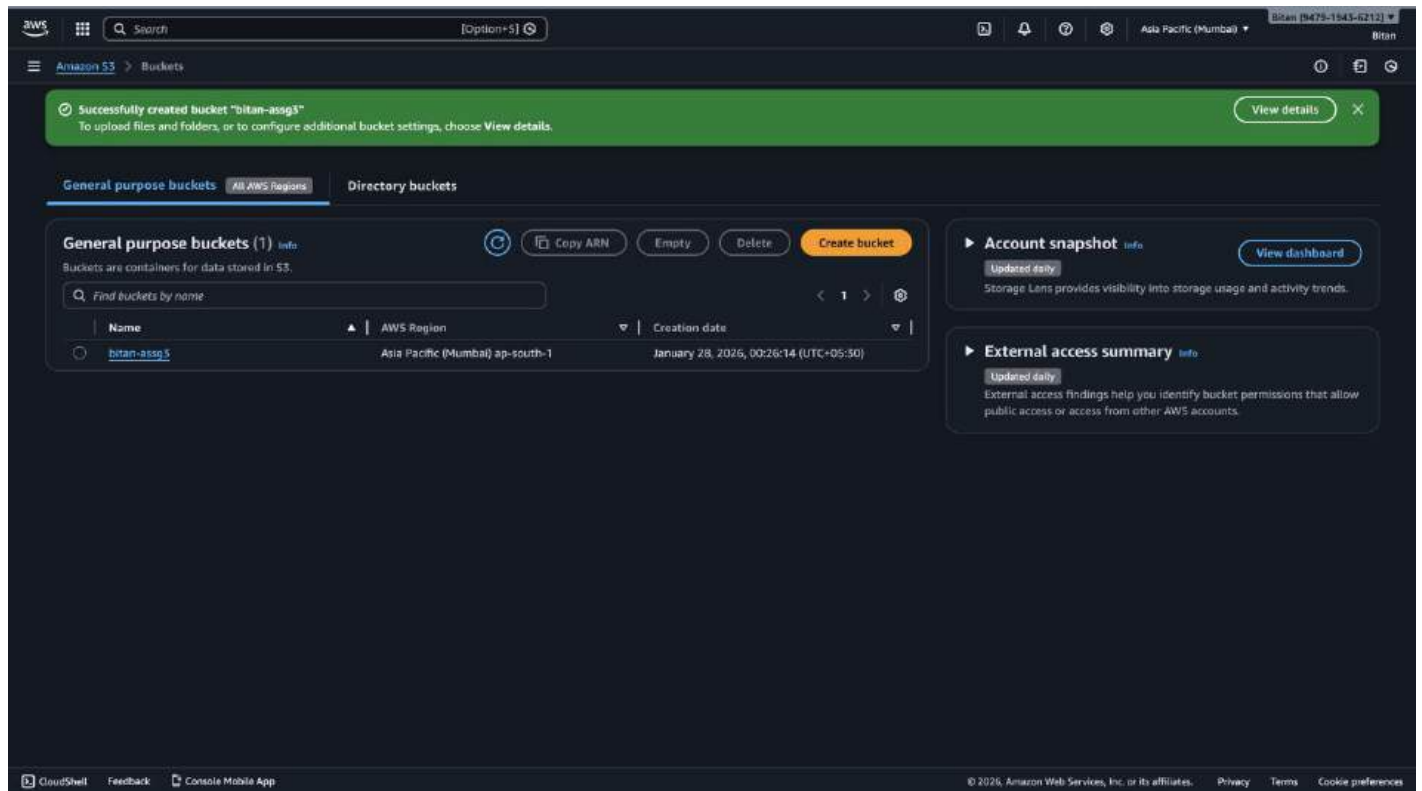
! After creating the bucket, you can upload files and folders to the bucket, and configure additional bucket settings.

[Cancel](#) [Create bucket](#)

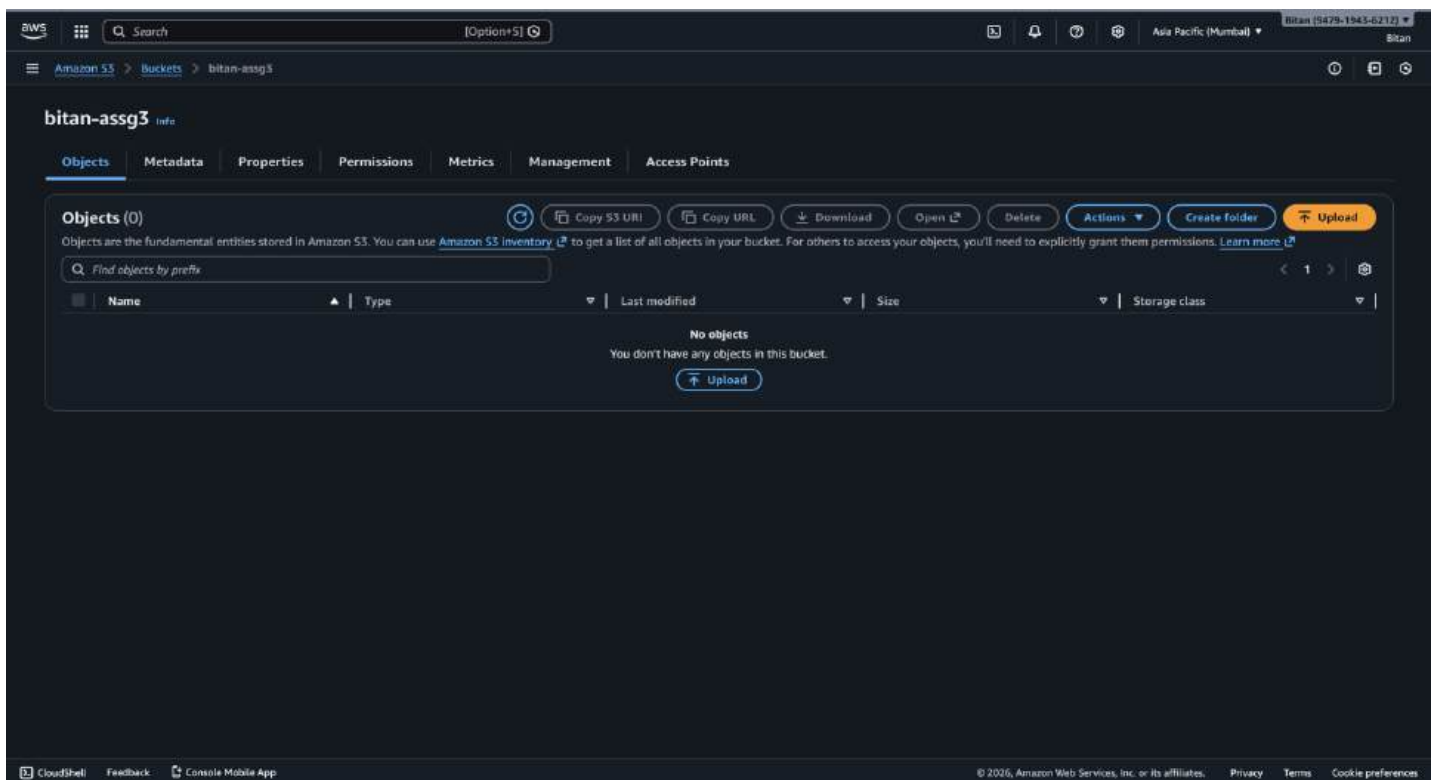
[CloudShell](#) [Feedback](#) [Console Mobile App](#)

© 2025, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

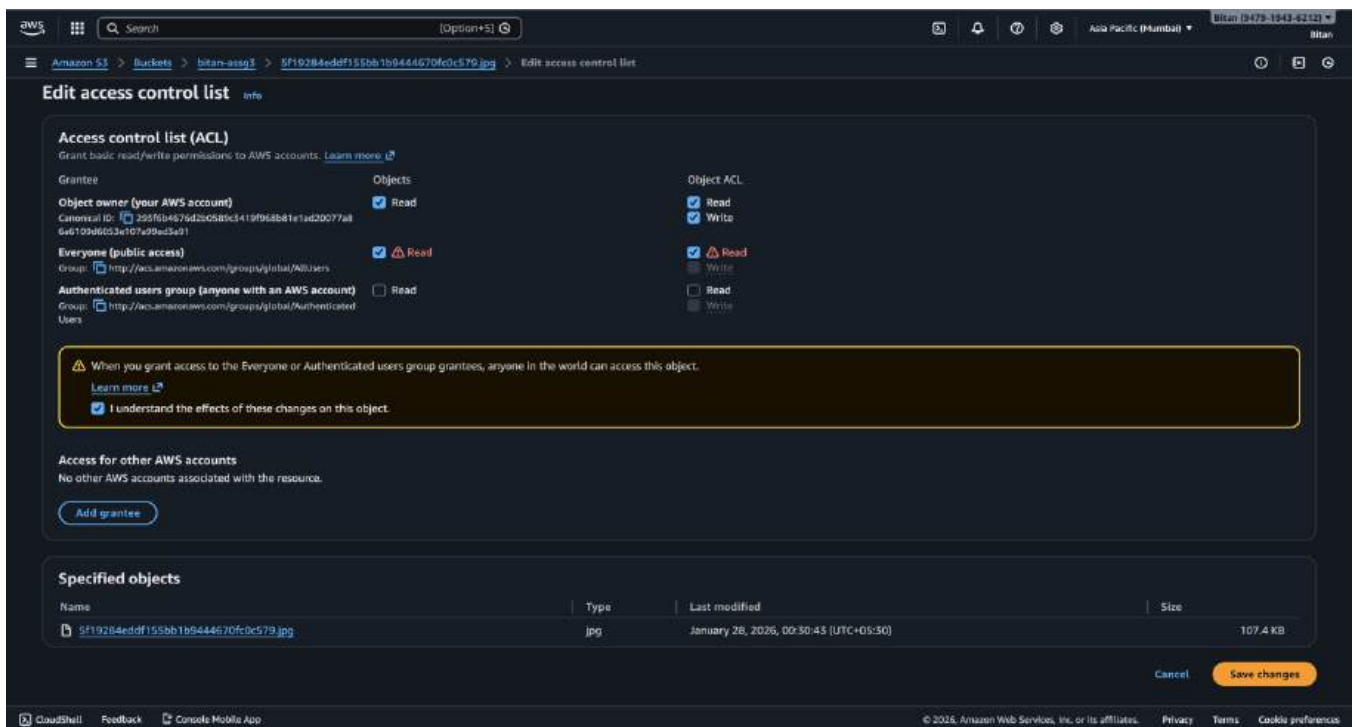
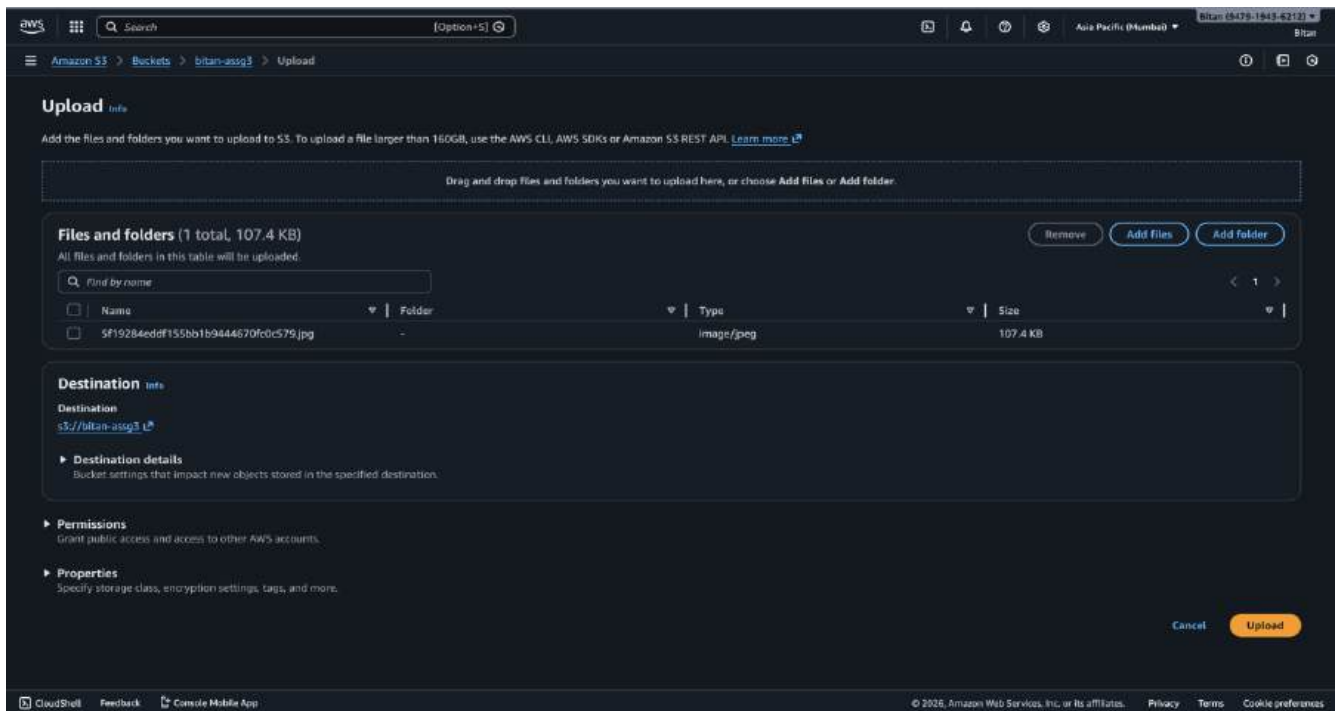
Step 3: A new bucket will be created. Click on the newly created bucket



Step 4: Within the bucket, click **Upload**.



Step 5: On the **upload** page, click Add files and select the file(s) to upload. Under **Permissions** select **Specify individual ACL permissions**. Then Grant Public Access. Check the Read permission boxes for Object and Object's ACL under **Everyone (public access)**. Tick the acknowledgment box confirming you understand the changes. Click **Upload** to complete the process.



Step 6: Go to the Properties tab of the uploaded file. Copy the Object URL. Paste the URL into a another web browser or incognito window. The file should be publicly accessible.

aws

Search

[Optional]

Assets

Alerts

Settings

Help

Asia Pacific (Mumbai)

Bitan (5479-1543-6212)

Amazon S3> Buckets> bitan-assg3> 5f19284eddf155bb1b9444670fc0c579.jpg

Successfully edited access control list for object "5f19284eddf155bb1b9444670fc0c579.jpg".

5f19284eddf155bb1b9444670fc0c579.jpg

Copy S3 URIDownloadOpenObject actions

PropertiesPermissionsVersions

Object overview

Owner

293f6b4676d2b0589c3419f968b81e1ad20077a86a6109d6053e107e99ad3a91

AWS Region

Asia Pacific (Mumbai) ap-south-1

Last modified

January 28, 2026, 00:30:43 (UTC+05:30)

Size

107.4 KB

Type

jpg

Key

5f19284eddf155bb1b9444670fc0c579.jpg

S3 URI

s3://bitan-assg3/5f19284eddf155bb1b9444670fc0c579.jpg

Amazon Resource Name (ARN)

arn:aws:s3:::bitan-assg3/5f19284eddf155bb1b9444670fc0c579.jpg

Entity tag (ETag)

De564b2236bcf7cb4ab81e

Object URL Copied

https://bitan-assg3.s3.ap-south-1.amazonaws.com/5f19284eddf155bb1b9444670fc0c579.jpg

Object management overview

The following bucket properties and object management configurations impact the behavior of this object.

Bucket properties

Bucket Versioning

When enabled, multiple variants of an object can be stored in the bucket to easily recover from unintended user actions and application failures.

Disabled

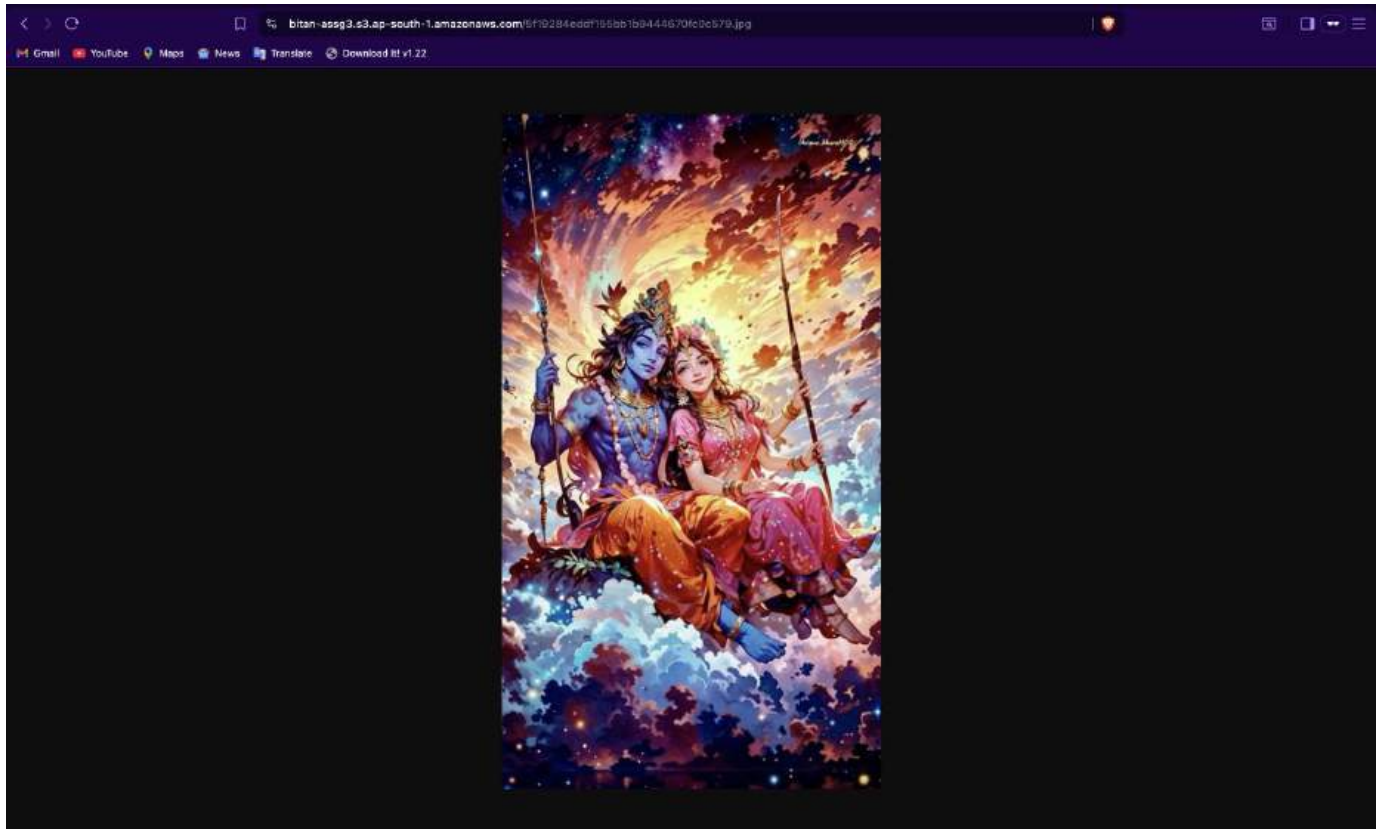
Management configurations

Replication status

When a replication rule is applied to an object the replication status indicates the progress of the operation.

-

View replication rules



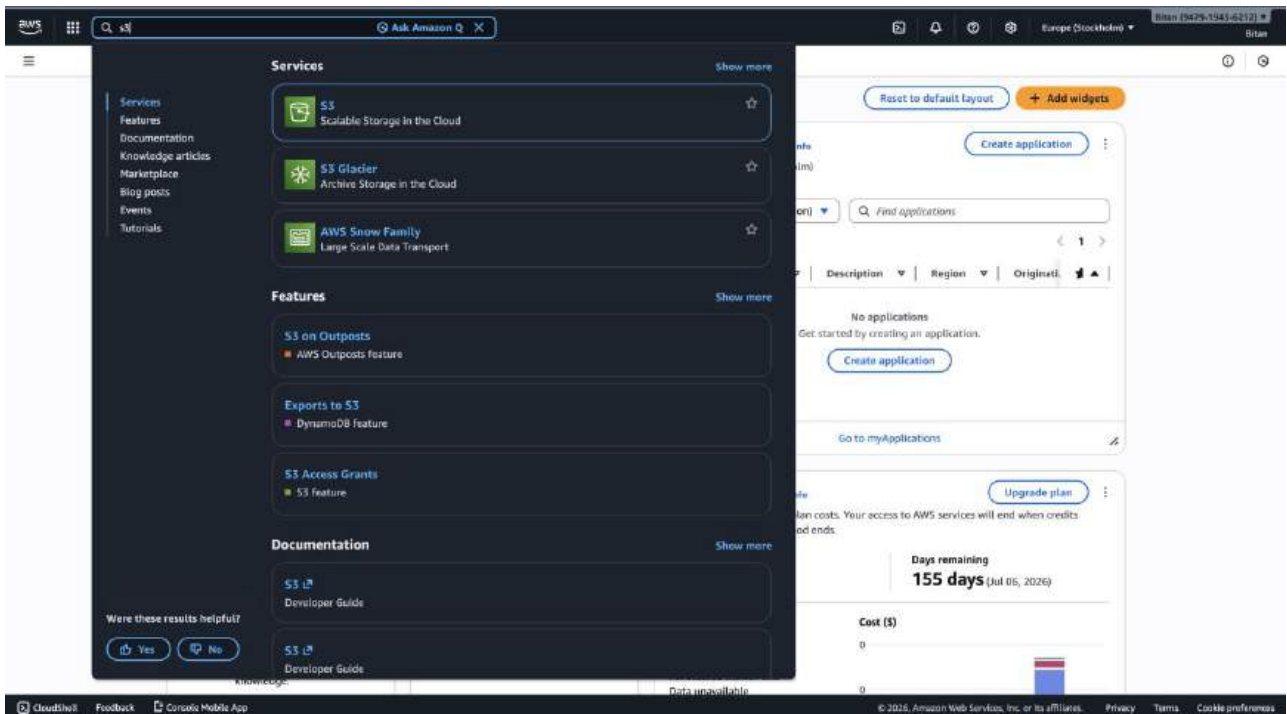
Assignment No.:5

Problem Statement: Create a private bucket in AWS. Upload a file and check by reassigned URL whether you can access the file or not.

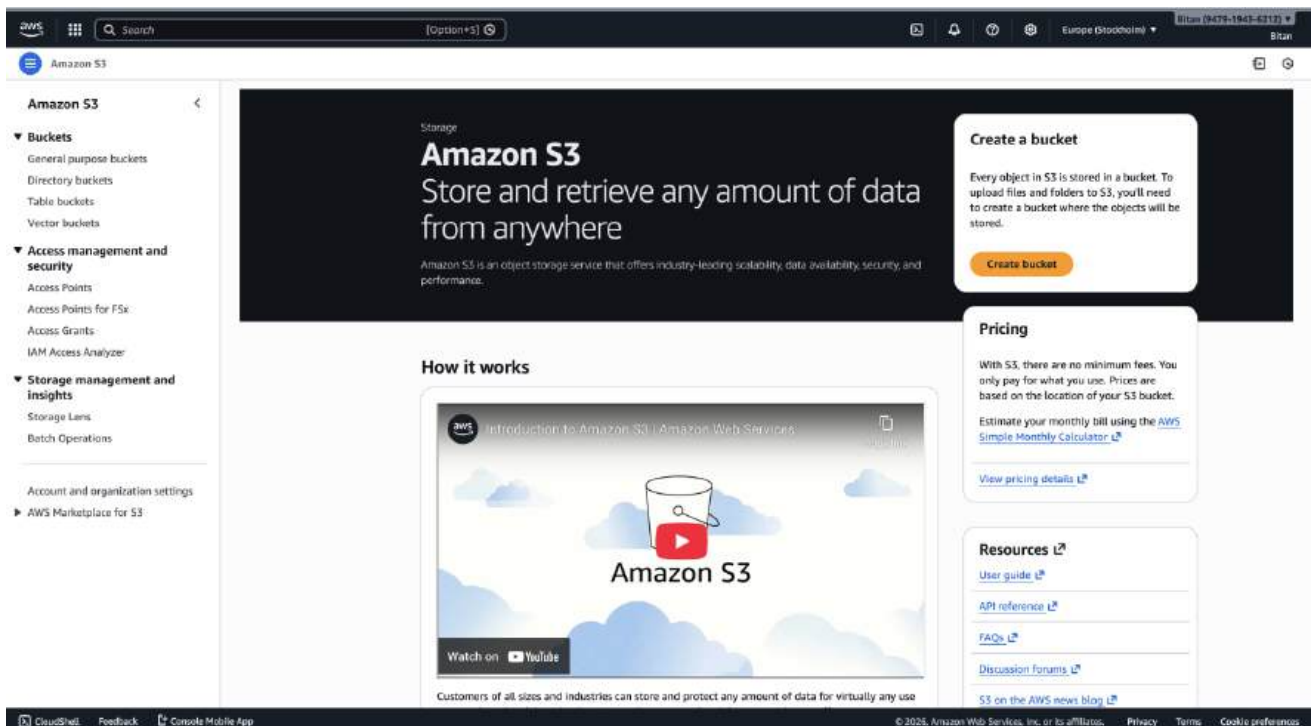
Procedure:

The steps are as follows: -

1. Access the AWS console, search for S3, and select the top option from the search results.



2. Click on 'Create bucket'.



3. Subsequent window appears. From the drop-down menu, select 'Asia Pacific (Mumbai) ap-south-1' as the AWS Region. Next, specify a bucket name , such as 'bk.mckv'.

The screenshot shows the AWS 'Create bucket' console. The 'AWS Region' is set to 'Asia Pacific (Mumbai) ap-south-1'. A dropdown menu is open, showing a list of regions. The 'Bucket type' is 'General purpose'. The 'Bucket name' is 'amazon-s3-demo-bucket'. The 'Object Ownership' is 'Bucket owner enforced'.

Region	Availability Zone
United States	
N. Virginia	us-east-1
Ohio	us-east-2
N. California	us-west-1
Oregon	us-west-2
Asia Pacific	
Hyderabad	ap-south-2
Mumbai	ap-south-1
Osaka	ap-northeast-3
Seoul	ap-northeast-2
Singapore	ap-southeast-1
Sydney	ap-southeast-2
Tokyo	ap-northeast-1
Canada	
Central	ca-central-1
Europe	
Frankfurt	eu-central-1
Ireland	eu-west-1
London	eu-west-2
Paris	eu-west-3
Stockholm	eu-north-1
South America	
São Paulo	sa-east-1

There are 16 Regions that are not enabled for this account.

Manage Regions Manage Local Zones

The screenshot shows the AWS 'Create bucket' console. The 'AWS Region' is set to 'Asia Pacific (Mumbai) ap-south-1'. The 'Bucket type' is 'General purpose'. The 'Bucket name' is 'bk.mckv'. The 'Object Ownership' is 'Bucket owner enforced'.

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn more](#)

Copy settings from existing bucket - optional

Only the bucket settings in the following configuration are copied.

Choose bucket

Format: s3://bucket/prefix

4. By default, the Object Ownership is set to '**ACLs disabled**' in this window. We will not make any adjustments as we are creating a private bucket. Similarly, we will leave the settings unchanged as the '**Block all public access**' checkbox is already selected by default, aligning with our intention to create a private bucket.

The screenshot shows the 'Create bucket' page in the AWS Management Console. The 'Object Ownership' section has 'ACLs disabled (recommended)' selected. The 'Block Public Access settings for this bucket' section has 'Block all public access' checked, with four sub-settings also checked. The 'Bucket Versioning' section has 'Disable' selected.

Object Ownership [info](#)
Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

Object Ownership

☒ **ACLs disabled (recommended)**
All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.

☐ **ACLs enabled**
Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.

Object Ownership
Bucket owner enforced

Block Public Access settings for this bucket
Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

☒ **Block all public access**
Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

- ☒ **Block public access to buckets and objects granted through new access control lists (ACLs)**
S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.
- ☒ **Block public access to buckets and objects granted through any access control lists (ACLs)**
S3 will ignore all ACLs that grant public access to buckets and objects.
- ☒ **Block public access to buckets and objects granted through new public bucket or access point policies**
S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.
- ☒ **Block public and cross-account access to buckets and objects through any public bucket or access point policies**
S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

Bucket Versioning
Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. [Learn more](#)

Bucket Versioning

☒ **Disable**

☐ **Enable**

6. Scroll down without any further modifications and proceed to click on 'Create Bucket'.

The screenshot shows the 'Create bucket' page in the AWS Management Console, scrolled down to show 'Tags - optional', 'Default encryption', and 'Advanced settings'. The 'Tags' section has a note about API permissions. The 'Default encryption' section has 'Server-side encryption with Amazon S3 managed keys (SSE-S3)' selected. The 'Advanced settings' section has a note about uploading files and folders.

Tags - optional [info](#)
You can use bucket tags to analyze, manage and specify permissions for a bucket. [Learn more](#)

☐ You can use s3:ListTagsForResource, s3:TagResource, and s3:UntagResource APIs to manage tags on S3 general purpose buckets for access control in addition to cost allocation and resource organization. To ensure a seamless transition, please provide permissions to s3:ListTagsForResource, s3:TagResource, and s3:UntagResource actions. [Learn more](#)

No tags associated with this bucket.

[Add new tag](#)

You can add up to 50 tags.

Default encryption [info](#)
Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption type [info](#)
Secure your objects with two separate layers of encryption. For details on pricing, see DSE-KMS pricing on the Storage tab of the [Amazon S3 pricing page](#).

☒ **Server-side encryption with Amazon S3 managed keys (SSE-S3)**

☐ **Server-side encryption with AWS Key Management Service keys (SSE-KMS)**

☐ **Dual-layer server-side encryption with AWS Key Management Service keys (DSSE-KMS)**

Bucket Key
Using an S3 Bucket Key for SSE-KMS reduces encryption costs by lowering calls to AWS KMS. S3 Bucket Keys aren't supported for DSSE-KMS. [Learn more](#)

☐ **Disable**

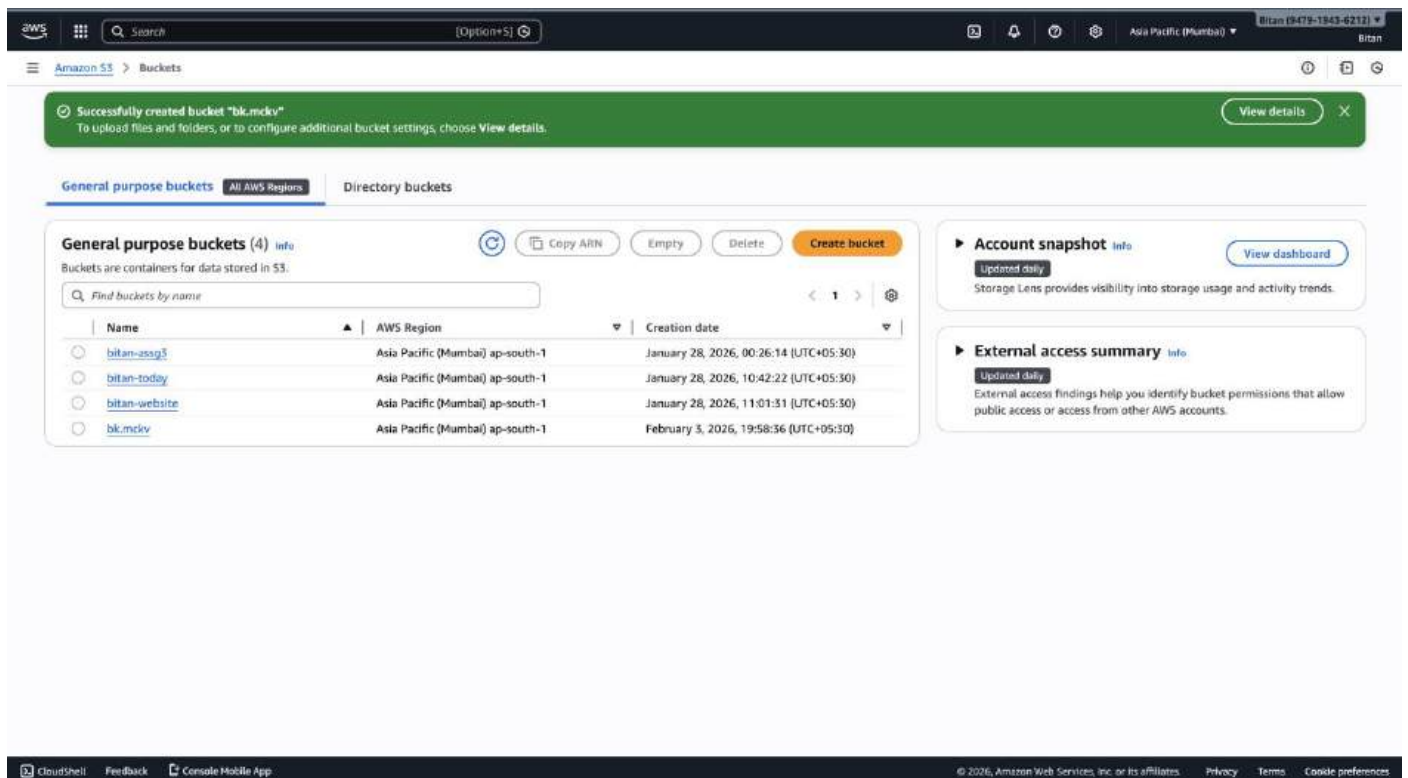
☒ **Enable**

Advanced settings

☐ After creating the bucket, you can upload files and folders to the bucket, and configure additional bucket settings.

[Cancel](#) [Create bucket](#)

7. It's evident that the bucket has been successfully created.



Successfully created bucket "blk.mckiv"
To upload files and folders, or to configure additional bucket settings, choose [View details](#).

General purpose buckets (4) [info](#)

Buckets are containers for data stored in S3.

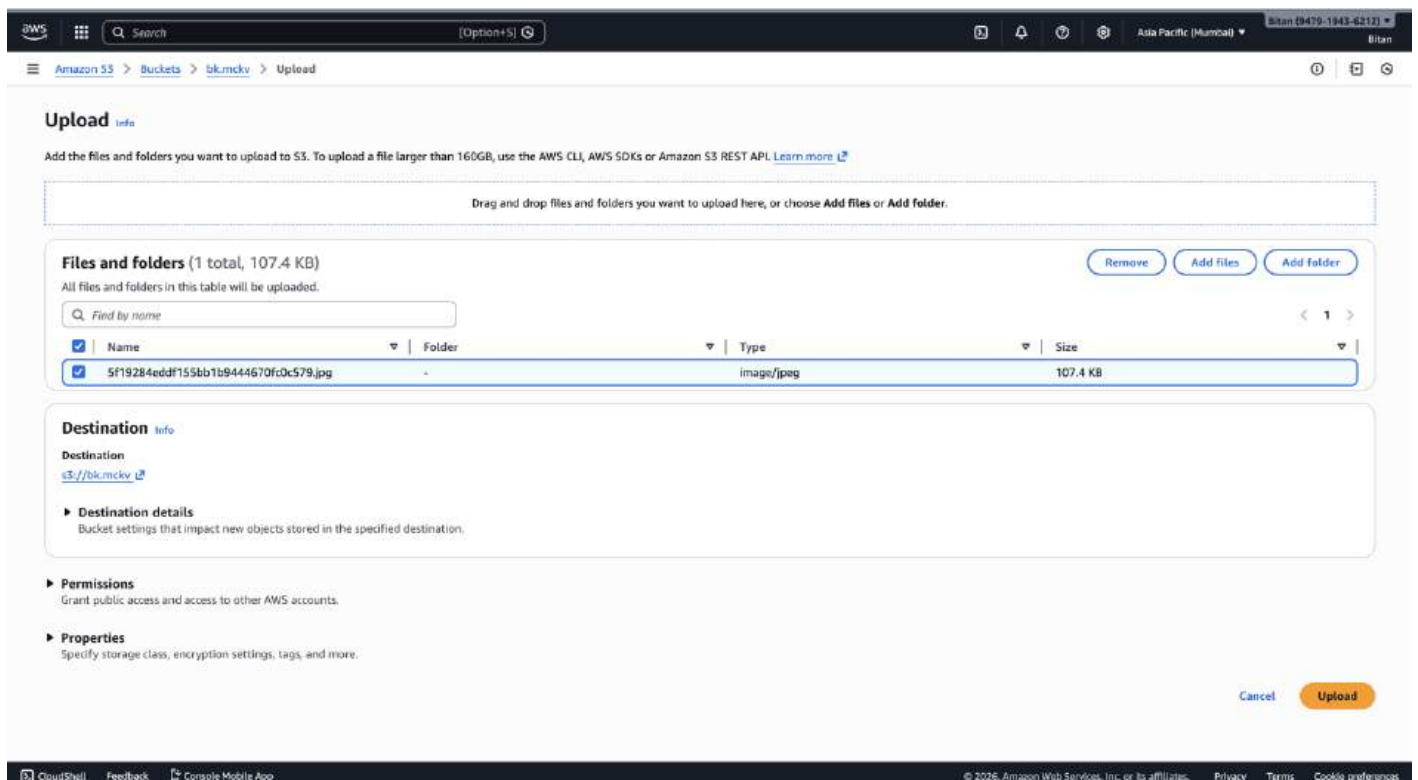
Find buckets by name

Name	AWS Region	Creation date
bitan-ssg3	Asia Pacific (Mumbai) ap-south-1	January 28, 2026, 00:26:14 (UTC+05:30)
bitan-today	Asia Pacific (Mumbai) ap-south-1	January 28, 2026, 10:42:22 (UTC+05:30)
bitan-website	Asia Pacific (Mumbai) ap-south-1	January 28, 2026, 11:01:31 (UTC+05:30)
blk.mckiv	Asia Pacific (Mumbai) ap-south-1	February 3, 2026, 19:58:36 (UTC+05:30)

Account snapshot [info](#)
Updated daily
Storage Lens provides visibility into storage usage and activity trends.

External access summary [info](#)
Updated daily
External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.

8. Now click on the bucket name. Choose the '**Upload**' option, which will lead to the opening of the subsequent window. From there, click on '**Add files**' and proceed to upload your desired file(s) into the selected bucket.



Upload [info](#)

Add the files and folders you want to upload to S3. To upload a file larger than 160GB, use the AWS CLI, AWS SDKs or Amazon S3 REST API. [Learn more](#)

Drag and drop files and folders you want to upload here, or choose [Add files](#) or [Add folder](#).

Files and folders (1 total, 107.4 KB)

All files and folders in this table will be uploaded.

Find by name

Name	Folder	Type	Size
5f19284eddf155bb1b9444670fc0c579.jpg	-	image/jpeg	107.4 KB

Destination [info](#)

Destination
[s3://blk.mckiv](#)

Destination details
Bucket settings that impact new objects stored in the specified destination.

Permissions
Grant public access and access to other AWS accounts.

Properties
Specify storage class, encryption settings, tags, and more.

[Cancel](#) [Upload](#)

9. Scroll down and proceed to click on 'Upload'. The notification confirms that the file has been successfully uploaded.

The screenshot shows the Amazon S3 console interface. At the top, a green notification bar states "Upload succeeded" with a link to the "Files and folders table". Below this, the "Upload: status" page is displayed. A blue box indicates that information is no longer available after navigating away. The "Summary" section shows the destination as "s3://bk.mckv", with 1 file (107.4 KB) succeeded and 0 files (0 B) failed. The "Files and folders" tab is active, showing a table with one file: "5f19284eddf155bb1b9444670fc0c579.jpg" (107.4 KB, image/jpeg, Succeeded).

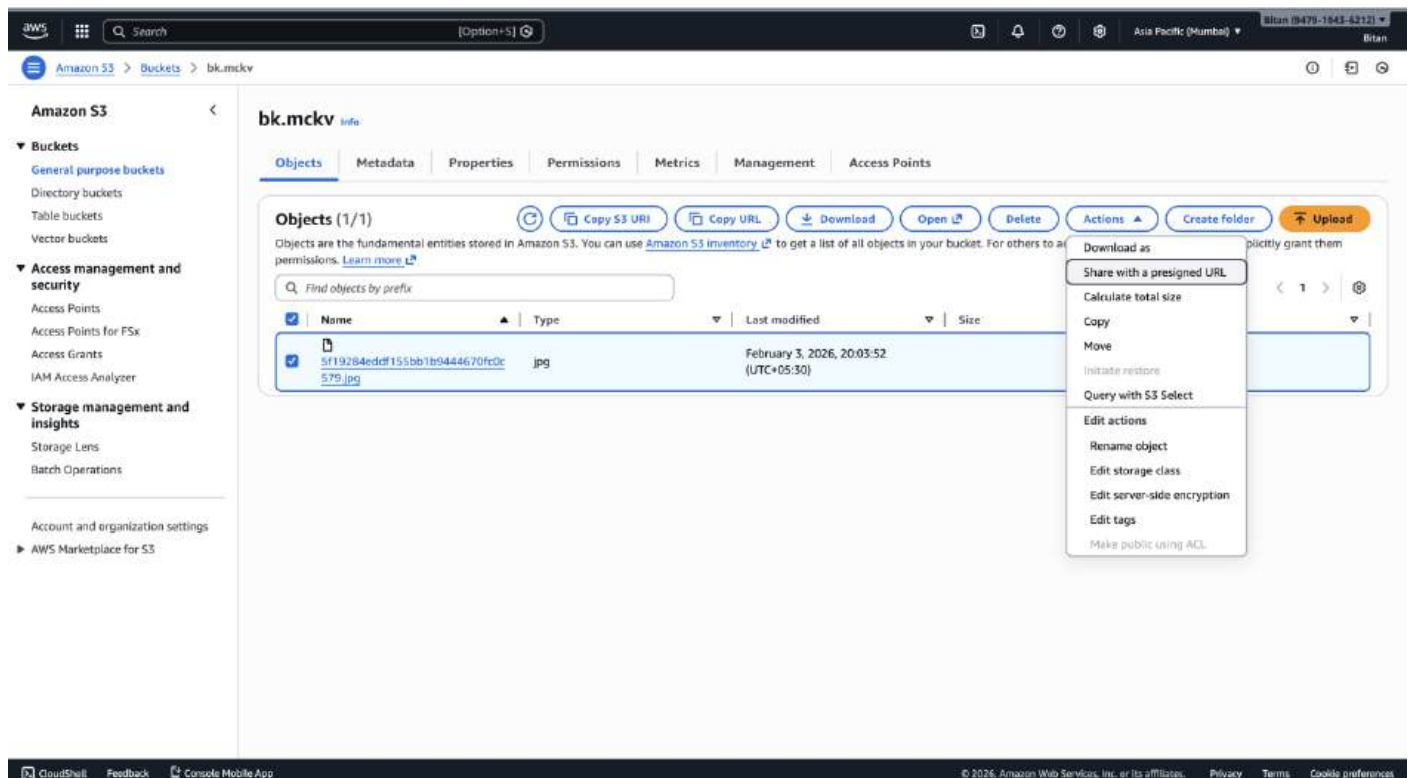
10. Now, click on one of the uploaded file, then 'Copy URL'.

The screenshot shows the Amazon S3 console interface with the details of a specific file. The file name is "5f19284eddf155bb1b9444670fc0c579.jpg". The "Object overview" section displays the owner, AWS Region (Asia Pacific (Mumbai) ap-south-1), last modified date (February 3, 2026, 20:03:52 UTC+05:30), size (107.4 KB), type (jpg), and key. The "Object management overview" section shows bucket properties, including that Bucket Versioning is disabled. The "Management configurations" section shows the replication status. The "Copy URL" button is highlighted, and a tooltip indicates "Object URL Copied". The copied URL is "https://s3-ap-south-1.amazonaws.com/bk.mckv/5f19284eddf155bb1b9444670fc0c579.jpg".

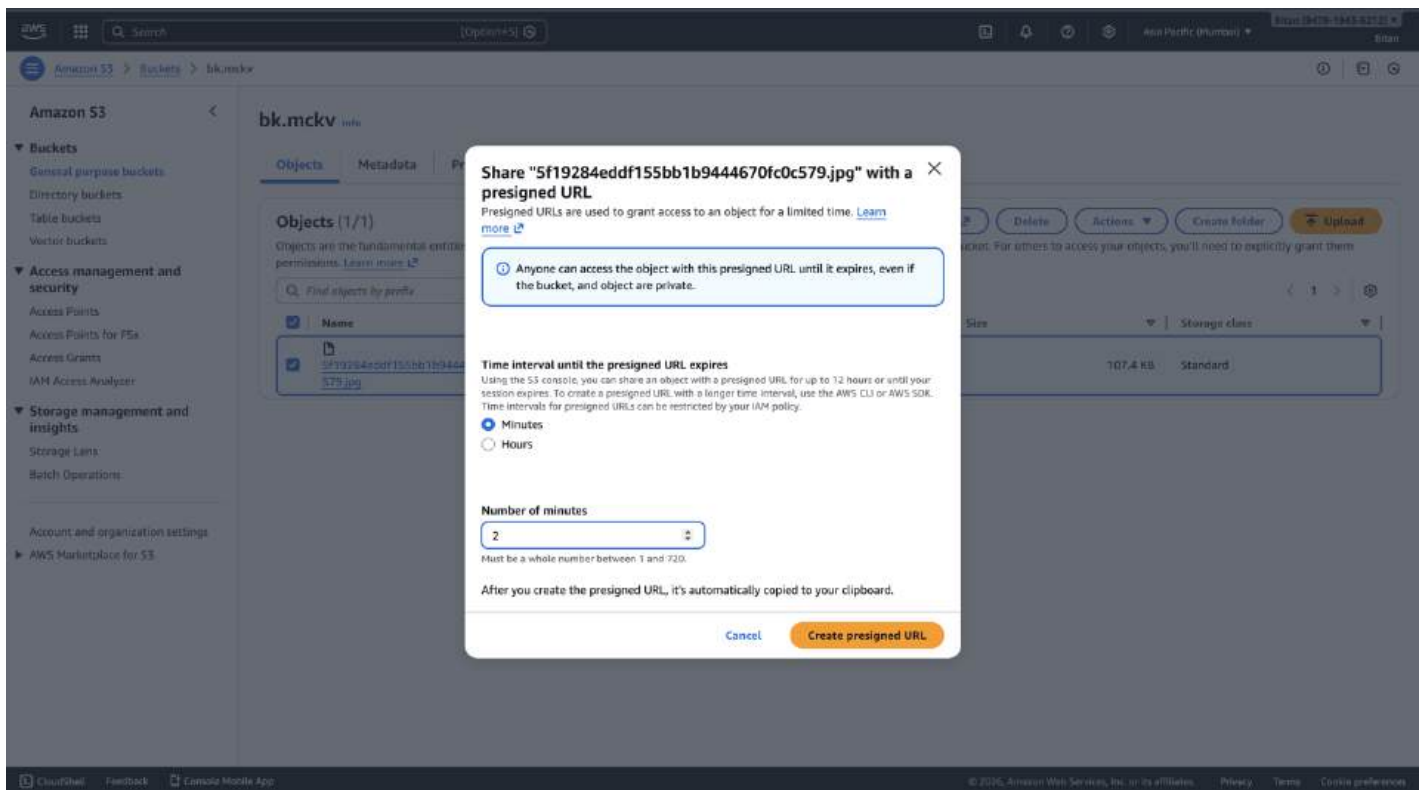
11. Navigate on a new window and paste this URL. An error appears indicating that access to the contents is restricted.



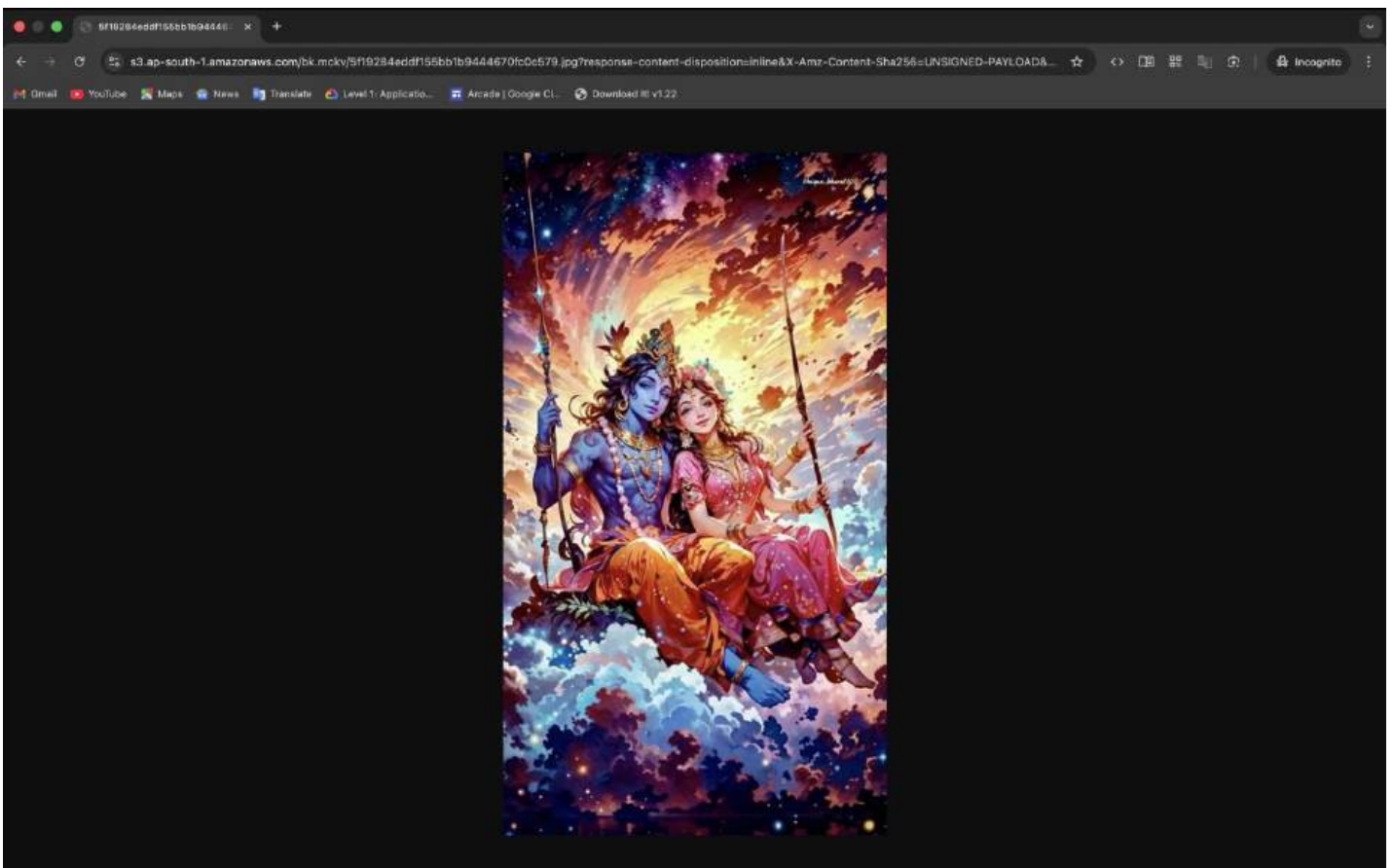
12. Close the current window, then navigate to 'Actions' and choose 'Share with a presigned URL'.



13. Select the desired duration until you want the presigned URL to expire.



14. Copy the pre-assigned URL and paste it on a new window. Now, you can view the uploaded file.



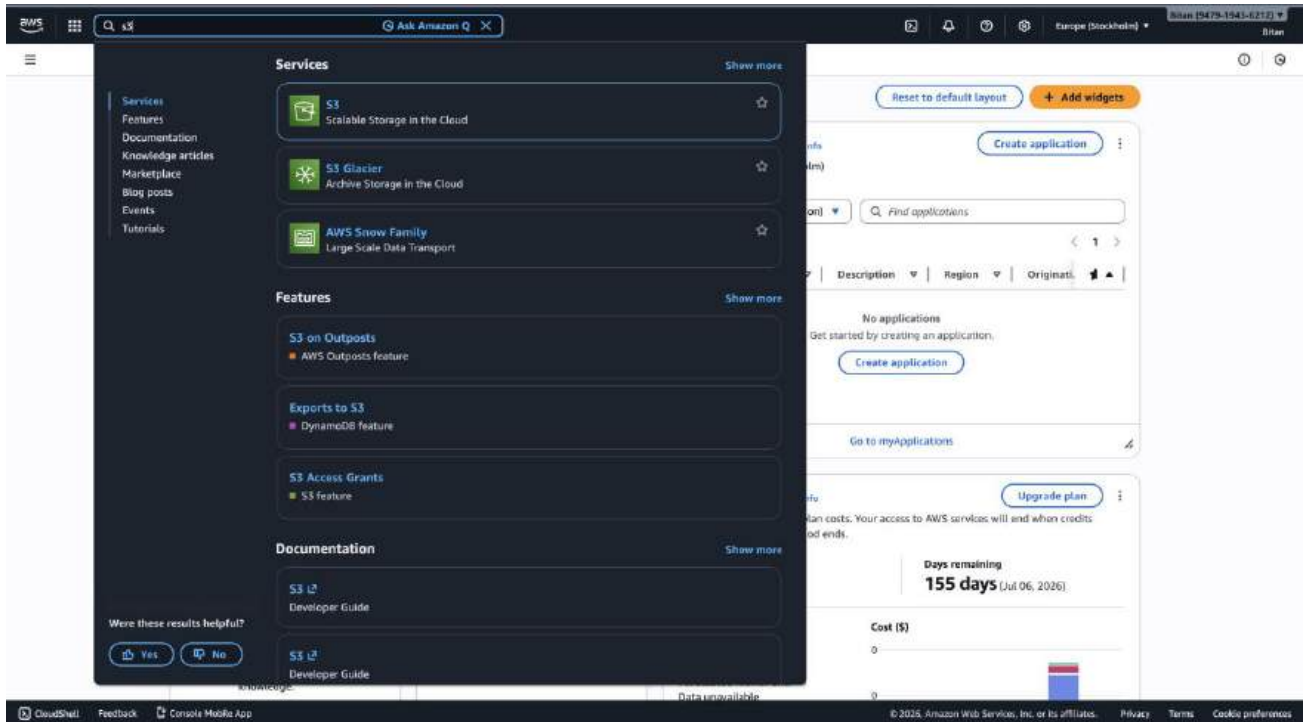
Assignment No.:6

Problem Statement: Upload a static website on S3.

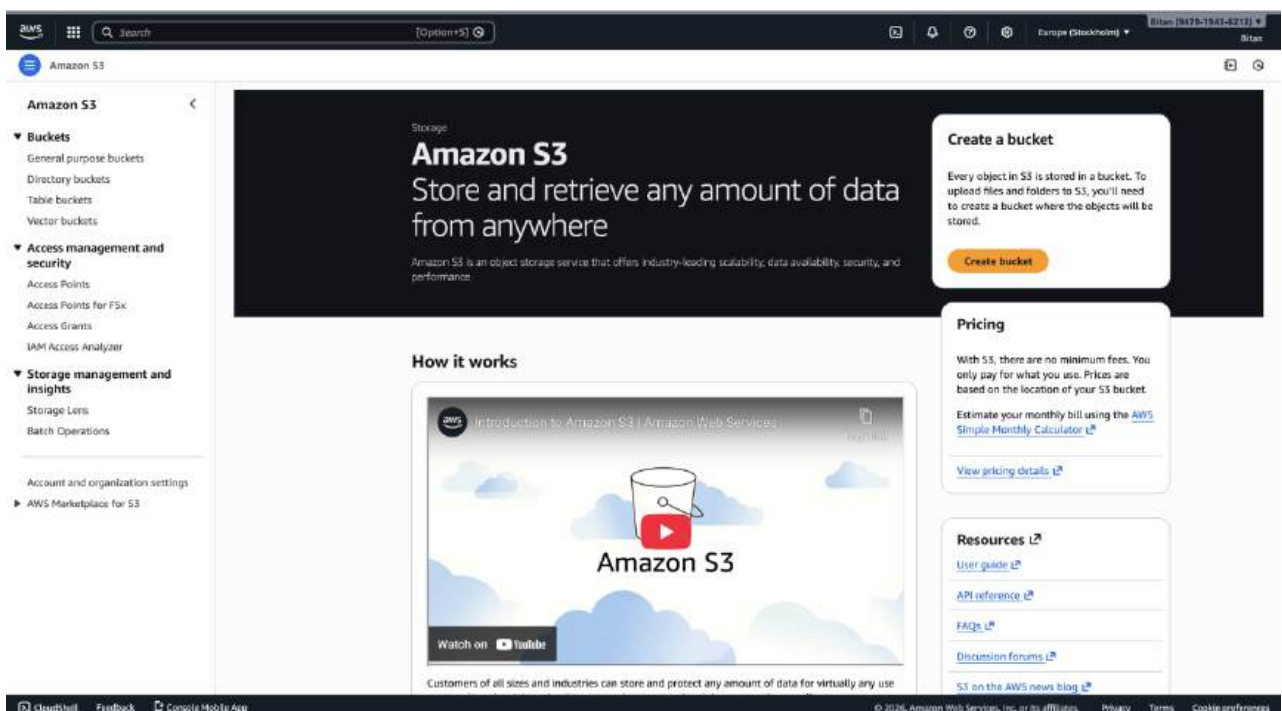
Procedure:

The steps are as follows: -

1. Access the AWS console, search for S3, and select the top option from the search results.



2. Click on 'Create bucket'.



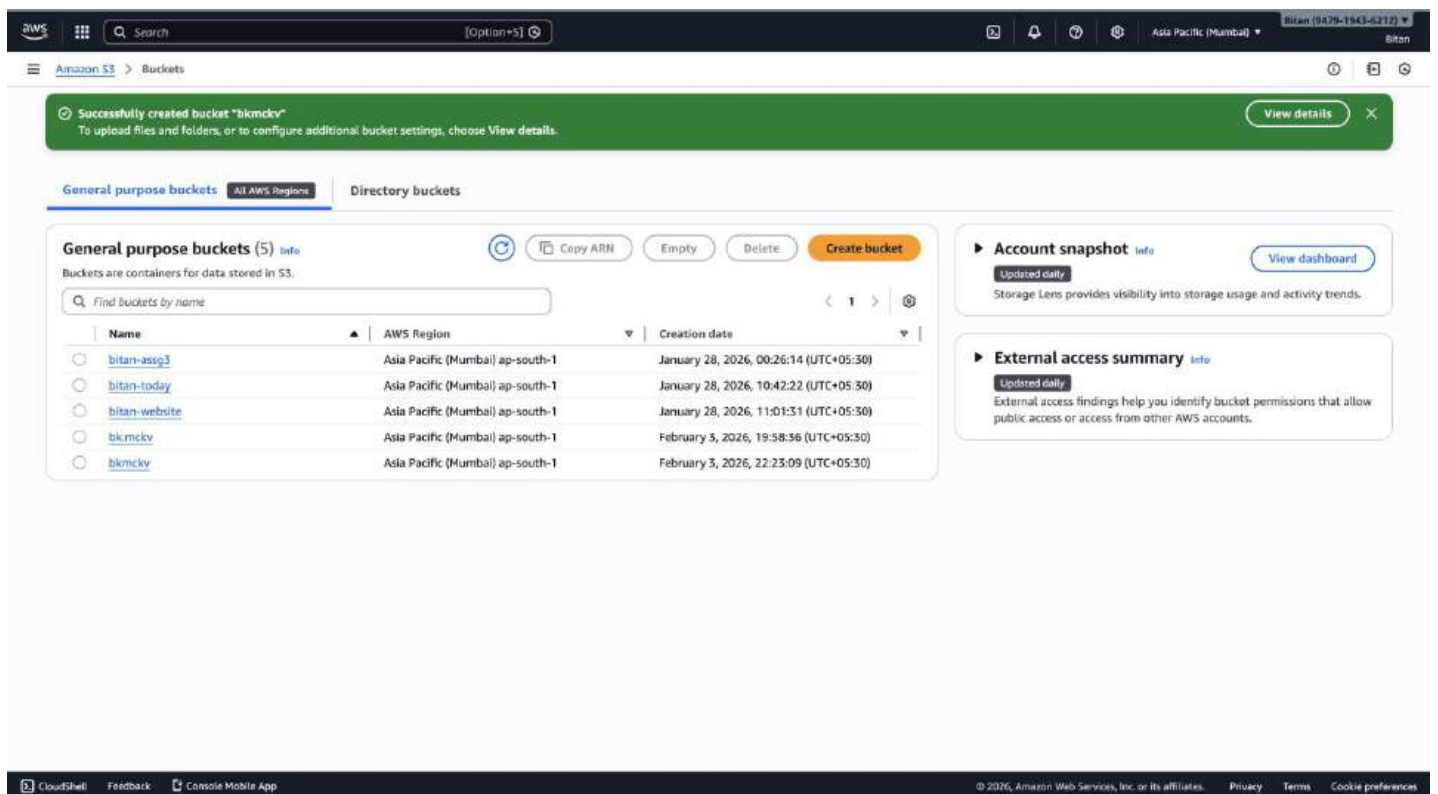
3. Fill up the required details: 'Aws Region', 'Bucket name'. Click on 'ACLs enabled', uncheck 'Block all public access', tick 'I acknowledge...' and then click on 'Create bucket'.

The screenshot shows the 'Create bucket' page in the AWS Management Console. The 'General configuration' section is active. Under 'AWS Region', 'Asia Pacific (Mumbai)' is selected. Under 'Bucket type', 'General purpose' is selected. The 'Bucket name' field contains 'benidiv'. The 'Copy settings from existing bucket - optional' section has a 'Choose bucket' button. In the 'Object Ownership' section, 'ACLs enabled' is selected. A warning box states: 'We recommend disabling ACLs, unless you need to control access for each object individually or to have the object writer own the data they upload. Using a bucket policy instead of ACLs to share data with users outside of your account simplifies permissions management and auditing.' The 'Object Ownership' dropdown is set to 'Bucket owner preferred'.

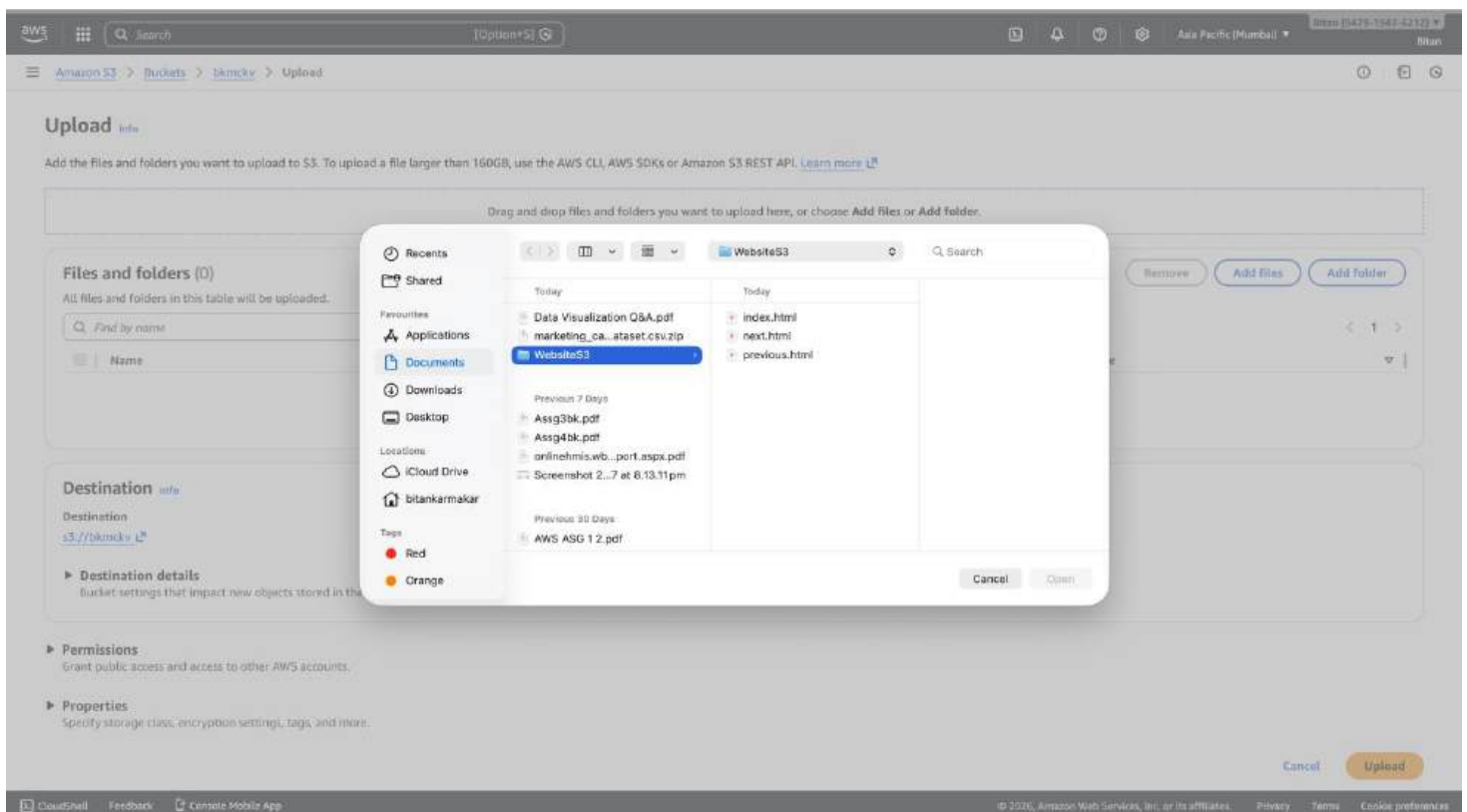
The screenshot shows the 'Block Public Access settings for this bucket' section. A warning box at the top states: 'If you want to enforce object ownership for new objects only, your bucket policy must specify that the bucket-owner-full-control canned ACL is required for object uploads.' Under 'Block all public access', the 'Block all public access' checkbox is checked. A warning box below states: 'Turning off block all public access might result in this bucket and the objects within becoming public. AWS recommends that you turn on block all public access, unless public access is required for specific and verified use cases such as static website hosting.' The 'I acknowledge that the current settings might result in this bucket and the objects within becoming public' checkbox is checked. The 'Bucket Versioning' section shows 'Bucket Versioning' set to 'Disable'.

The screenshot shows the 'Advanced settings' section. A warning box at the top states: 'You can use s3:ListTagForSource, s3:TagResource, and s3:UntagResource APIs to manage tags on S3 general purpose buckets for access control in addition to cost allocation and resource organization. To ensure a seamless transition, please provide permissions to s3:ListTagForSource, s3:TagResource, and s3:UntagResource actions.' Below this, there is a section for 'Default encryption' with 'Server-side encryption with Amazon S3 managed keys (SSE-S3)' selected. The 'Bucket Key' section shows 'Bucket Key' set to 'Disable'. At the bottom, there is a 'Create bucket' button.

4. It's evident that the bucket has been successfully created. Click on the bucket name.



5. Click on 'Upload' then choose the three html files and upload those.



6. We can see that the three html files are successfully uploaded.

Upload: status

After you navigate away from this page, the following information is no longer available.

Summary

Destination
s3://bkmckv

Succeeded
3 files, 831.0 B (100.00%)

Failed
0 files, 0 B (0%)

Files and folders | Configuration

Files and folders (3 total, 831.0 B)

Find by name

Name	Folder	Type	Size	Status	Error
index.html	-	text/html	307.0 B	Succeeded	-
next.html	-	text/html	256.0 B	Succeeded	-
previous.html	-	text/html	268.0 B	Succeeded	-

7. Now select all the three html files and click on 'Actions' Button and then 'Make Public Using ACL'. Then click on 'Make Public' option in the next screen.

bkmckv

Objects (3/3)

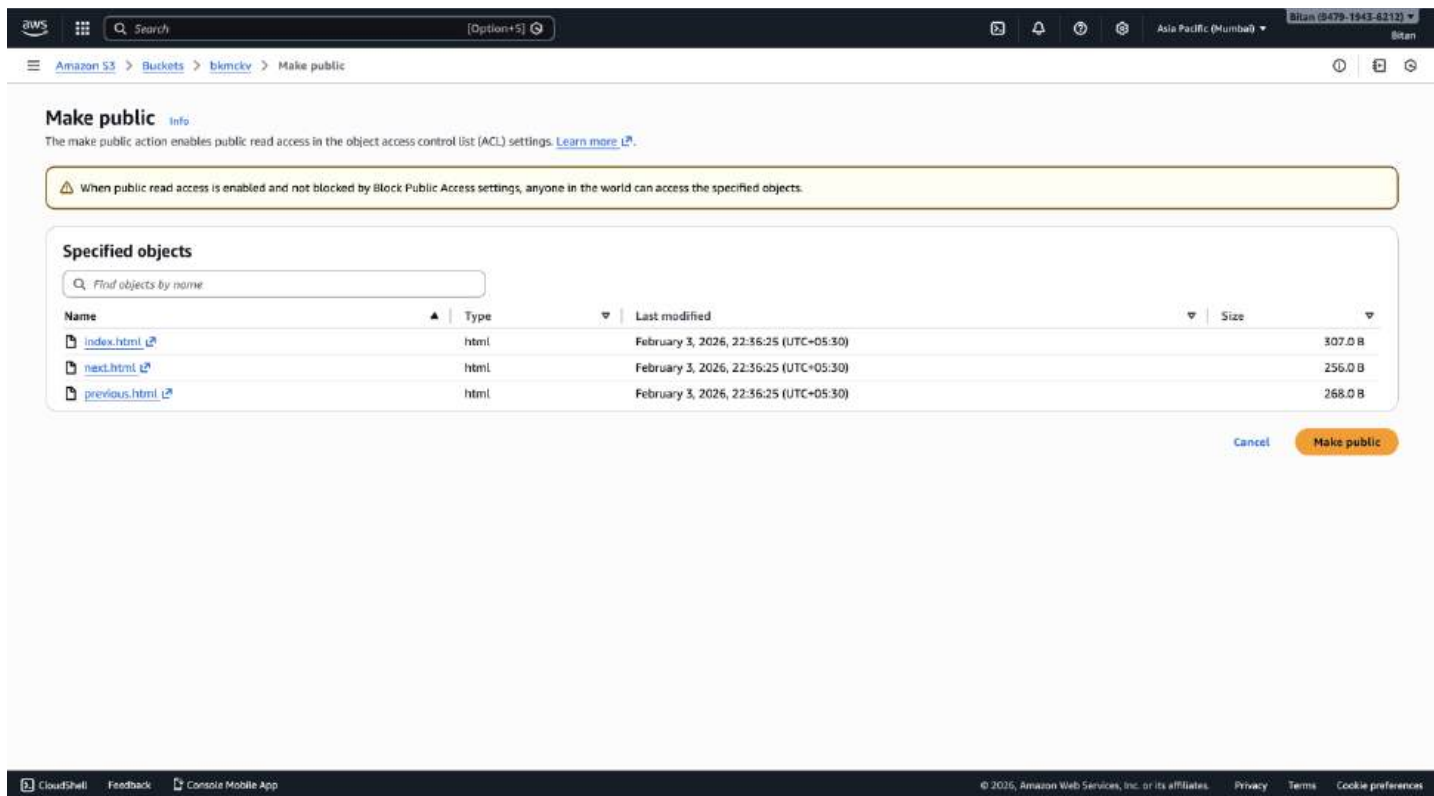
Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant permissions.

Find objects by prefix

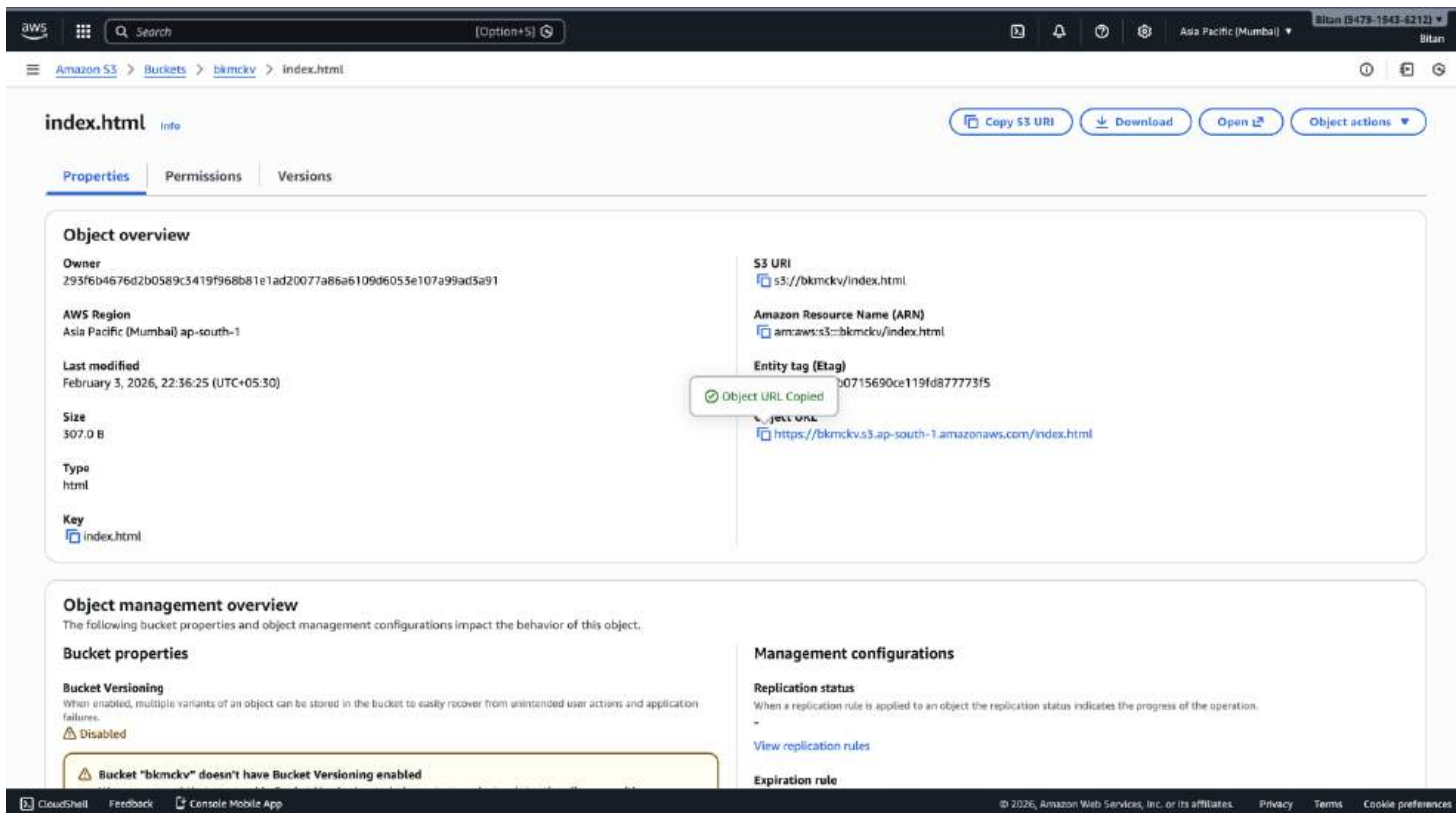
Name	Type	Last modified	Size
☒ index.html	html	February 3, 2026, 22:36:25 (UTC+05:30)	
☒ next.html	html	February 3, 2026, 22:36:25 (UTC+05:30)	
☒ previous.html	html	February 3, 2026, 22:36:25 (UTC+05:30)	

Actions

- Download as
- Share with a presigned URL
- Calculate total size
- Copy
- Move
- Initiate restore
- Query with S3 Select
- Edit actions**
 - Rename object
 - Edit storage class
 - Edit server-side encryption
 - Edit tags
 - Make public using ACL**



8. Now click on the main html file which is index.html here in this case. Then copy the URL of the index.html file.



9. Paste the URL copied in a new inprivate window. You will see the expected results as follows. Click on the hyperlink 'Go to the Next page'.



This is a Heading

This is a paragraph.

[Go to the Next Page Go to the Previous Page](#)

10. Go back to next page by clicking on the hyperlink.

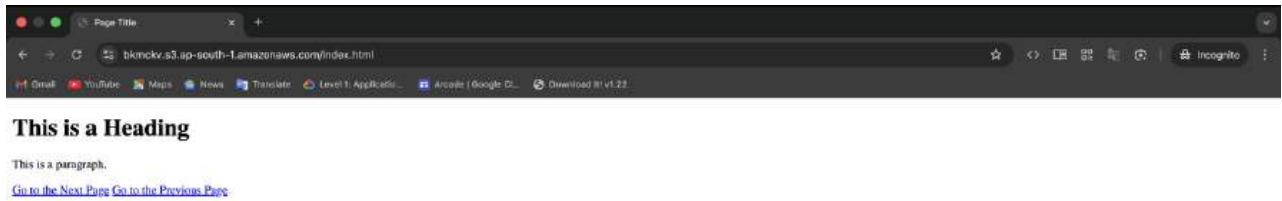


This is the Next Page

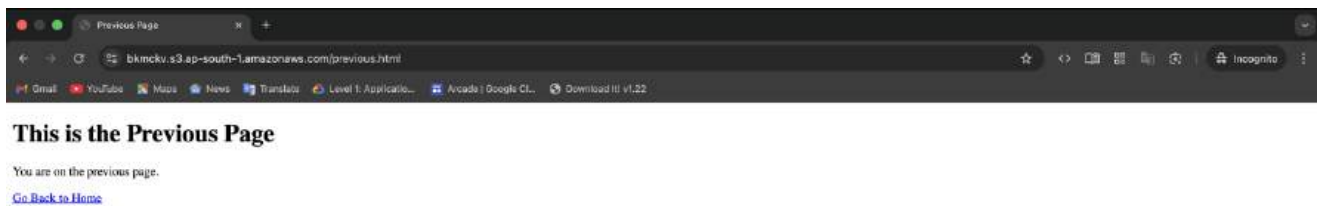
You are on the next page.

[Go Back to Home](#)

11. Now click on the next hyperlink .



12. Again go back to previous page by clicking on the hyperlink shown in the page.



13. So by above steps we have successfully uploaded a static website on S3.

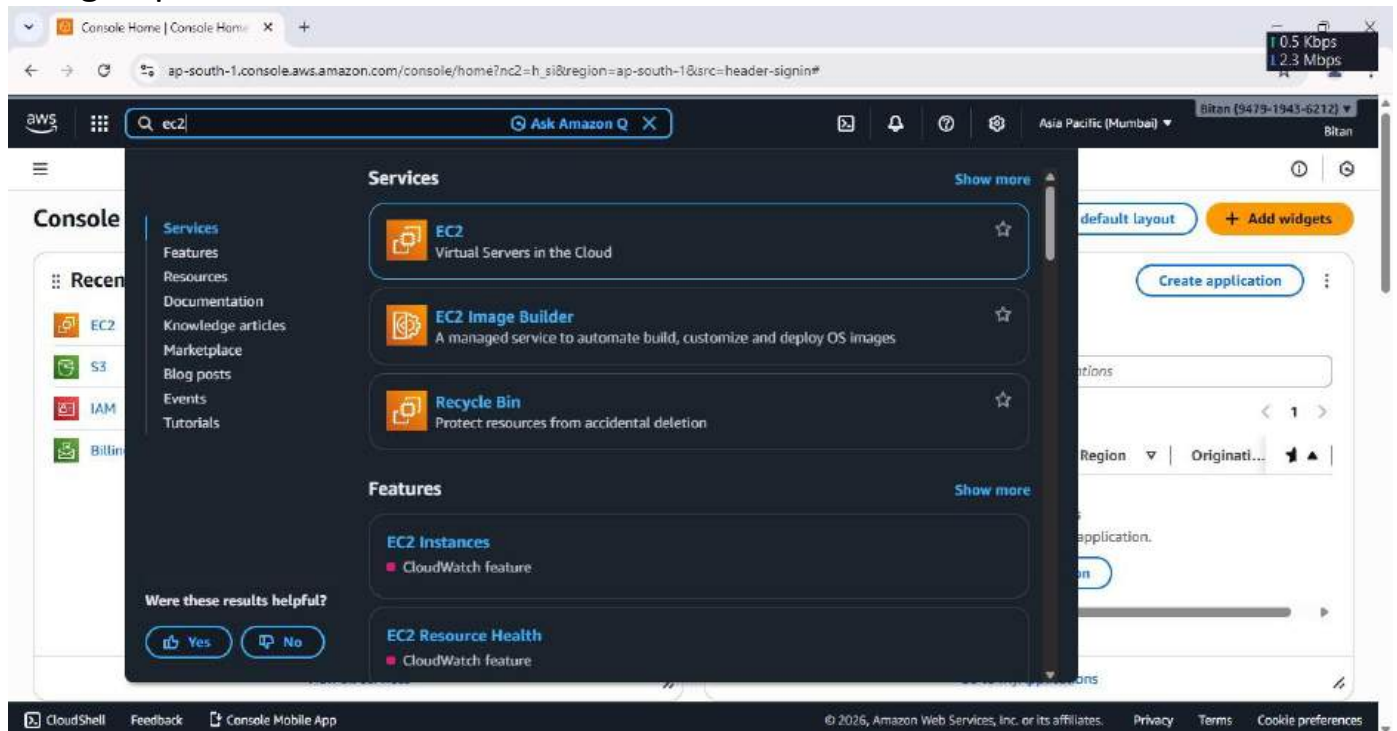
Assignment No.:7

Problem Statement: Hosting a website on EC2.

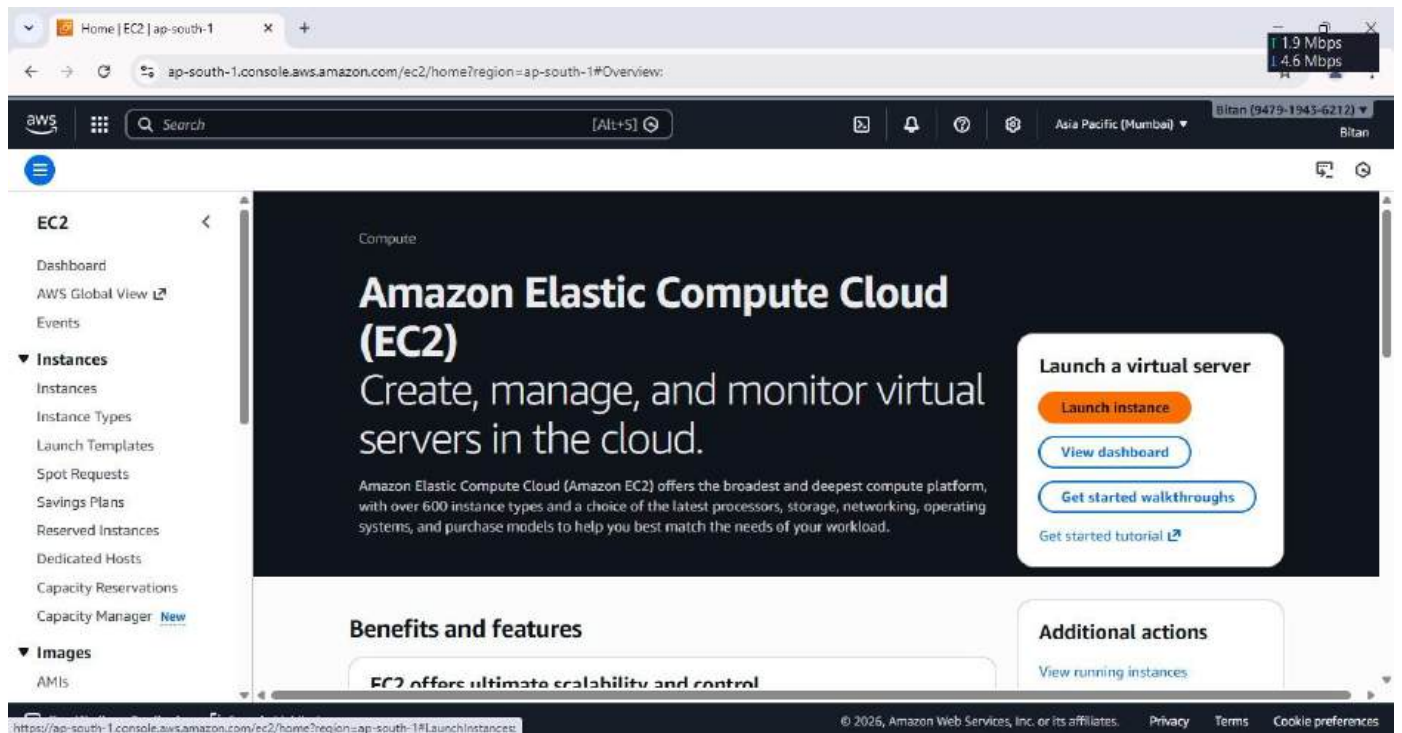
Procedure:

The steps are as follows: -

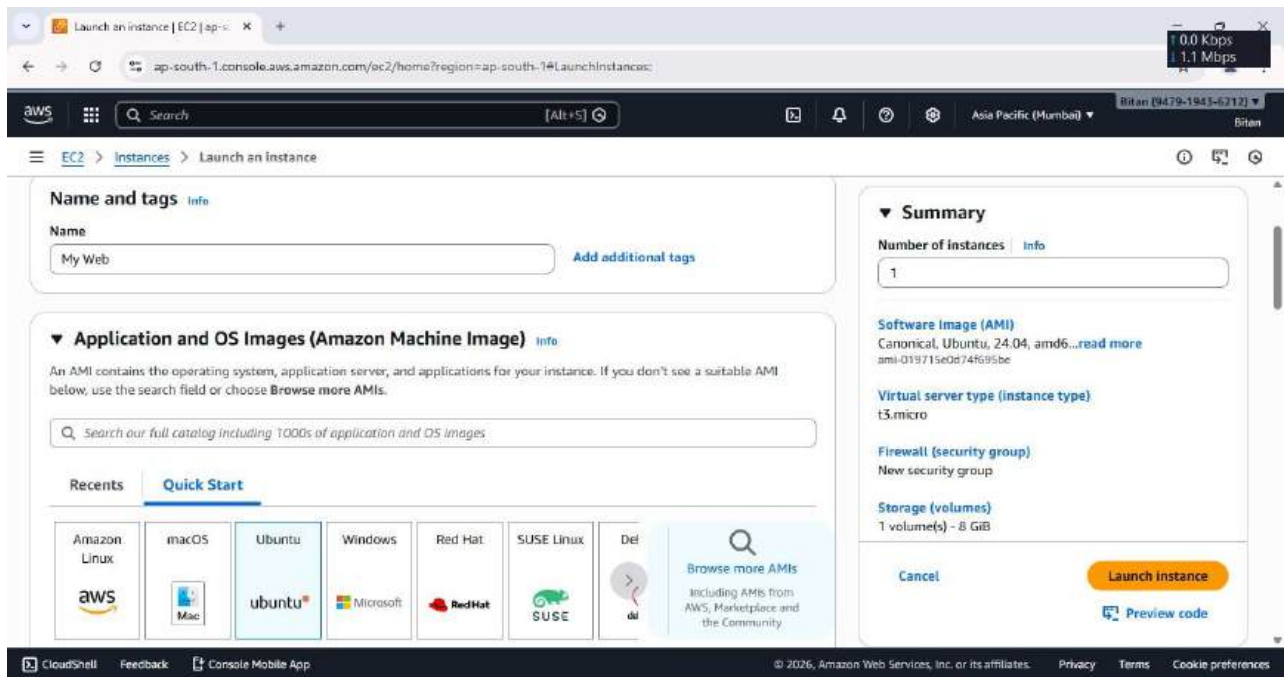
1. Sign up for an AWS account, search for 'EC2' then click on it.



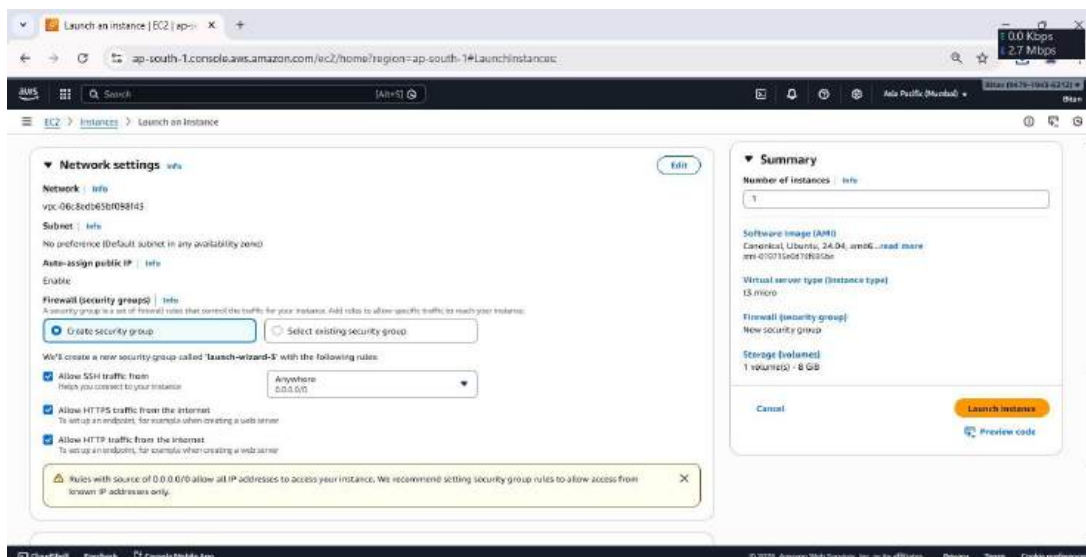
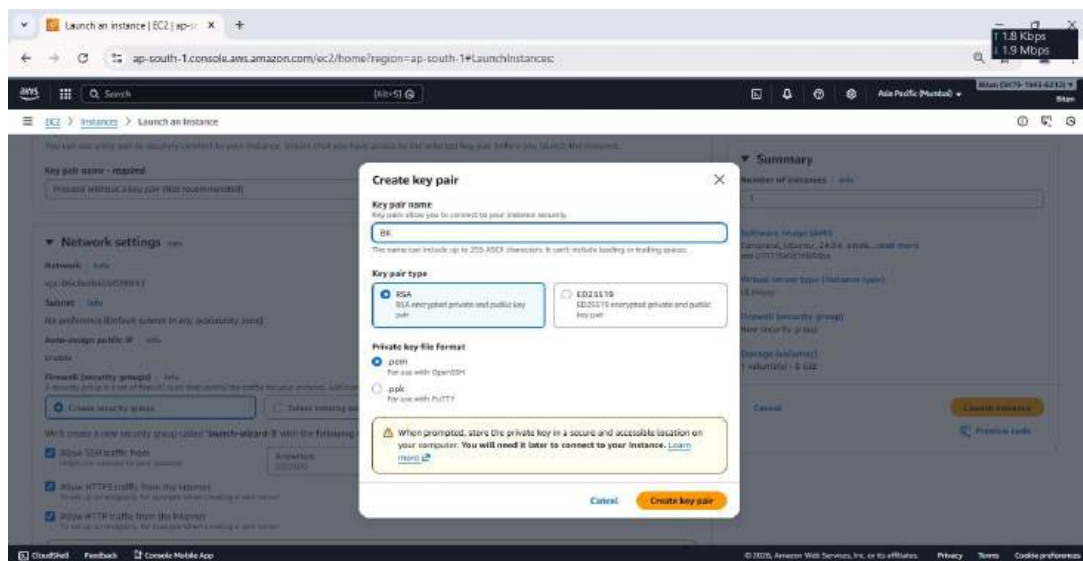
2. Click on 'Launch instance'.



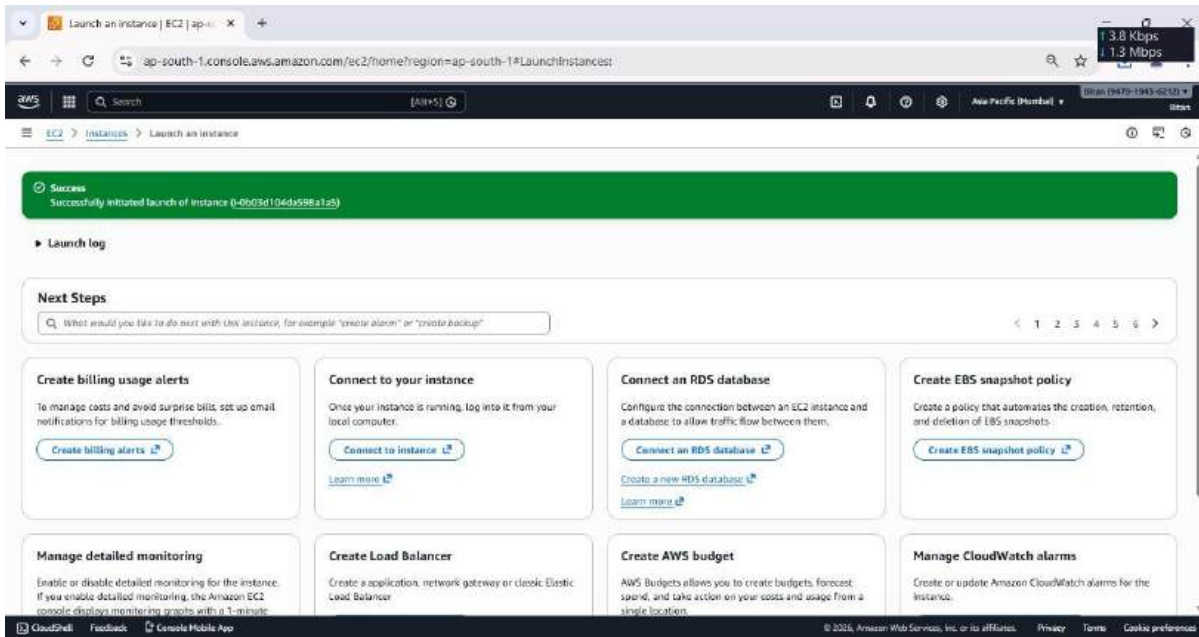
3. Fill up the required details->'Name' and under 'Application and OS Images' select 'ubuntu'.



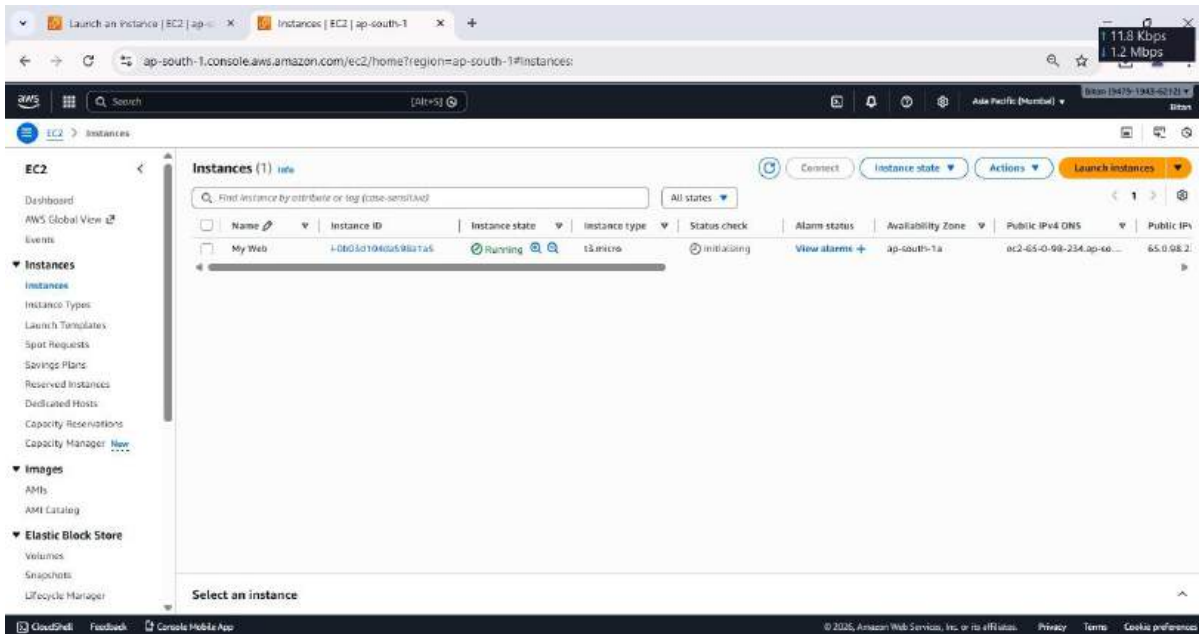
4. In key pair(login)', click on 'Create new key pair', give 'key pair name' and create it. Under 'Network Settings' tick on all checkboxes. Launch instance.



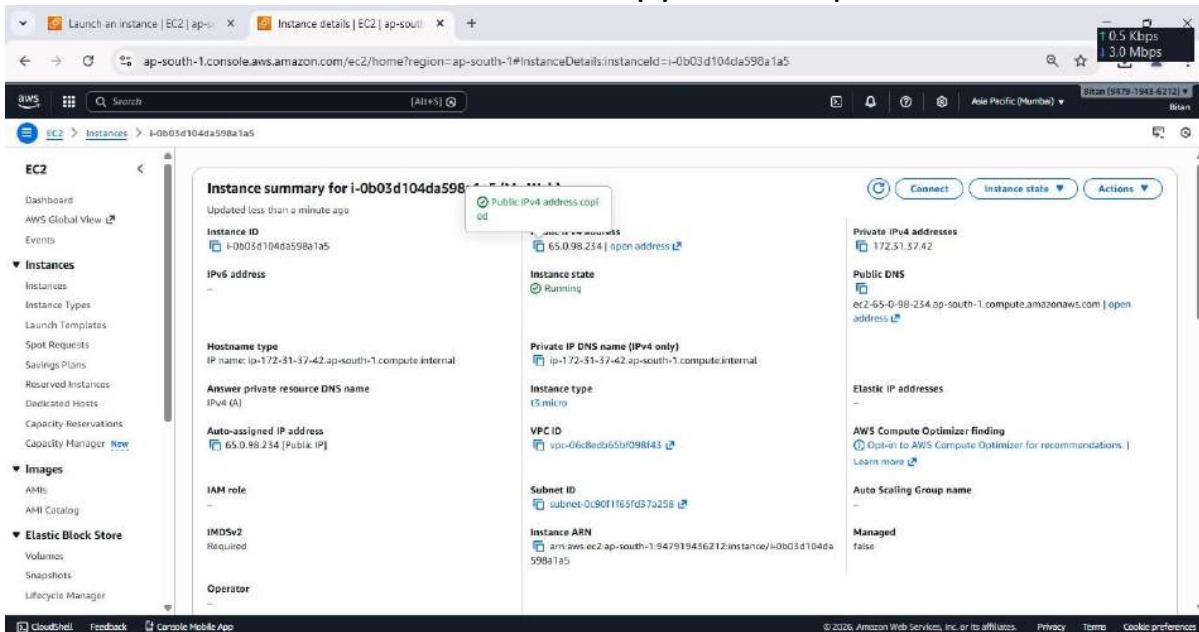
5. 'My Web' instance is created successfully. Then click on 'Instances'.



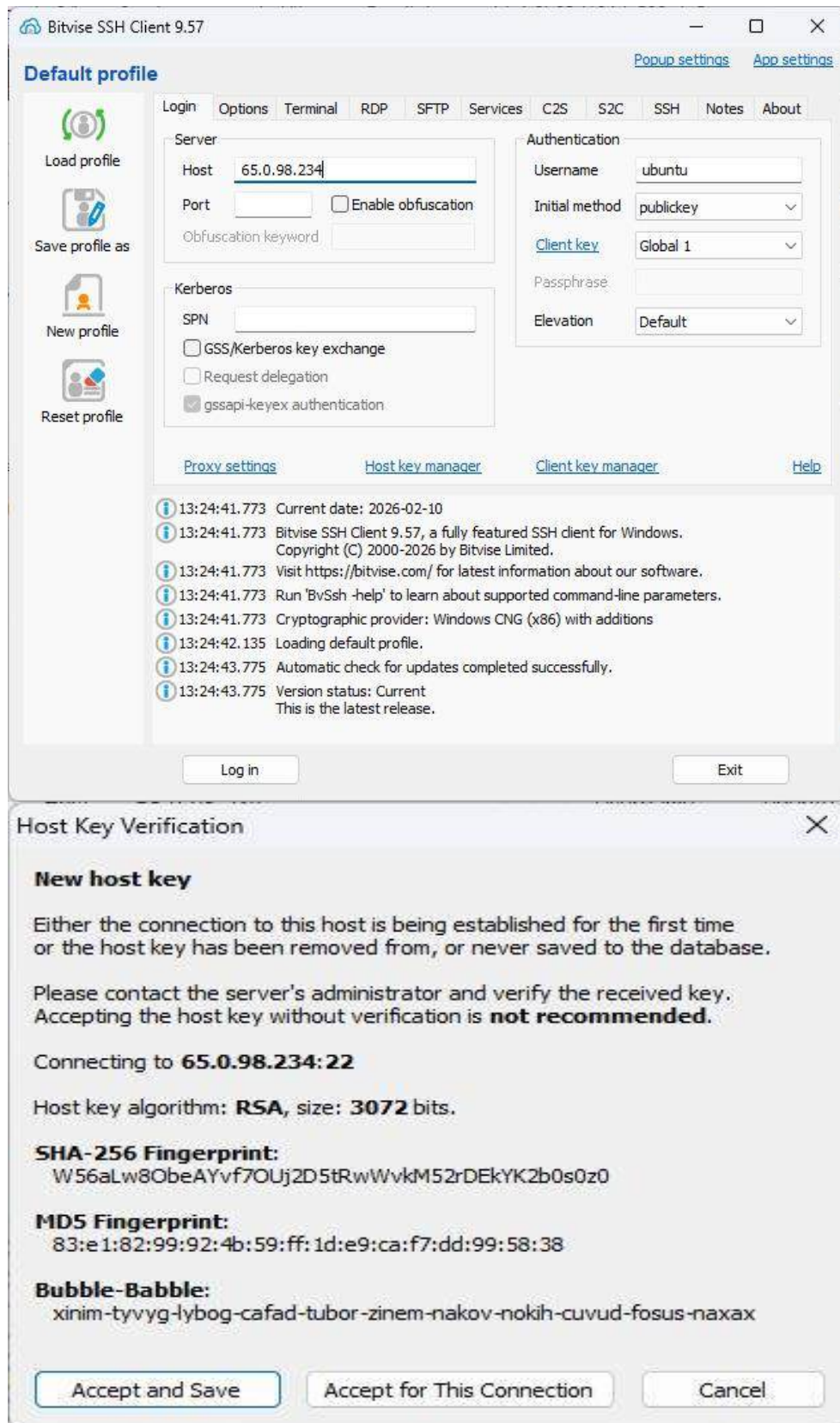
6. Now click on 'Instance ID'.



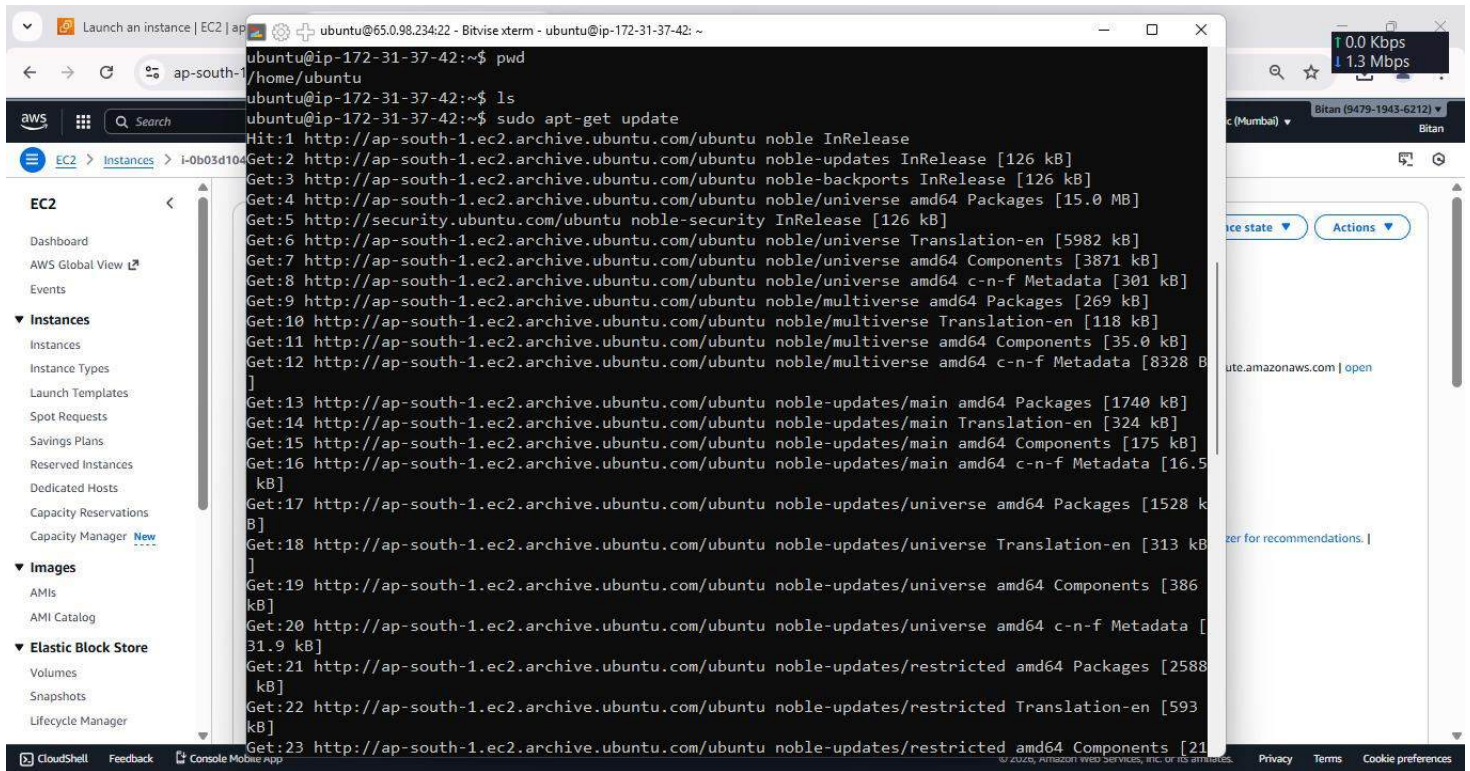
7. Click on 'Public IPV4 address' & copy it then open 'Bitwise SSH Client'.



8. In Bitvise SSH Client, paste the 'Public IPV4 address' 7 click on 'Client key Manager'. Under 'Client key manager', if there is any existing key then remove it and click on 'Import' , select the created key 'BK' then again click on 'Import' and then the key is successfully imported. Now under 'Default profile' give the 'Username' as 'ubuntu' then select Global1 from 'Client Key Manager' then click on 'Log in' & 'Accept and save'.

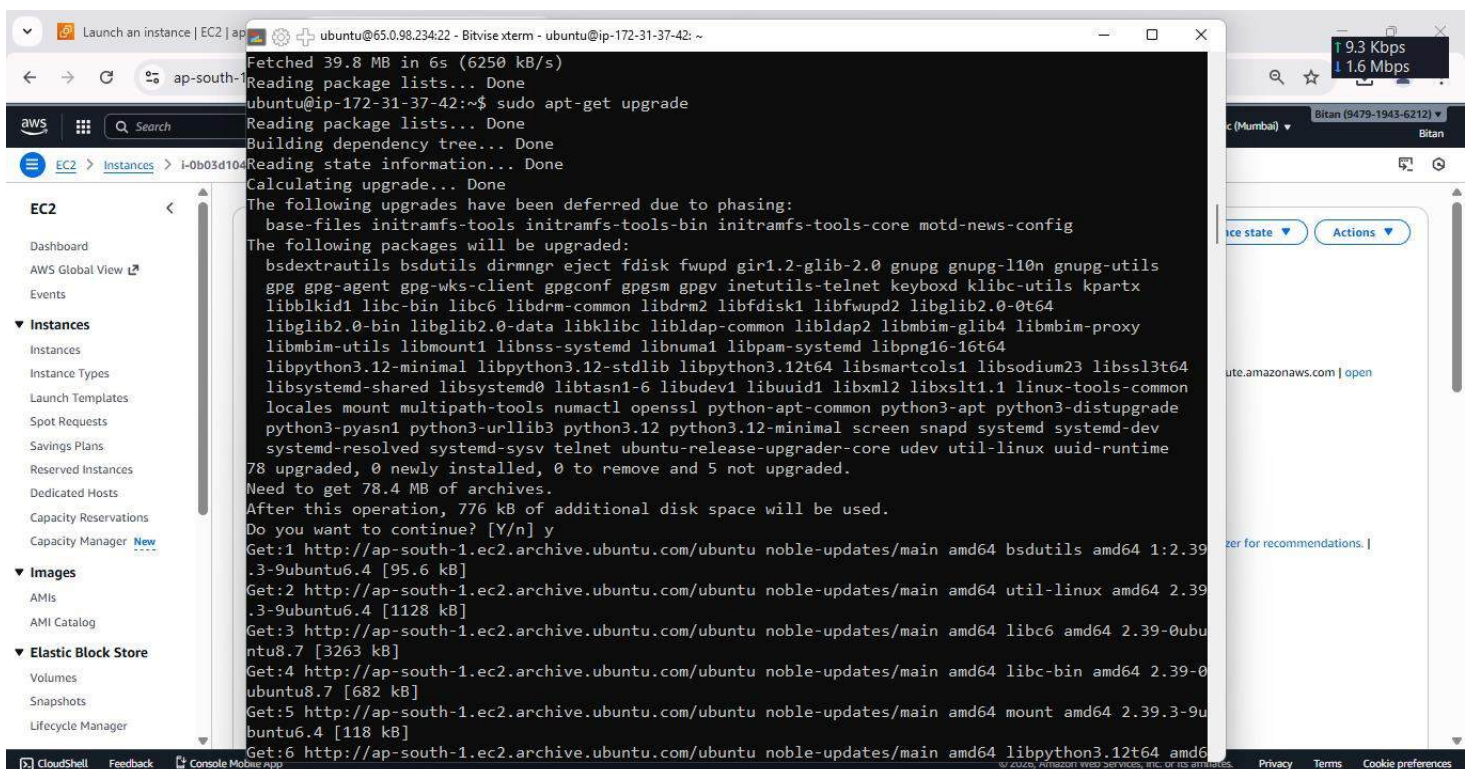


9. In 'Bitwise SSH client', under 'Default profile' click on 'New terminal console' and there run 'pwd' command to check where we are. in terminal console, type the following commands.-> 'sudo apt-get update' , 'sudo apt-get upgrade' , 'sudo apt-get install nginx'.



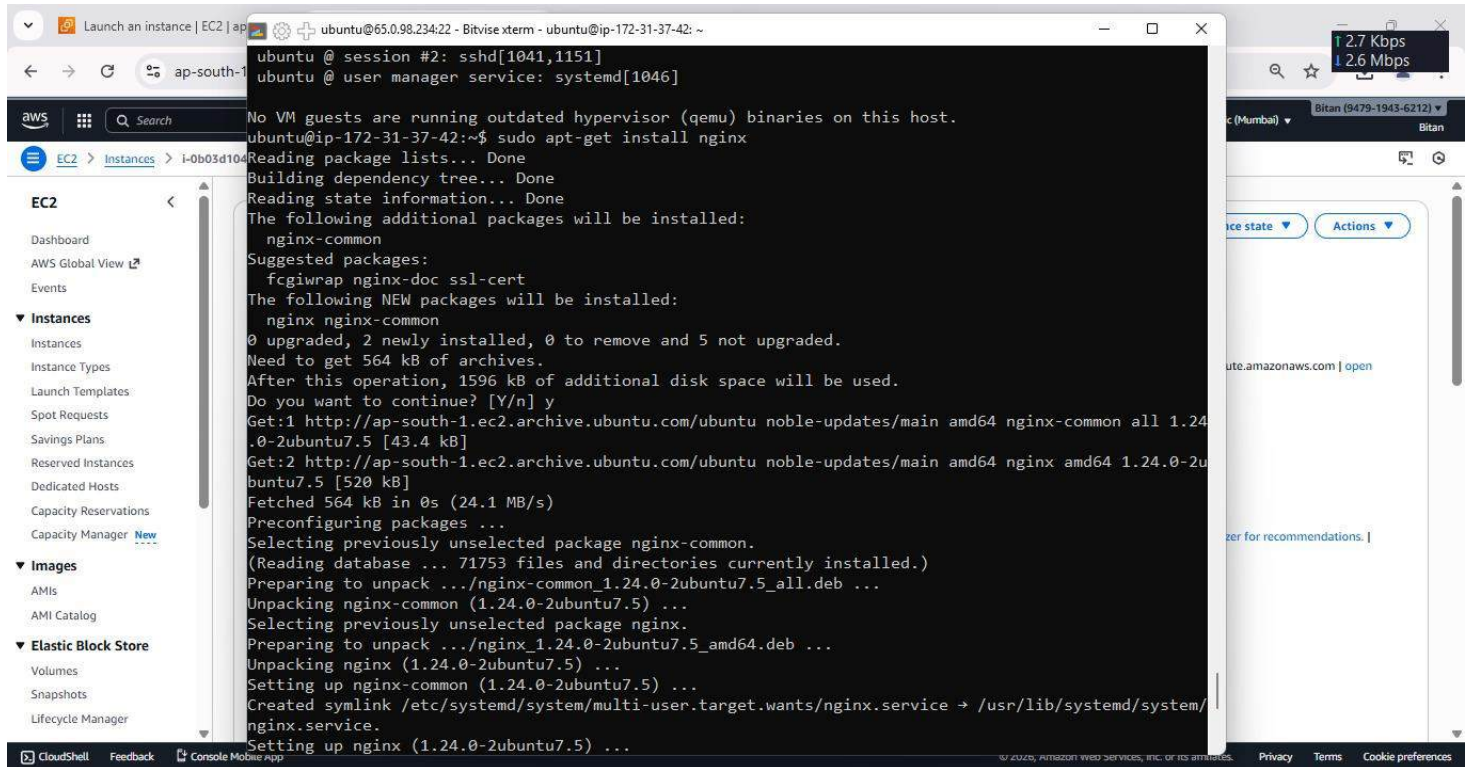
The screenshot shows the AWS Management Console with the 'Instances' page selected. A terminal window is open for an Ubuntu instance (ip-172-31-37-42). The terminal output shows the following commands and their results:

```
ubuntu@ip-172-31-37-42:~$ pwd
/home/ubuntu
ubuntu@ip-172-31-37-42:~$ ls
ubuntu@ip-172-31-37-42:~$ sudo apt-get update
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:5 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:6 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:7 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:8 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:9 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:10 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:11 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:12 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:13 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1740 kB]
Get:14 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [324 kB]
Get:15 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:16 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.5 kB]
Get:17 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1528 kB]
Get:18 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [313 kB]
Get:19 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:20 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [31.9 kB]
Get:21 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [2588 kB]
Get:22 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [593 kB]
Get:23 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [21
```

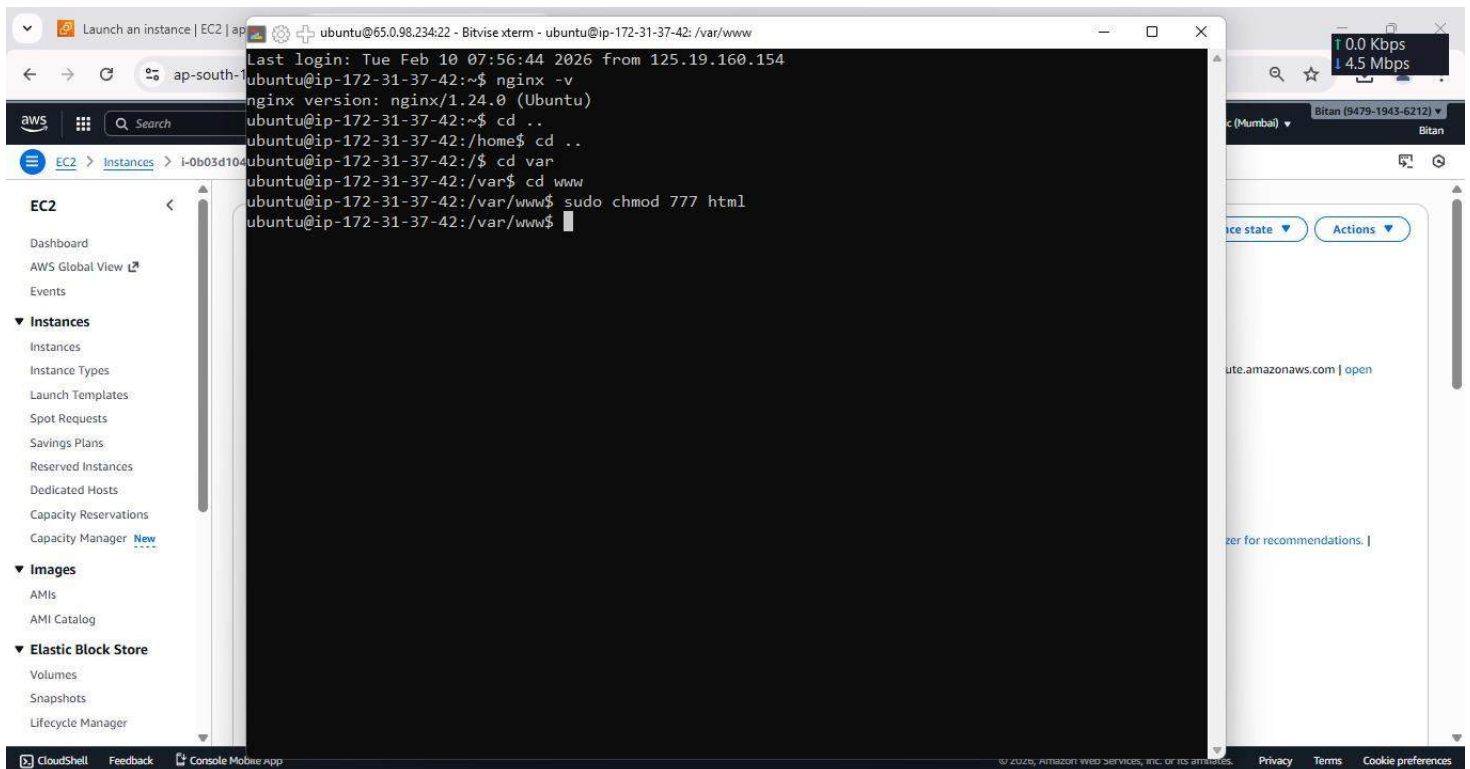


The screenshot shows the AWS Management Console with the 'Instances' page selected. A terminal window is open for an Ubuntu instance (ip-172-31-37-42). The terminal output shows the following commands and their results:

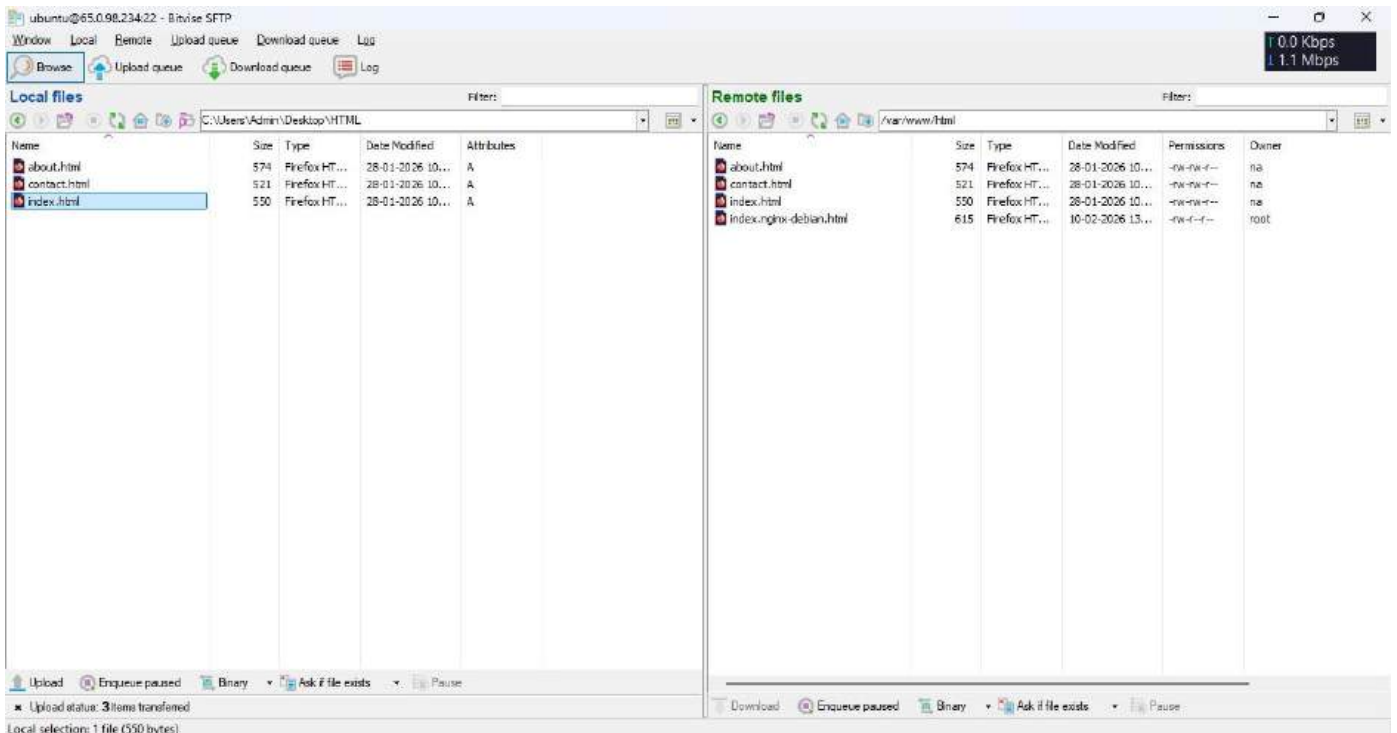
```
ubuntu@ip-172-31-37-42:~$ sudo apt-get upgrade
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following upgrades have been deferred due to phasing:
base-files inramfs-tools inramfs-tools-bin inramfs-tools-core motd-news-config
The following packages will be upgraded:
bsdextrautils bsdutils dirmngr eject fdisk fwupd gir1.2-glib-2.0 gnupg gnupg-110n gnupg-utils
gpg gpg-agent gpg-wks-client gpgconf gpgsm gpgv inetutils-telnet keyboxd klibc-utils kpartx
libblkid1 libc-bin libc6 libdrm-common libdrm2 libfdisk1 libfwupd2 libglib2.0-0t64
libglib2.0-bin libglib2.0-data libklibc libldap-common libldap2 libmbim-glib4 libmbim-proxy
libmbim-utils libmount1 libnss-systemd libnuma1 libpam-systemd libpng16-16t64
libpython3.12-minimal libpython3.12-stdlib libpython3.12t64 libsmartcols1 libsodium23 libssl3t64
libsystemd-shared libsystemd0 libtasn1-6 libudev1 libuuid1 libxml2 libxslt1.1 linux-tools-common
locales mount multipath-tools numactl openssl python3-apt-common python3-apt python3-distupgrade
python3-pyasn1 python3-urllib3 python3.12 python3.12-minimal screen snapd systemd systemd-dev
systemd-resolved systemd-sysv telnet ubuntu-release-upgrader-core udev util-linux uuid-runtime
78 upgraded, 0 newly installed, 0 to remove and 5 not upgraded.
Need to get 78.4 MB of archives.
After this operation, 776 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 bsdutils amd64 1:2.39
.3-9ubuntu6.4 [95.6 kB]
Get:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 util-linux amd64 2.39
.3-9ubuntu6.4 [1128 kB]
Get:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 libc6 amd64 2.39-0ubu
ntu8.7 [3263 kB]
Get:4 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 libc-bin amd64 2.39-0
ubuntu8.7 [682 kB]
Get:5 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 mount amd64 2.39.3-9u
buntu6.4 [118 kB]
Get:6 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 libpython3.12t64 amd64
```



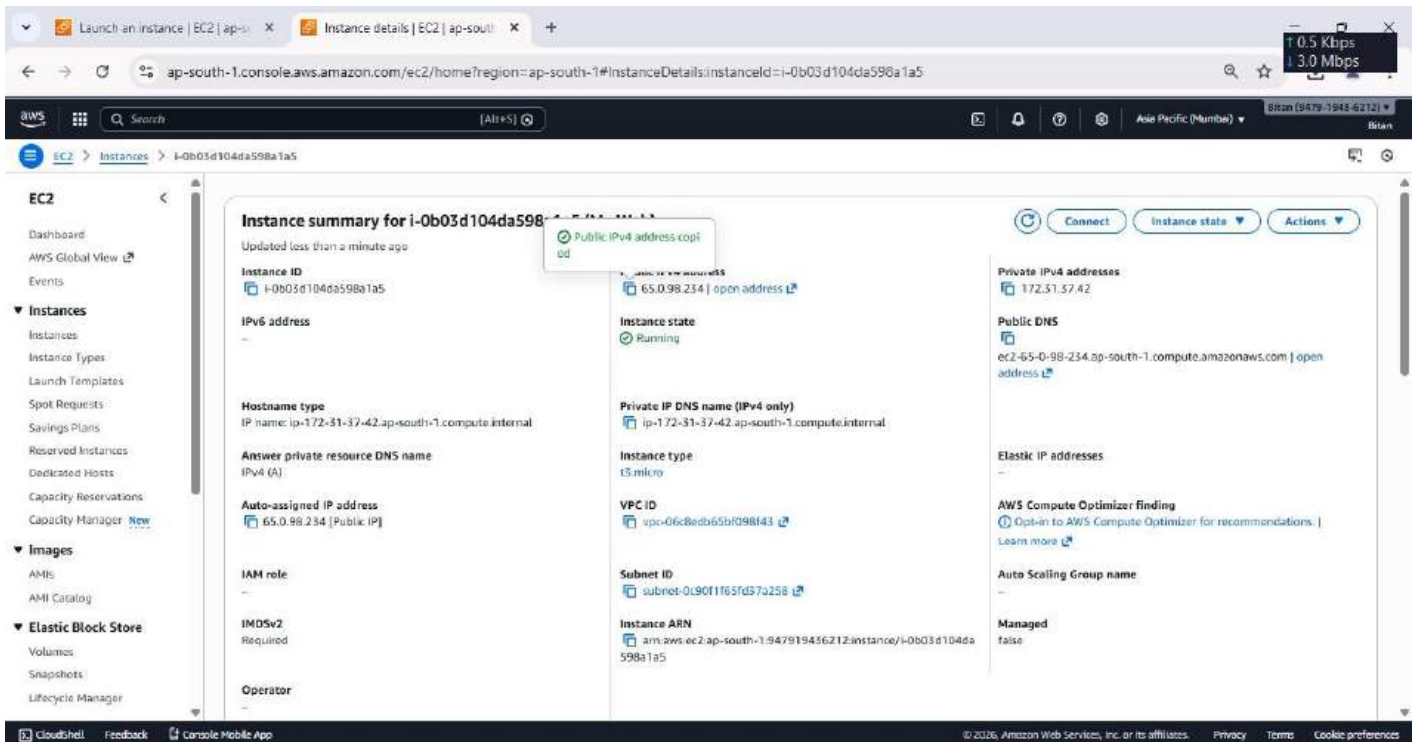
10. Check if `www` folder appears or not by using `ls` command inside `'var'` folder. Now to give the file access permissions to it go back to the `'www'` directory, and type the command `'sudo chmod 777 html'` and press `'Enter'`.



11. Now going back to the 'SFTP Window' under the 'Remote Files' open the HTML directory and drag & drop the HTML files.



12. Now go back to the 'AWS Window' and copy the 'IPV4 address'.



13. In a new window paste the 'IPV4 address'.



[Home](#) [About](#) [Contact](#)

Welcome

This is a static website hosted on Amazon S3.



[Home](#) [About](#) [Contact](#)

About This Site

This website is a simple example of a static site hosted using Amazon S3.



[Home](#) [About](#) [Contact](#)

Contact

Email: example@example.com

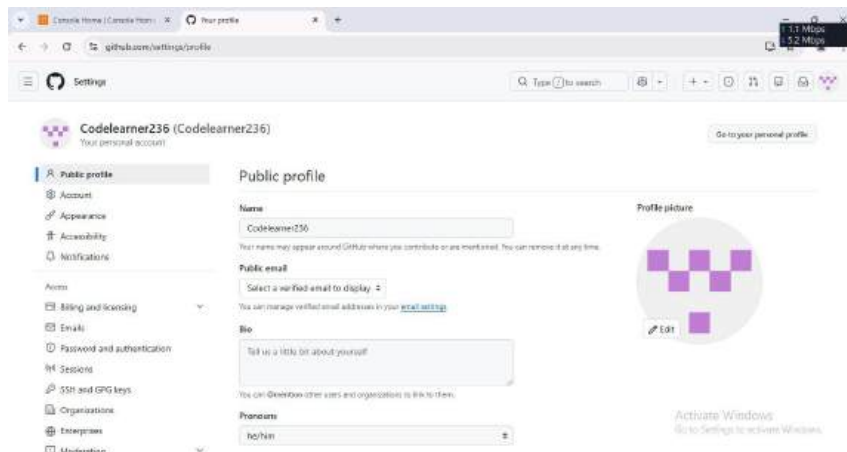
Assignment No:8

Problem Statement:

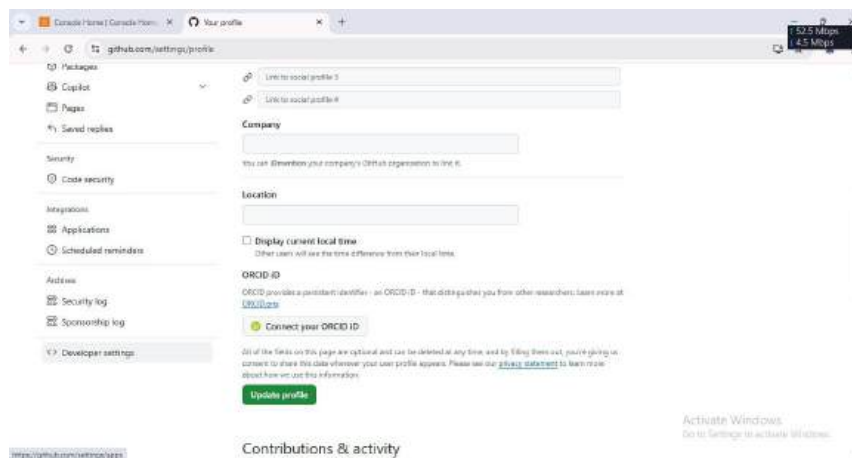
Deploy a project from a local machine to GitHub and vice versa.

Procedure:

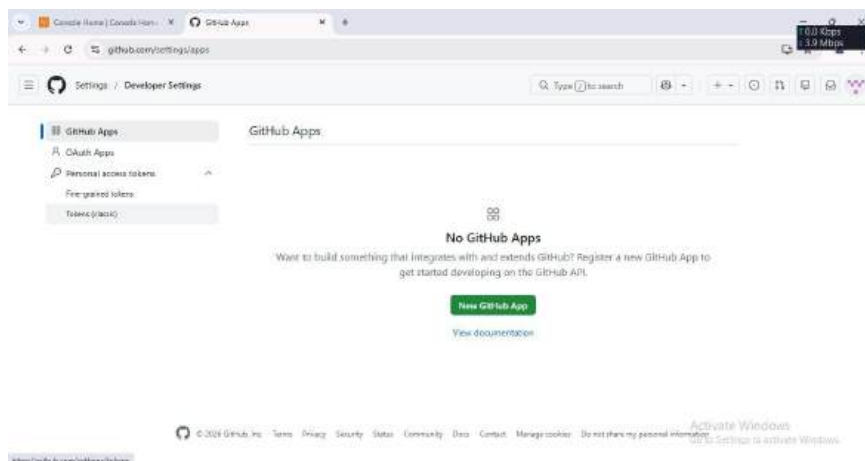
1. Login to the GitHub account and from there click on the profile picture and from there click on "Settings"



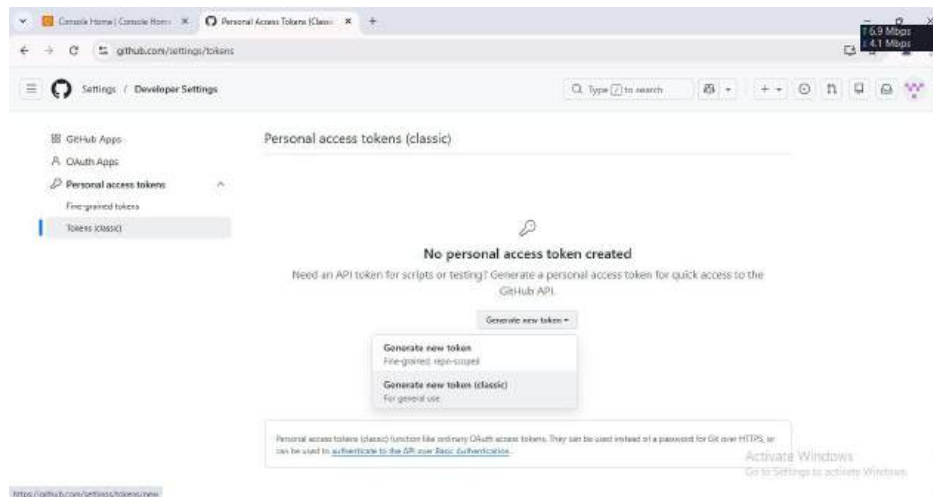
2. And then scroll down then click on "Developer Settings"



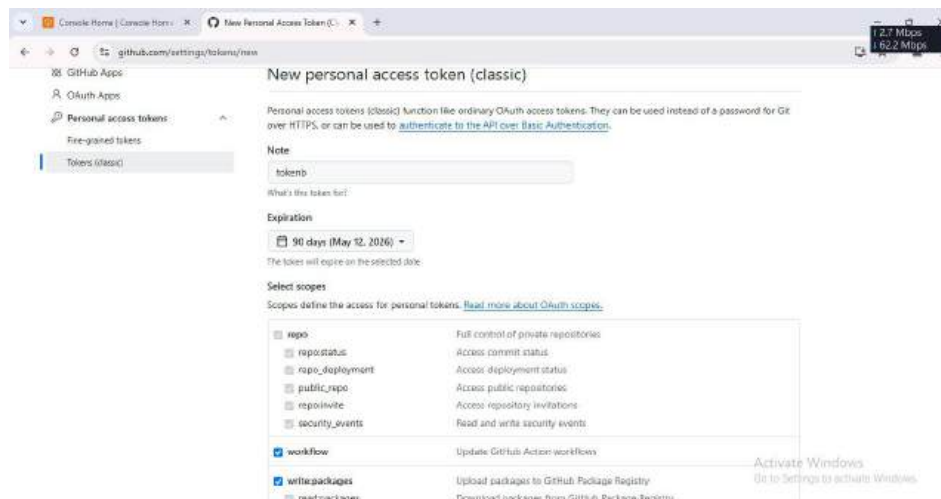
3. Now click on "Personal access tokens" drop down then click on "Tokens(classic)" and then click on "New GitHub app"



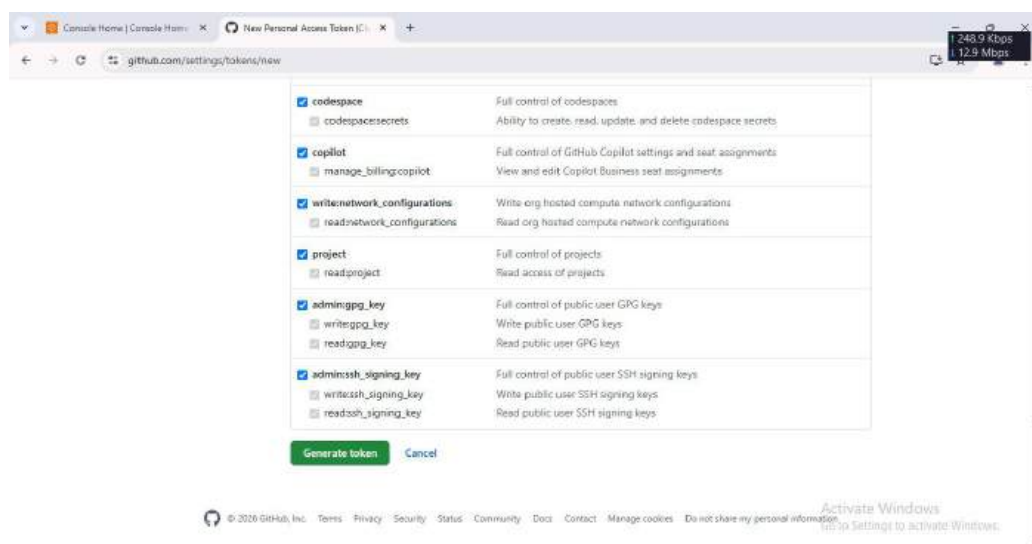
4. Now click on "Generate new token" drop down then click on "Generate new token(classic)".



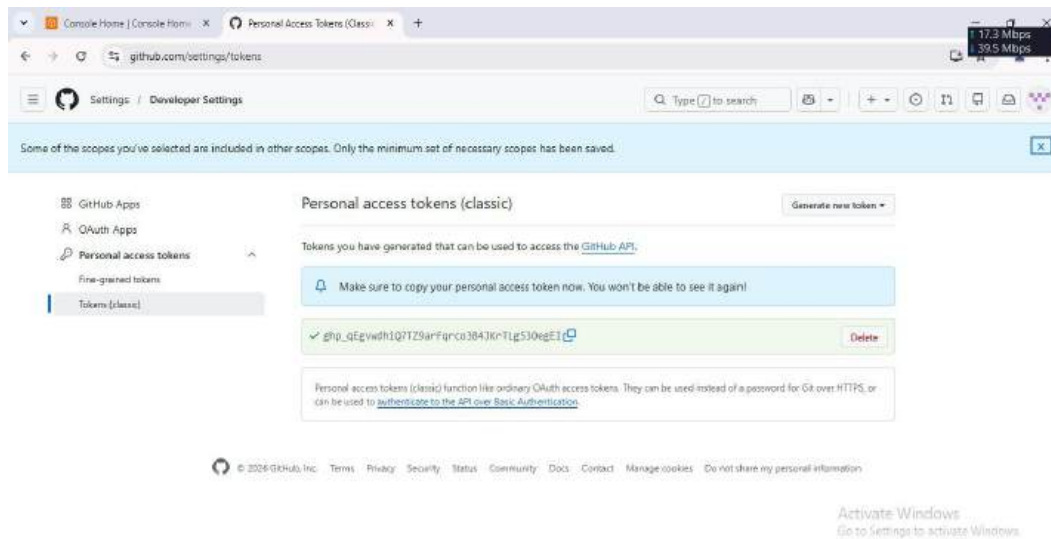
5. Now enter the token name and select "Expiration" as "90 days". Here the name is "tokenb"



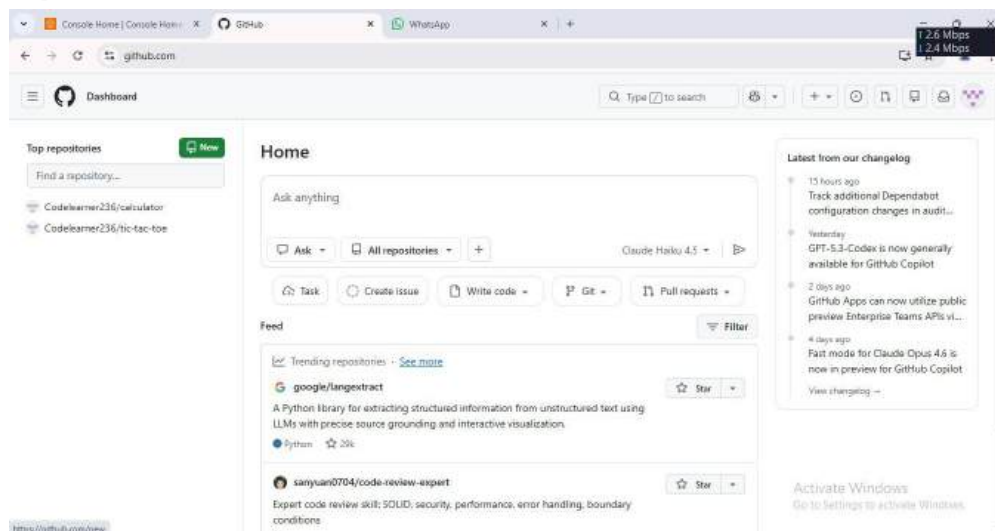
6. Now check all the check-boxes present there and click on "Generate token".



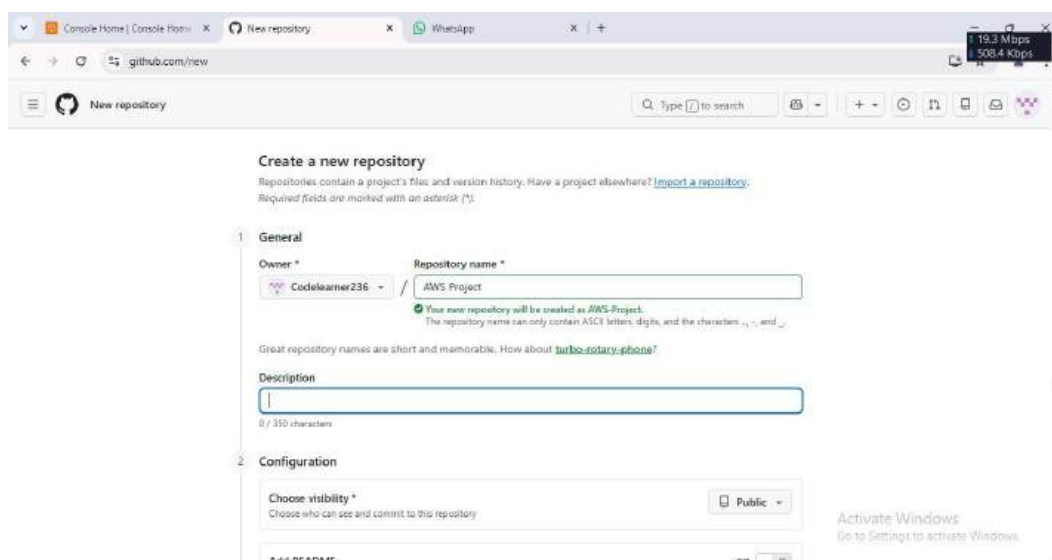
7. Now copy the token address and paste it into the text file



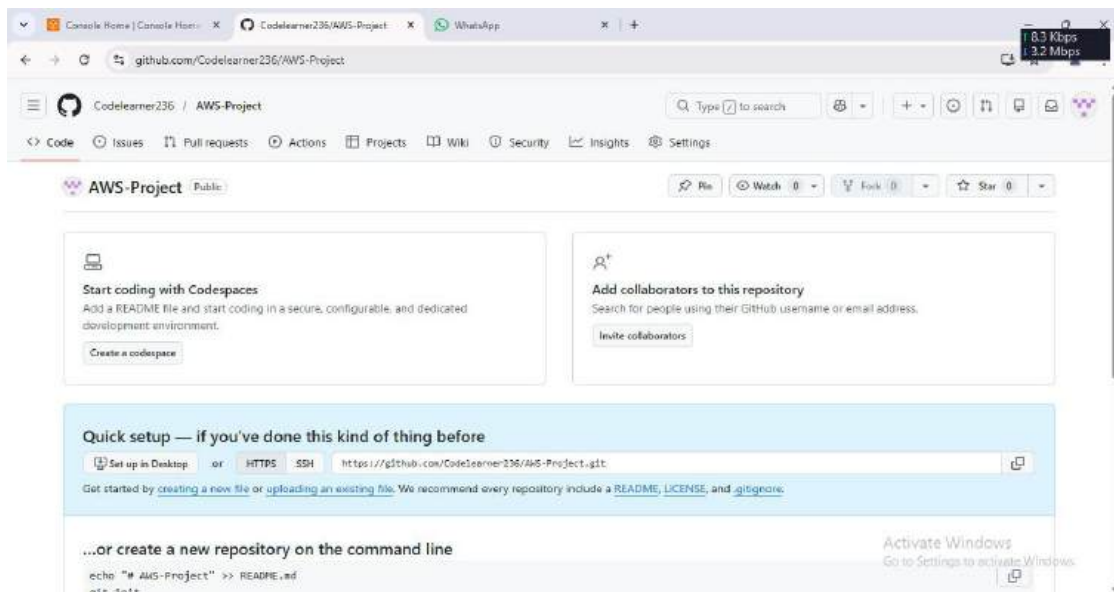
8. Now again come back to the GitHub dashboard and from there click on "Create Repository".



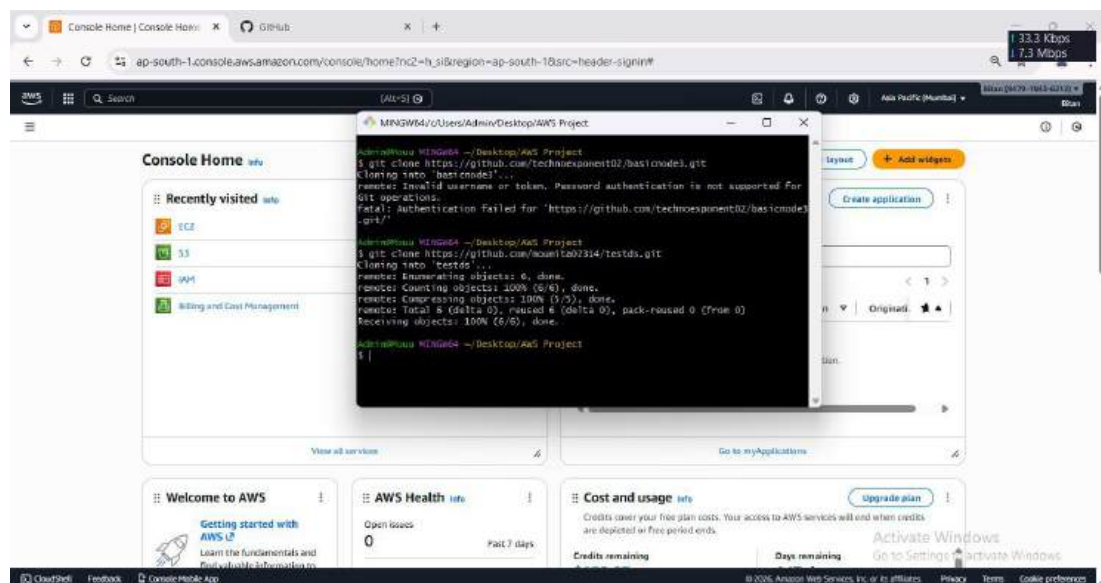
9. Now enter the Repository name and click on "Create repository". Here the repository name is "AWS Project".



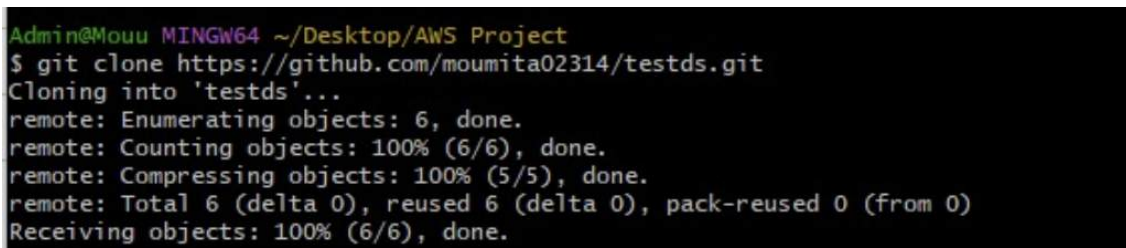
10 Now copy the Repository Address "https://github.com/Codelearner236/AWS-Project.git" and paste it into the same text file. Now we are ready to deploy any project from Github to our local machine.



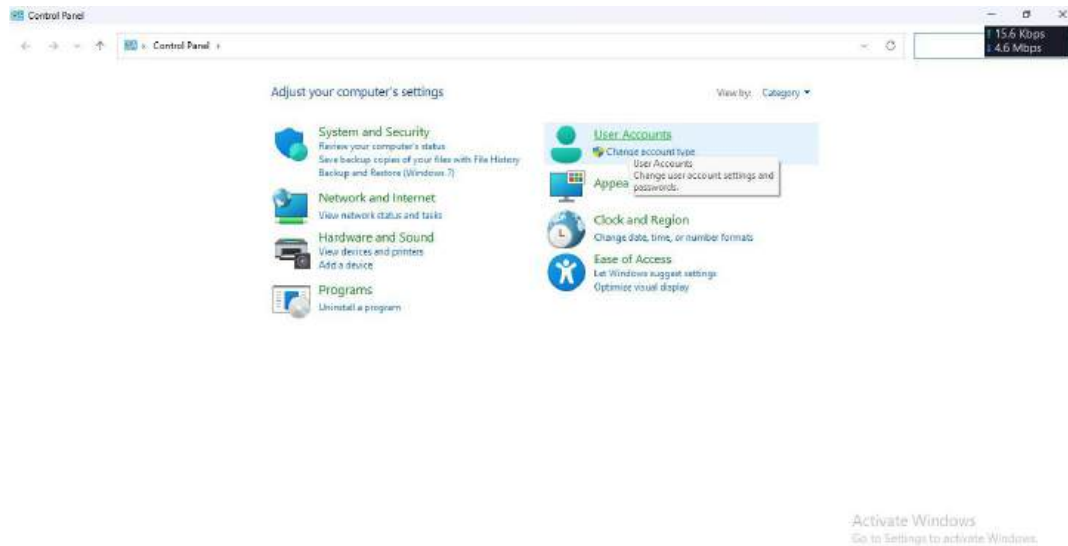
11 So to do so, open the folder that we have created and from there we will open Git Bash. Here the folder name is "AWS Project".



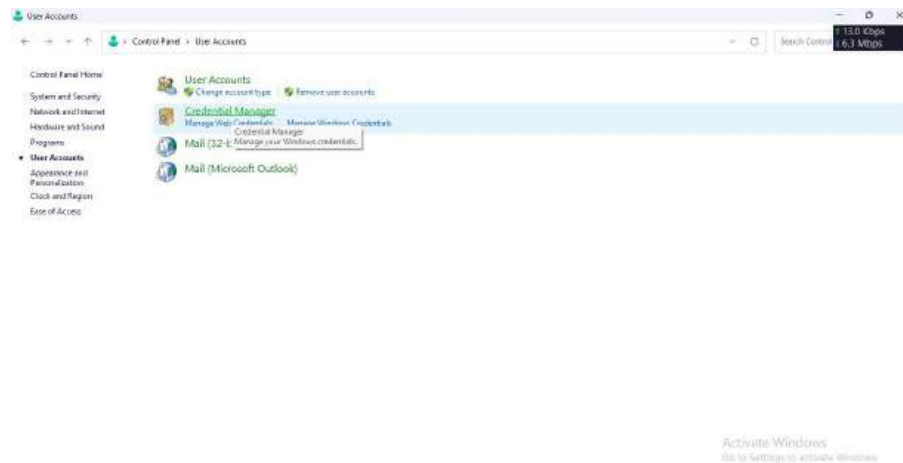
12 Clone "testds" repository from GitHub of user "moumita02314" to the local machine using the following command "git clone https://github.com/moumita02314/testds.git".



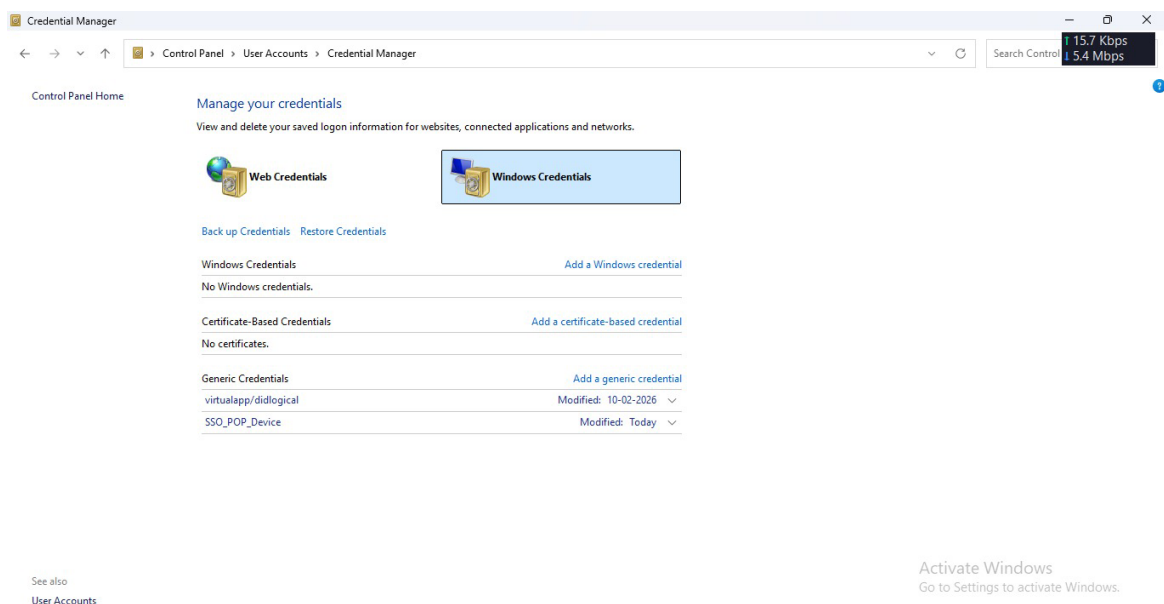
13 Now we will open Control Panel and navigate to User Accounts.



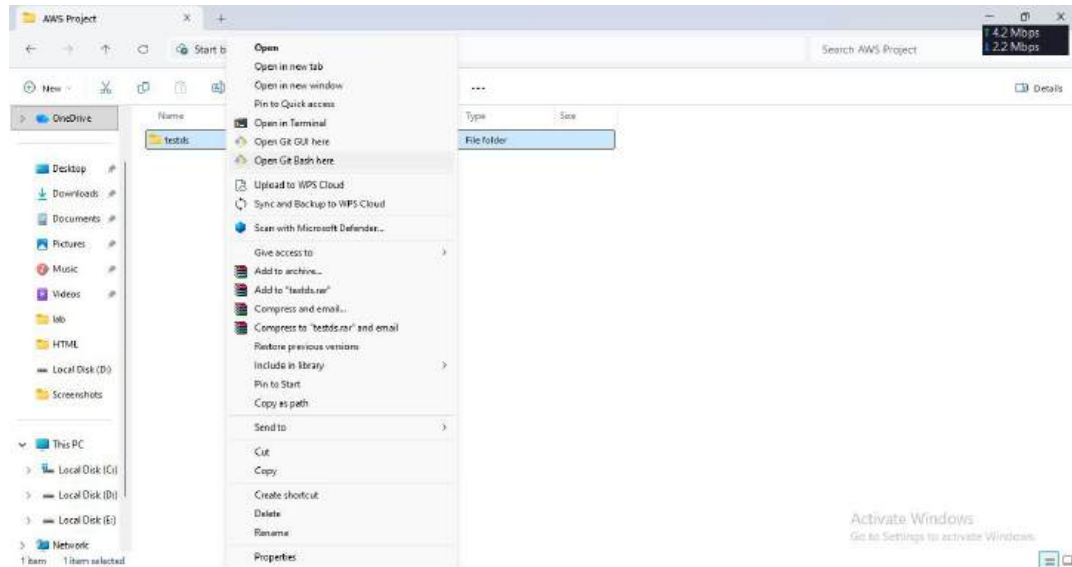
14 Then we will open Credential Manager and select Windows Credentials.



15 If any existing GitHub credentials are present, then we will remove them to ensure fresh authentication during the Git push process. Now we will not face any type issue during Authentication.



16 As we can see we have successfully deployed the project from GitHub to our local machine , Now we will deploy the contents of "testds" from our local machine to GitHub repo. To do we will open the "testds" folder and delete the ".gitignore" file and now again open Git Bash from this "testds" folder only.



17 Now run the given commands one by one :

(git config --global --list) -->

(git config --global user.name <user name>) -->

(git config --global user.email <user email>) -->

(git config --global --list) -->

(git init) -->

(git add .) -->

(git status) -->

(git commit -m "Done") -->

(git remote add origin <Repository Address>) -->

(git push -u origin master).

```
Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds
$ git config --global --list
user.name=pausali-sengupta
user.email=pausali2004@gmail.com
user.email=pausali2004@gmail.com
```

```
Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds
$ git config --global user.name "Codelearner236"
```

```
Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds
$ git config --global user.email "bitankarmakar22@gmail.com"
```

```
Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds
$ git config --global --list
user.name=Codelearner236
user.email=bitankarmakar22@gmail.com
```

```
Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds
$ git init
Initialized empty Git repository in C:/Users/Admin/Desktop/AWS Project/testds/.git/
```

```
Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds (master)
$ git add .
```

```
Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds (master)
$ git status
On branch master
```

No commits yet

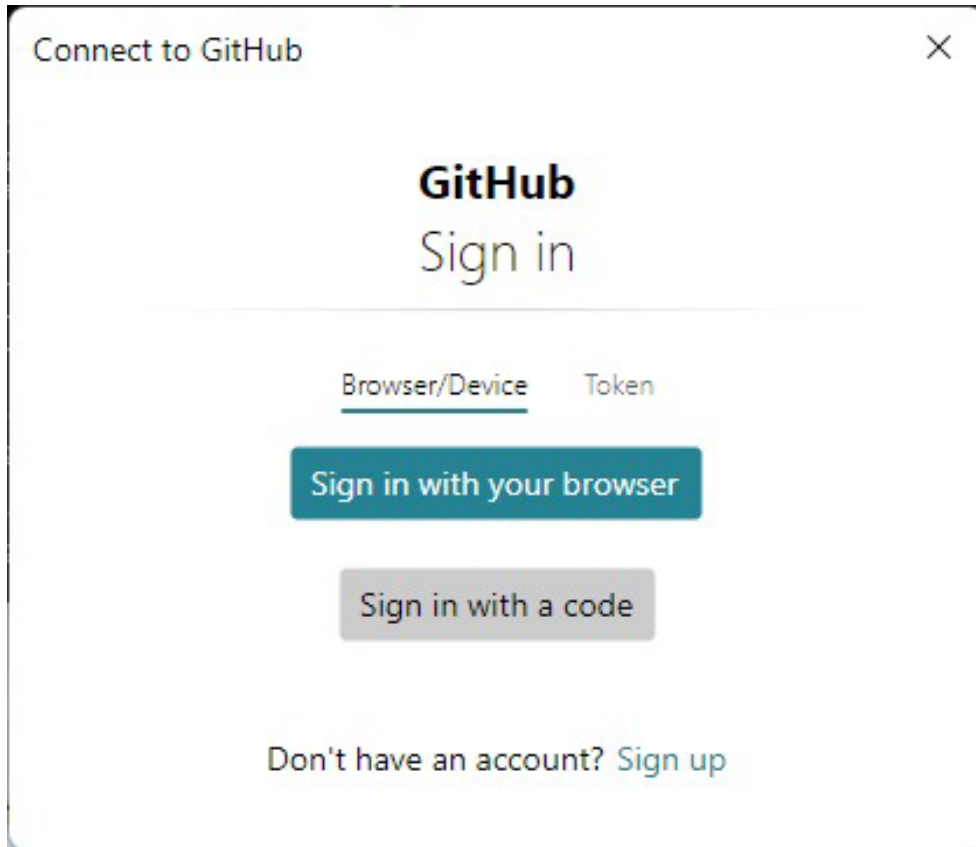
```
Changes to be committed:
(use "git rm --cached <file>..." to unstage)
new file:   .gitignore
new file:   New Text Document.txt
new file:   index.js
new file:   package.json
```

```
Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds (master)
$ git commit -m "done"
[master (root-commit) a5130bd] done
4 files changed, 53 insertions(+)
create mode 100644 .gitignore
create mode 100644 New Text Document.txt
create mode 100644 index.js
create mode 100644 package.json
```

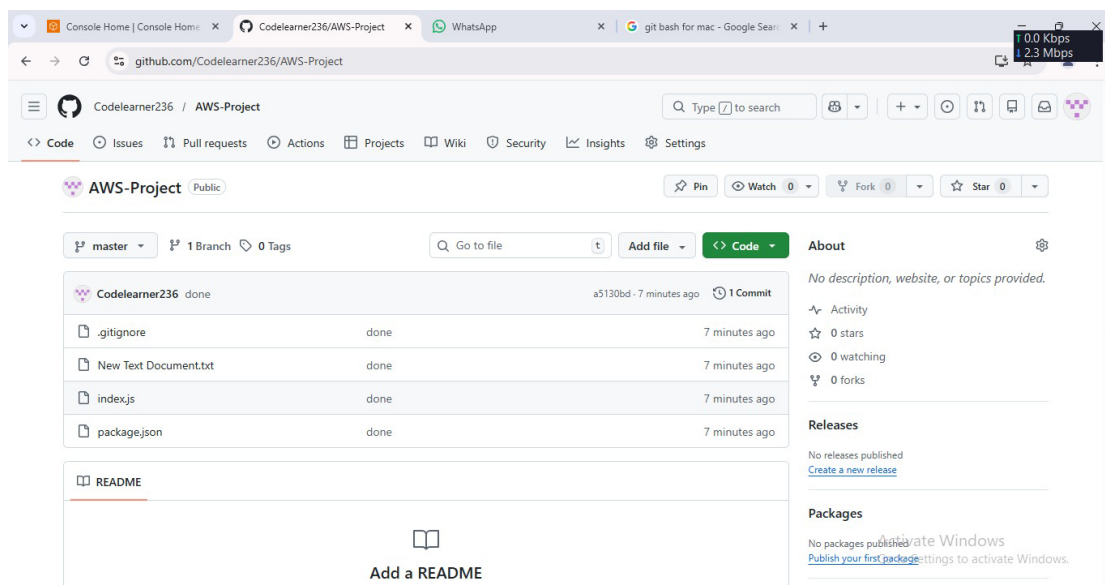
```
Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds (master)
$ git remote add origin https://github.com/Codelearner236/AWS-Project.git
```

```
Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds (master)
$ git push -u origin master
```


18 After running all the commands "GitHub OAuth Authorization Window" will open, paste the token address that we have created during token generation and sign in.



19 As we can see after successful authentication, the files were deployed and displayed in the GitHub repository.



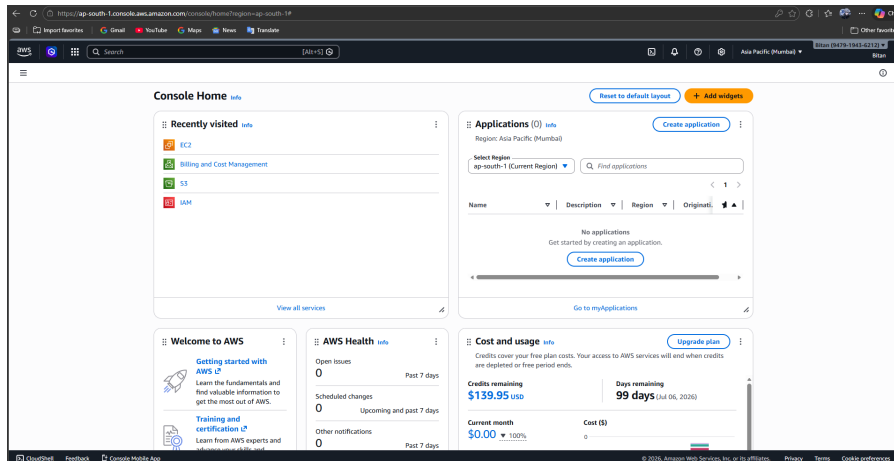
ASSIGNMENT – 9

PROBLEM STATEMENT:

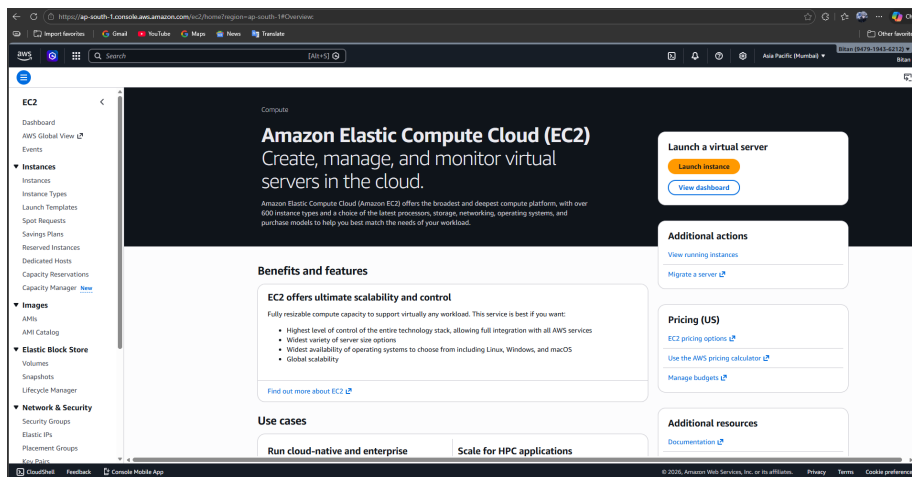
Deploy a project from Github to EC2

Steps:

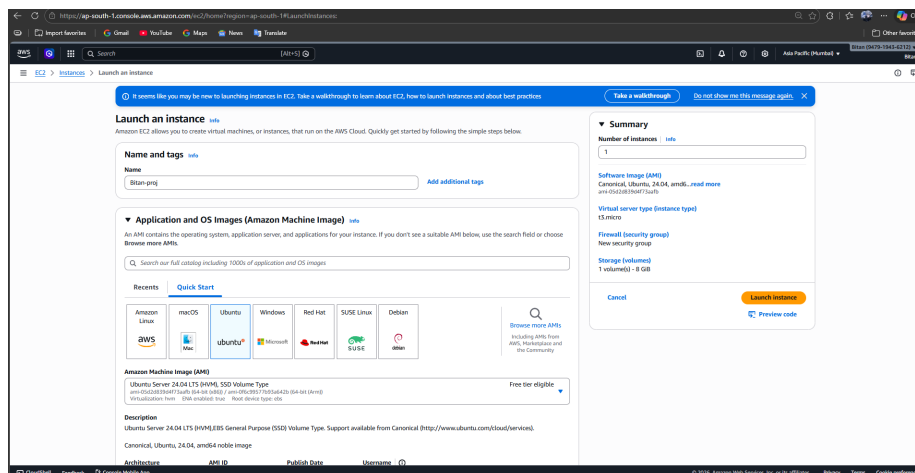
1. Login to the AWS Management Console and from the home page click on "EC2".



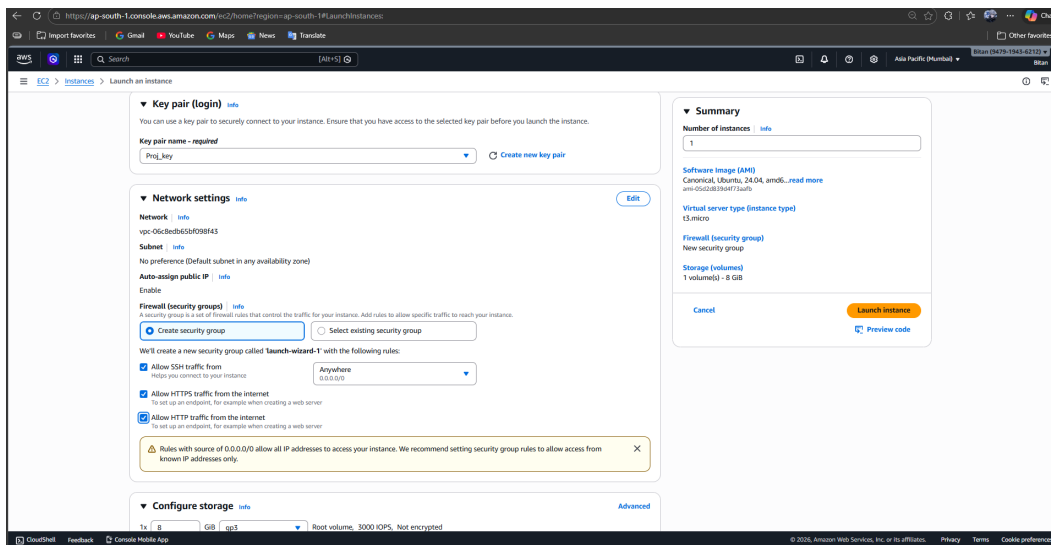
2. Now click on "Launch instance"



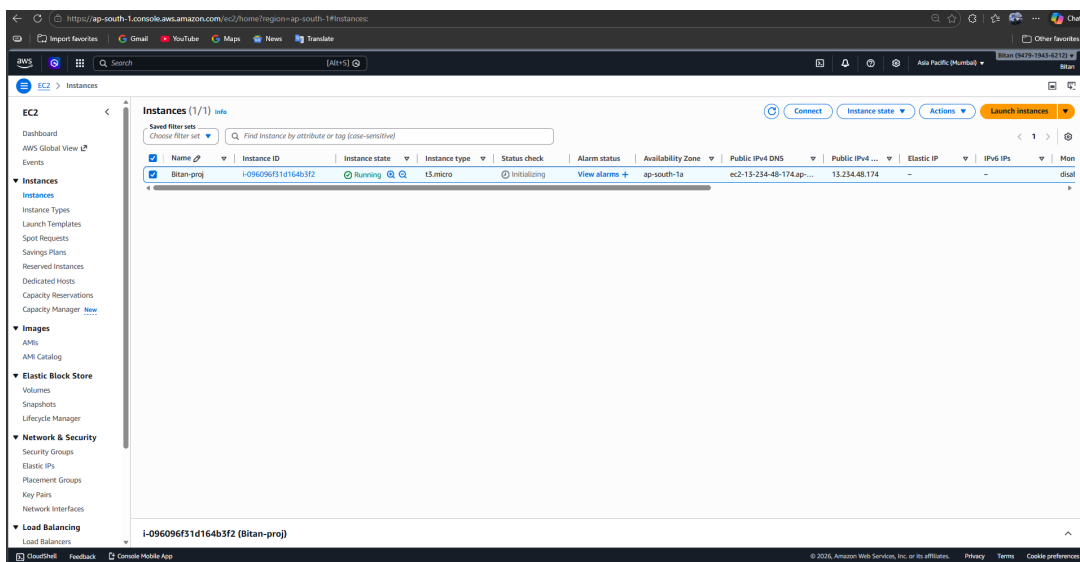
3. Now enter any name of the instance, Here it is “Bitan-proj”, and select “Ubuntu” from Application and OS Images.



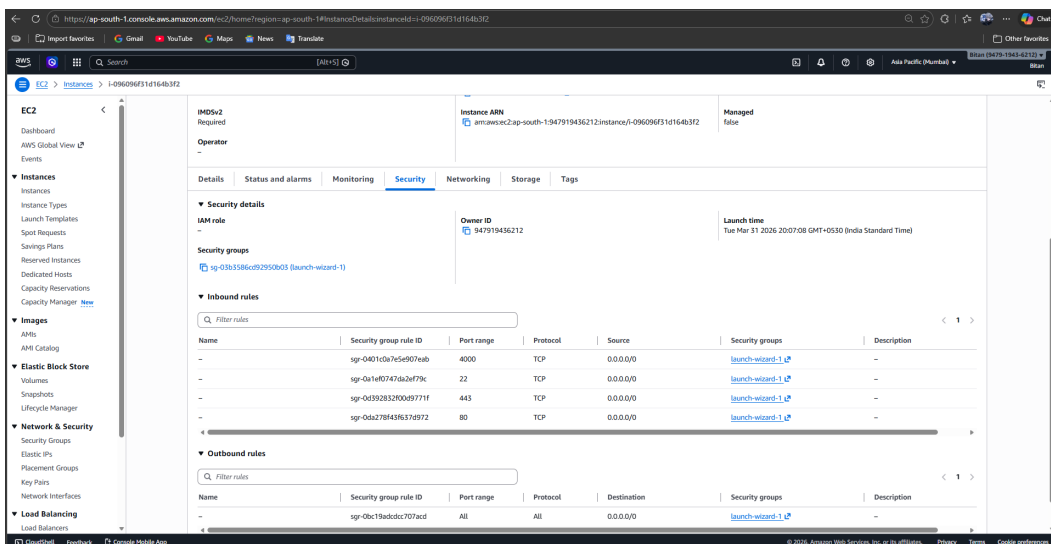
4. Now enter key pair, Here it is “Proj_key” and select all three check boxes from the Network settings that are “Allow SSH traffic from”, “Allow HTTPS traffic from the internet” and “Allow HTTP traffic from the internet” and click on “Launch instance”.



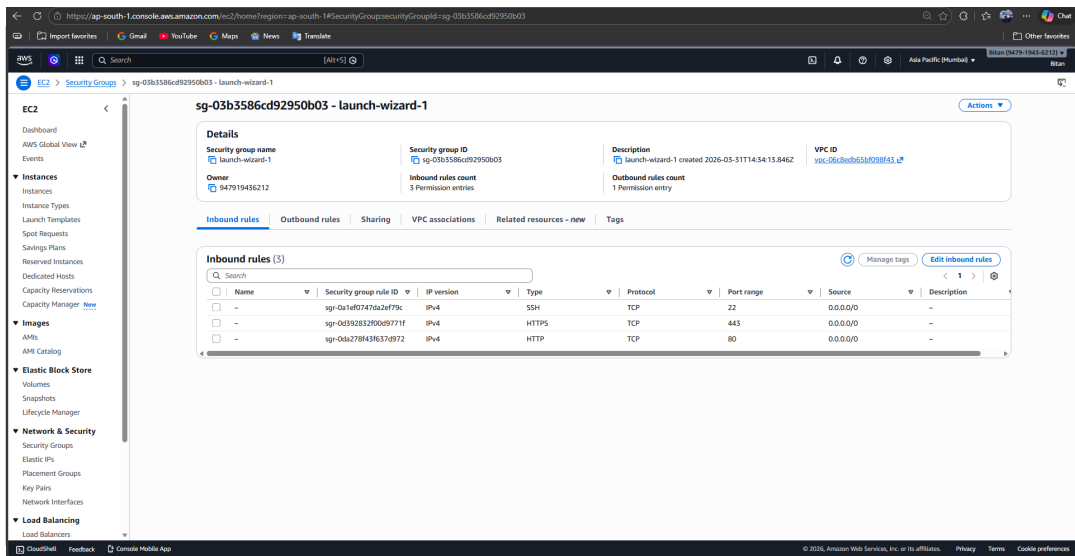
5. Now move back to the instances and, from there click on instance id.



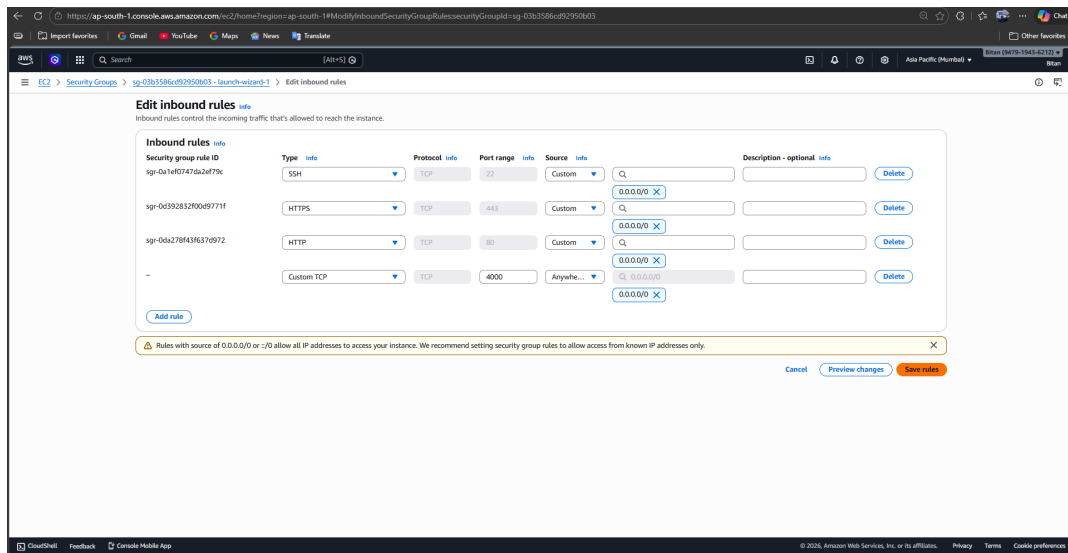
6. Now click on “security” tab from there, the click on “Security groups”.



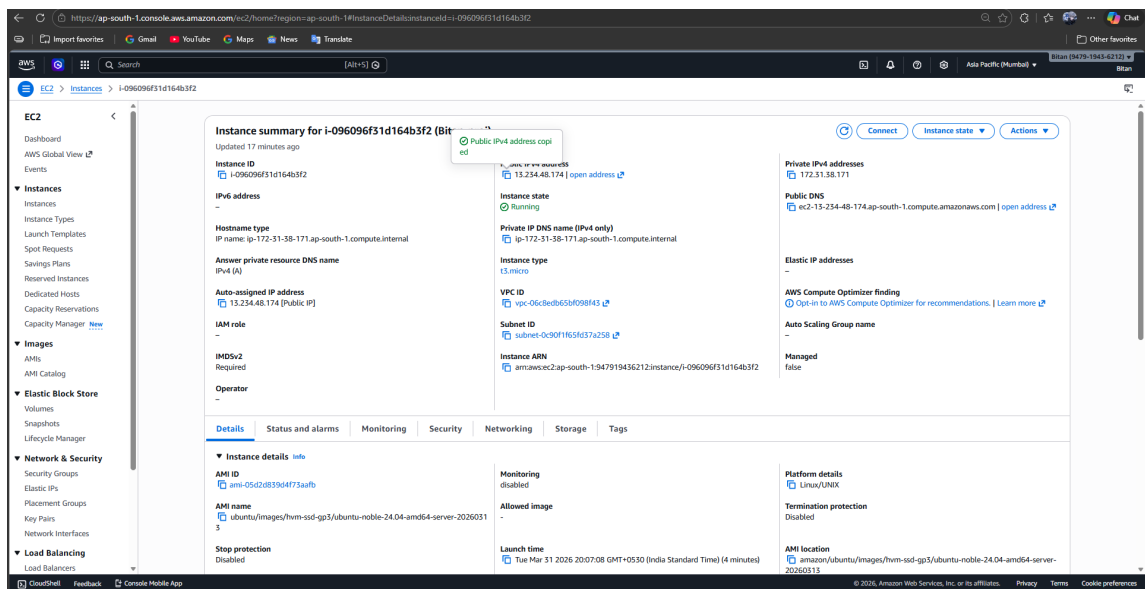
7. Now click on “Edit Inbound rules”



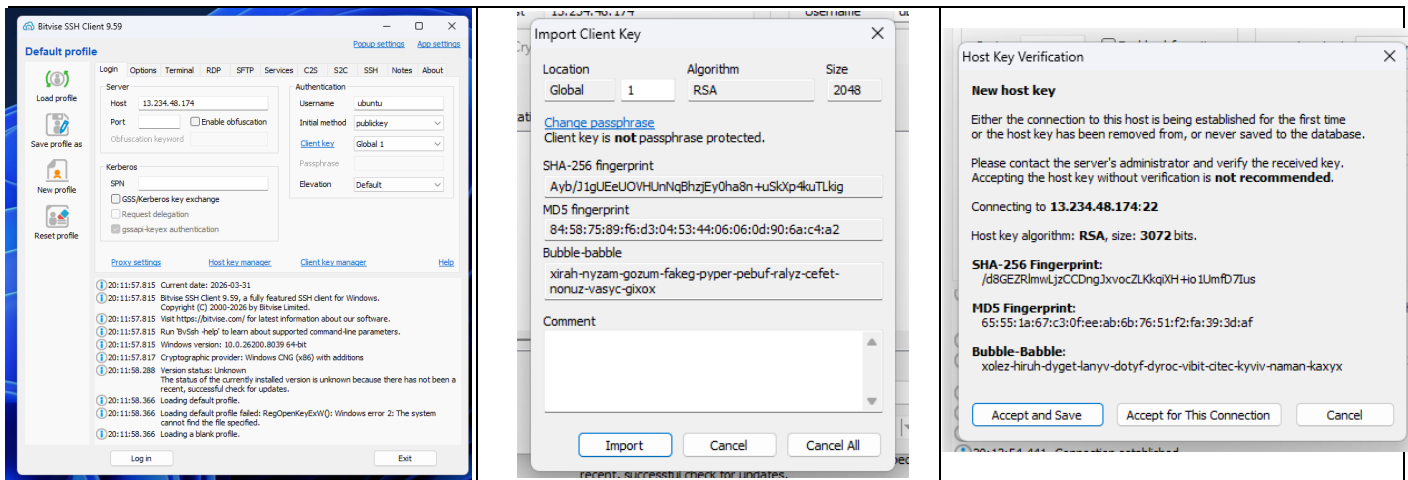
8. Now click on “add rule” button, then a new section named as : “Custom TCP” will open in that section, enter (Port range = 4000), and enter 0.0.0.0/0 in the search pannel.



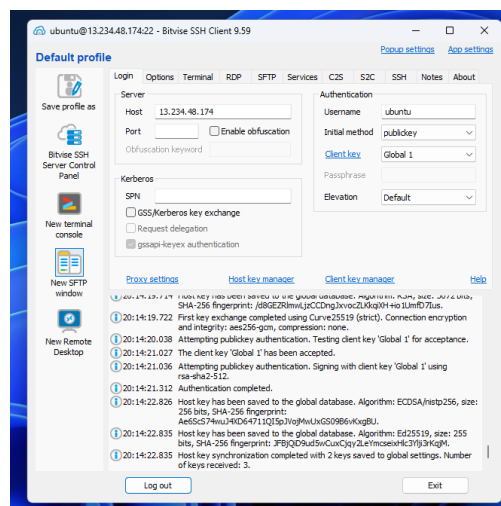
9. Now copy the “Public IPV4 Address”



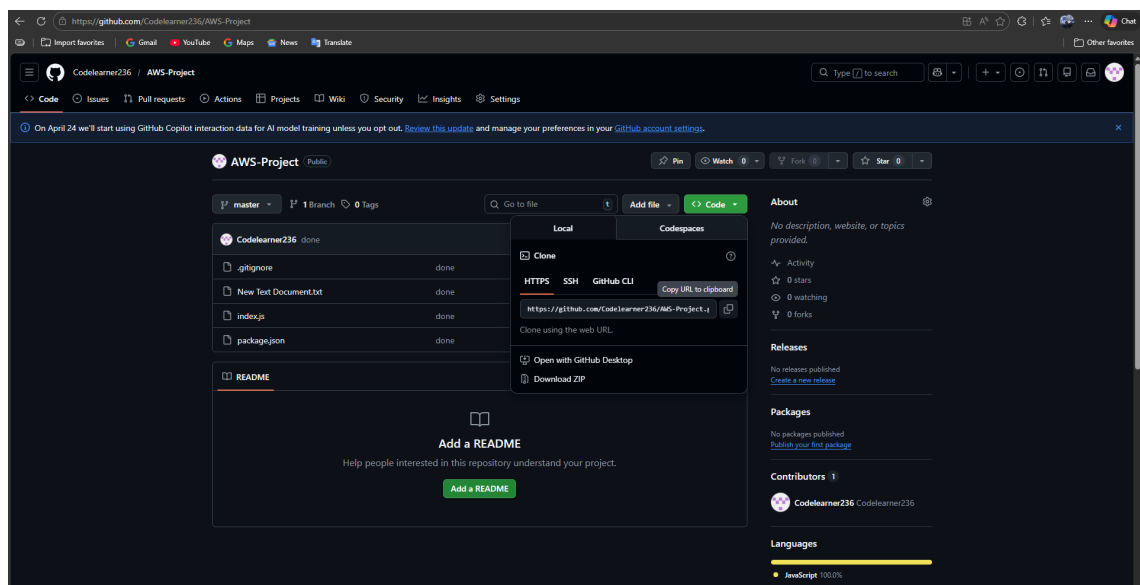
10 Now open “Bitwise SSH Client” and paste the Public IPV4 address that we have just copied from the instance, and then click on “Client key manager” to import the key pair and finally we can login.



11 Now click on “New Terminal Console”



12 Now login to the GitHub account and goto the repository where the projects are stored, now from there click on “code” button and copy the link.



13 Now we run the given commands one by one using the terminal :

(sudo apt-get update)

(sudo apt-get upgrade)

(sudo apt install nginx)

(curl -SL https://deb.nodesource.com/setup_16.x|sudo -E bash -)

(sudo apt install nodejs)

(git clone "Paste the project repository link")

(dir)

(cd "repository name")

(npm install)

(node index.js)

To start the server.

```
ubuntu@ip-172-31-38-171:~$ sudo apt-get update
```

```
ubuntu@ip-172-31-38-171:~$ sudo apt-get upgrade
```

```
ubuntu@ip-172-31-38-171:~$ sudo apt-get install nginx
```

```
ubuntu@ip-172-31-38-171:~$ curl -SL https://deb.nodesource.com/setup_16.x|sudo -E bash -
```

```
ubuntu@ip-172-31-38-171:~$ sudo apt install nodejs
```

```
ubuntu@ip-172-31-38-171:~$ nodejs -v
```

```
ubuntu@ip-172-31-38-171:~$ git clone https://github.com/Codelearner236/AWS-Project.git
```

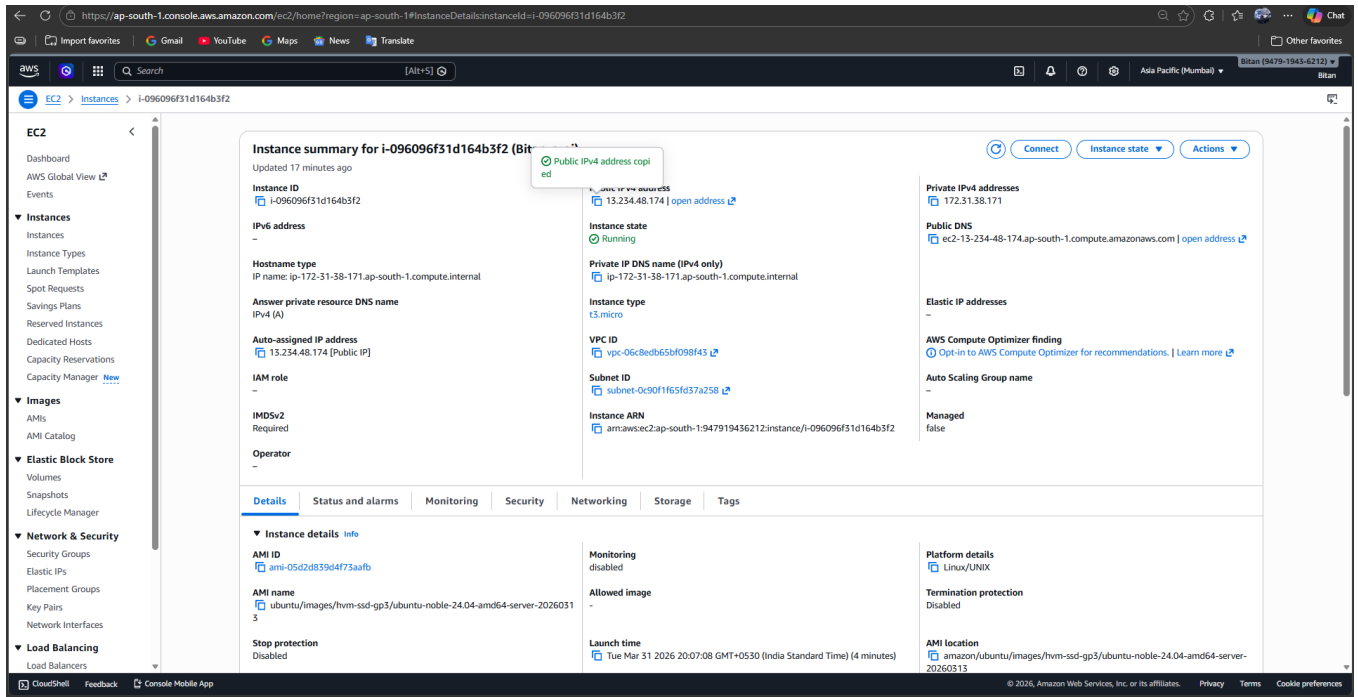
```
ubuntu@ip-172-31-38-171:~$ dir
```

```
ubuntu@ip-172-31-38-171:~$ cd AWS-Project/
```

```
ubuntu@ip-172-31-38-171:~/AWS-Project$ npm install
```

```
ubuntu@ip-172-31-38-171:~/AWS-Project$ node index.js  
Started server
```

14 Now copy the “Public IPV4 Address” again from the instance.



The screenshot shows the AWS Management Console for an EC2 instance. The instance ID is i-096096f31d164b3f2. The instance is in the 'Running' state. The Public IPv4 address is 13.234.48.174. A tooltip indicates that this address has been copied. The instance is running on the t3.micro instance type, using the ami-05d2d839d4f73aafb AMI. The instance is located in the ap-south-1 region, in the Mumbai Availability Zone. The instance is part of the ec2-13-234-48-174-ap-south-1-compute-amazonaws-com Auto Scaling Group.

Instance summary for i-096096f31d164b3f2 (Bit)
Instance ID: i-096096f31d164b3f2
IPv6 address: -
Hostname type: IP name: ip-172-31-38-171.ap-south-1.compute.internal
Answer private resource DNS name: IPv4 (A)
Auto-assigned IP address: 13.234.48.174 [Public IP]
IAM role: -
IMDSv2: Required
Operator: -
Instance state: Running
Private IP DNS name (IPv4 only): ip-172-31-38-171.ap-south-1.compute.internal
Instance type: t3.micro
VPC ID: vpc-06c8edb65bf098f43
Subnet ID: subnet-0c90f1f65d37a258
Instance ARN: arn:aws:ec2:ap-south-1:947919436212:instance/i-096096f31d164b3f2
Private IPv4 addresses: 172.31.38.171
Public DNS: ec2-13-234-48-174.ap-south-1.compute.amazonaws.com
Elastic IP addresses: -
AWS Compute Optimizer finding: Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto Scaling Group name: -
Managed: false

Details	Status and alarms	Monitoring	Security	Networking	Storage	Tags
Instance details AMI ID: ami-05d2d839d4f73aafb AMI name: ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-20260313 Stop protection: Disabled	Monitoring: disabled Allowed image: - Launch time: Tue Mar 31 2026 20:07:08 GMT+0530 (India Standard Time) (4 minutes)	Platform details: Linux/UNIX Termination protection: Disabled AMI location: amazon/ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-20260313				

15 And paste it into a new tab and press enter,
As we can see the content of the project is displayed on the screen i.e.
“Hello mckv”.



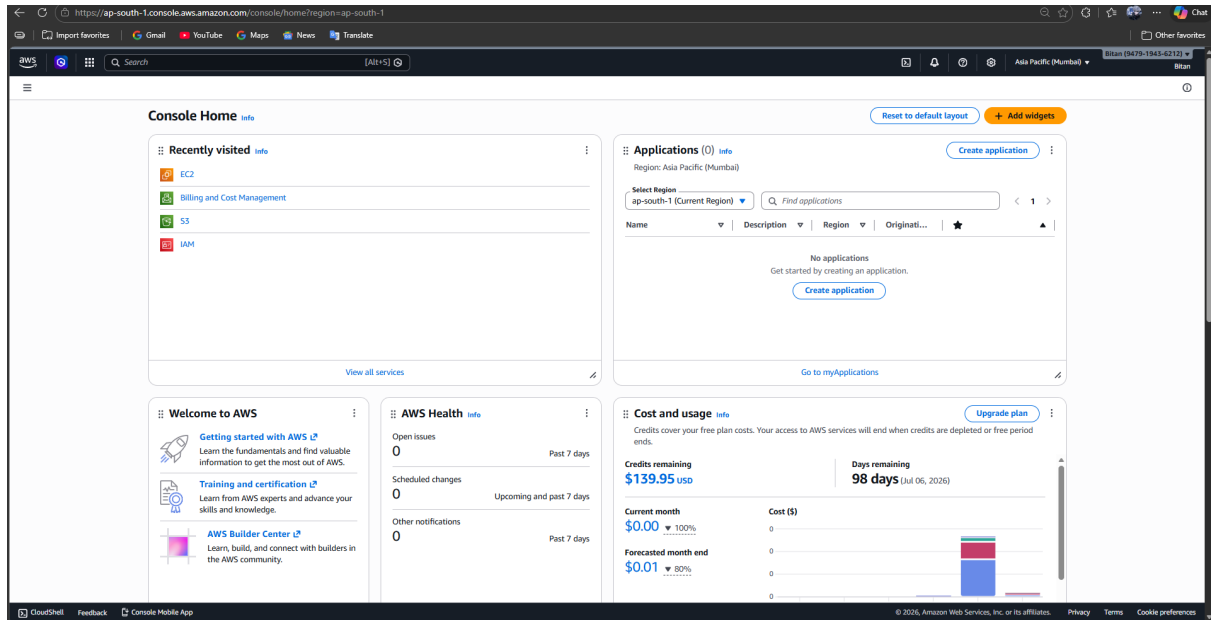
The screenshot shows a web browser with the address bar displaying '13.234.48.174:4000'. The browser shows the output 'Hello mckv'.

ASSIGNMENT – 10

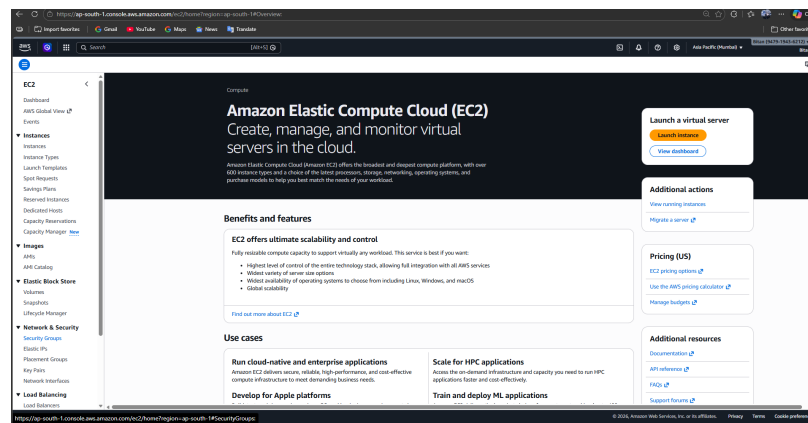
PROBLEM STATEMENT: Deploy a project from GitHub to EC2 by creating a new security group and user data.

Steps:

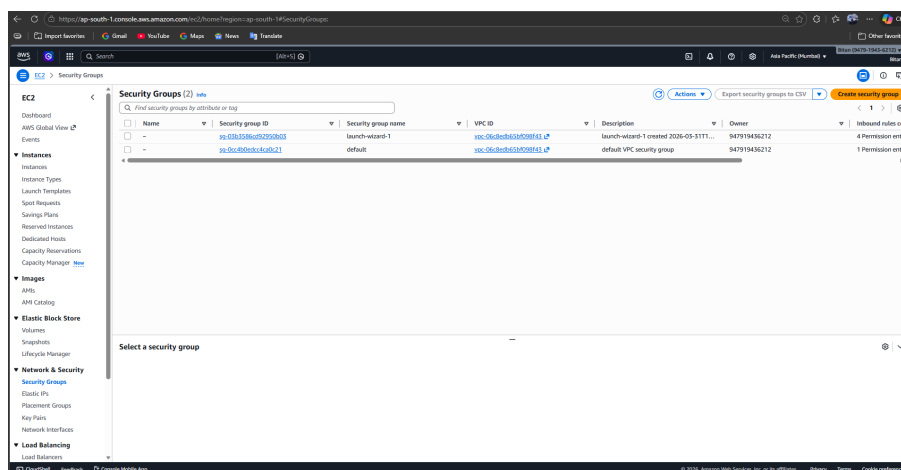
1. Login to the AWS Management Console and from the home page click on "EC2".



2. Now click on "Security groups"



3. Now delete all the existing security groups except the "default" one. Then click on "Create security group" from the top right corner.

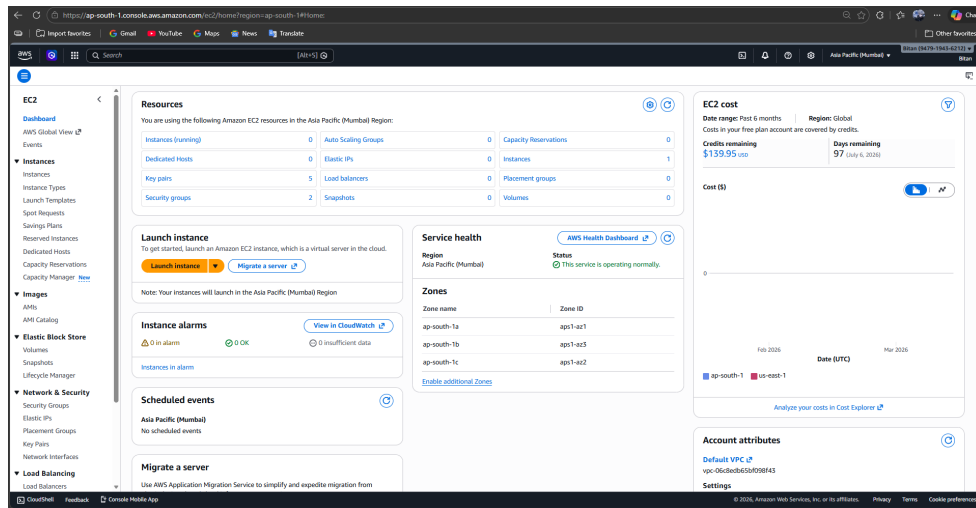


4. Now enter the basic details of the security group. Here security group name is "ProjectSecurity" and Description is "For Security".

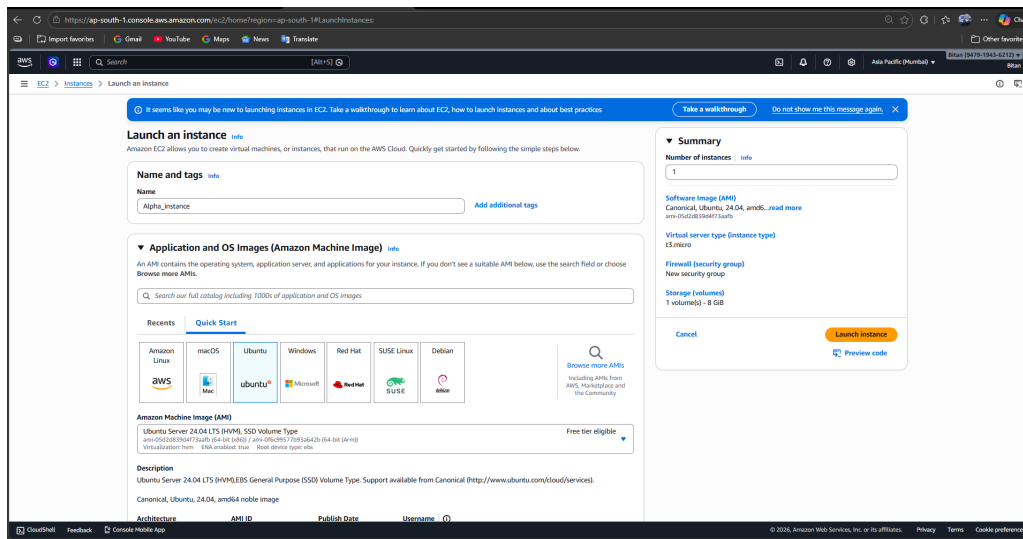
5. Scroll down and add the four inbound rules as shown below. Do not change anything in Outbound rules section. Finally click on "Create Security Group".

6. The new security group "ProjectSecurity" is created successfully.

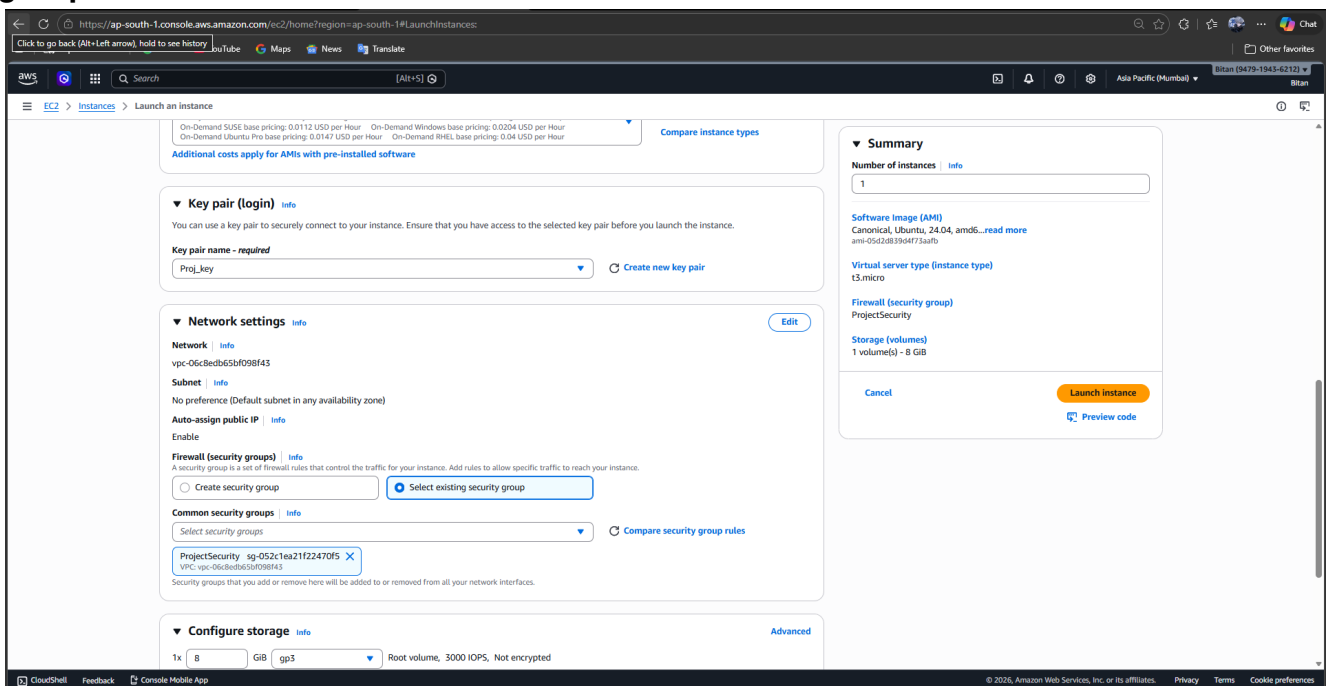
7. Now go back to the EC2 dashboard and click on “Launch Instance”



8. Enter the name of the instance (here Alpha_instance) and select the UBUNTU as the operating system.



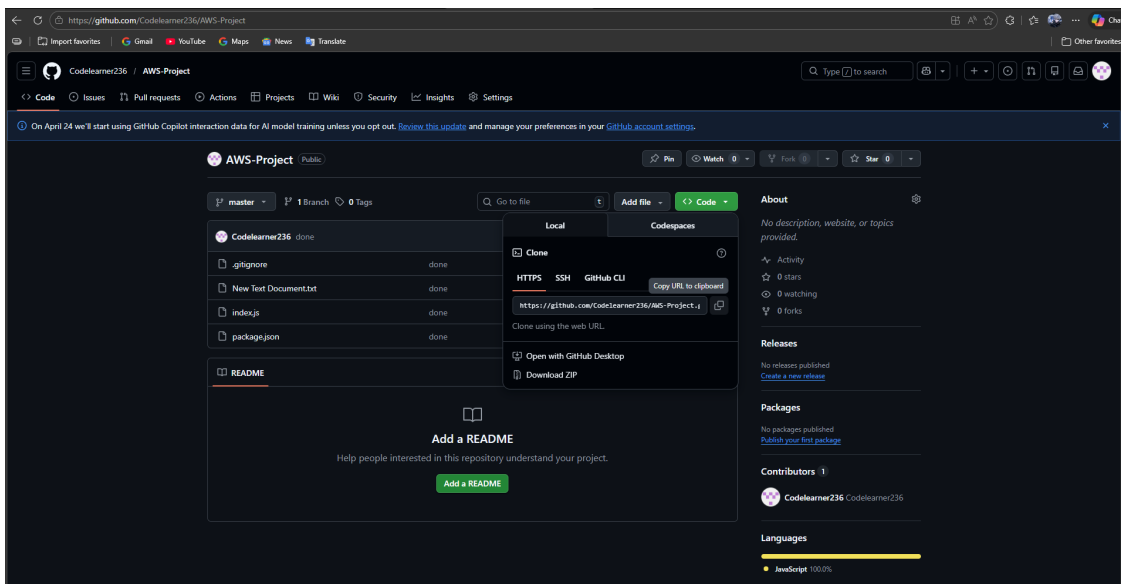
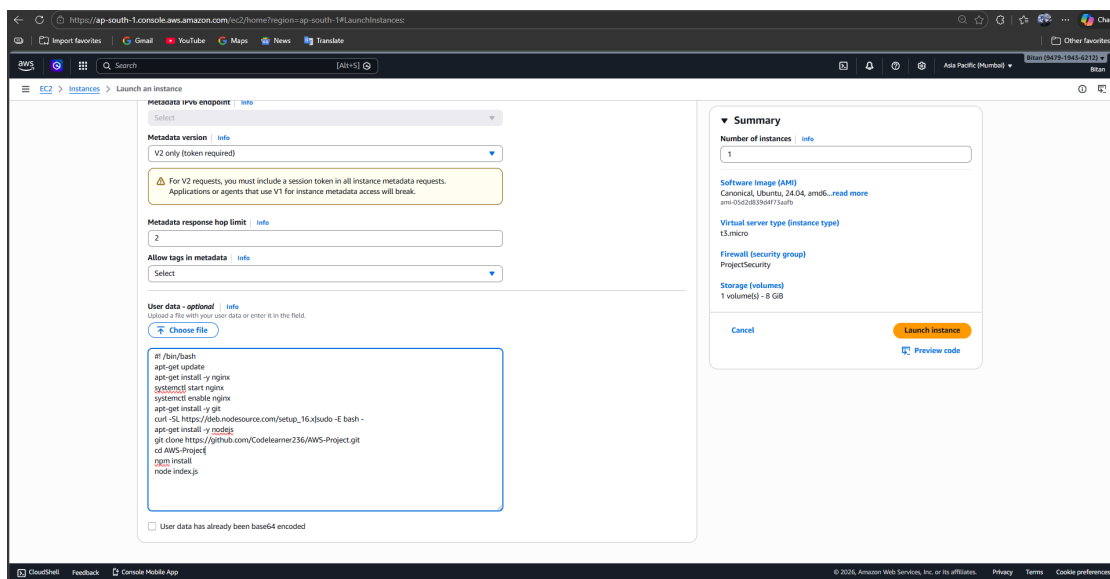
9. Scroll down and select the keypair which is present in your device. If not then create a new keypair. Here keypair selected is “Proj_key” and click on “Select existing security group”.



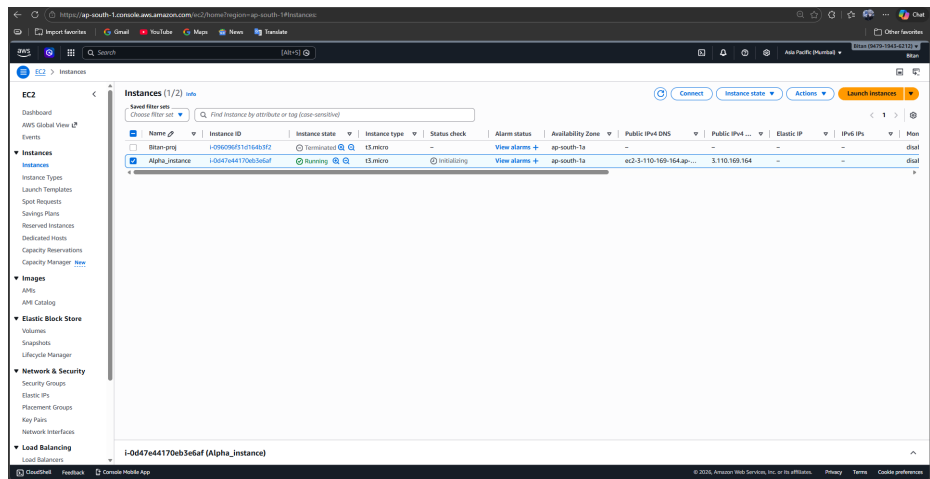
10. Go to Advanced settings and scroll down to the end. Copy paste the given code in the “User data-optional” blue box. After that click on Launch Instance.

```
#!/bin/bash
apt-get update
apt-get install -y nginx
systemctl start nginx
systemctl enable nginx
apt-get install -y git
curl -SL https://deb.nodesource.com/setup_16.x|sudo -E bash -
apt-get install -y nodejs
git clone <your repository address>
cd <your repository folder name>
npm install
node index.js
```

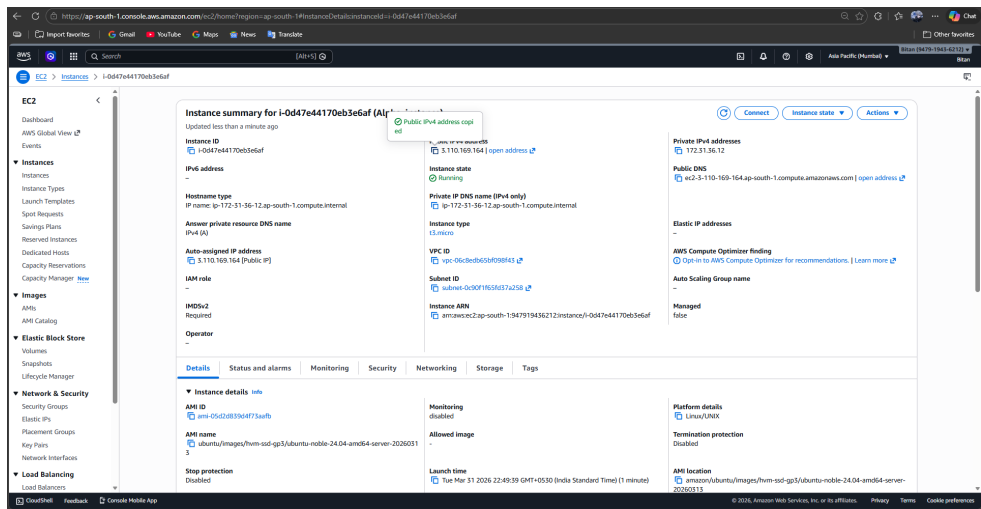
Login to the GitHub account and go to the repository where the projects are stored, now from there click on “code” button and copy the link.



11.Instance (here Alpha_instance) is created successfully.



12.Click on the Instance ID and then copy the Public IPv4 address.



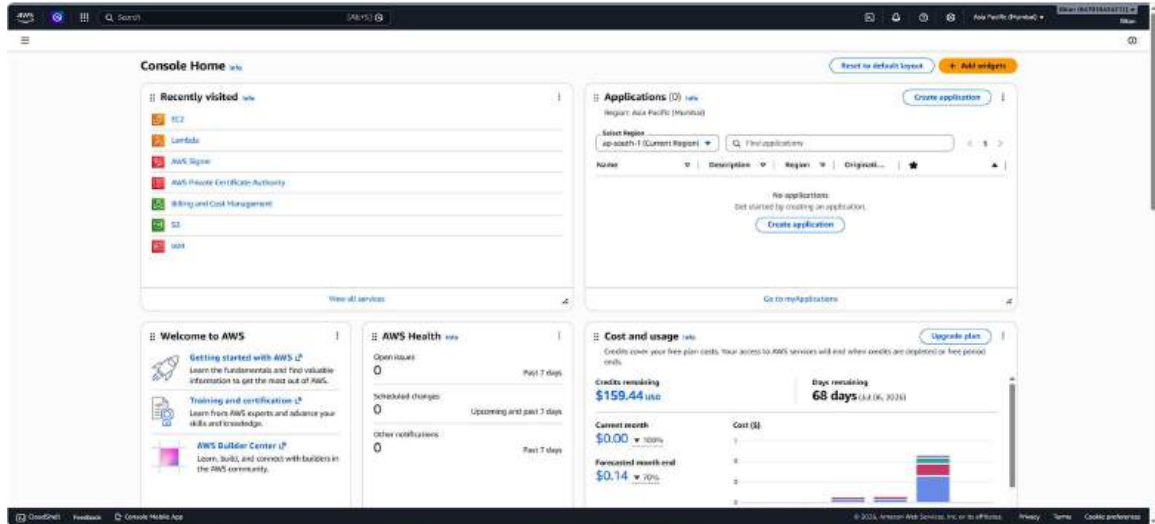
13.And paste it into a new tab add “:4000” to it, then press enter, we can see the content of the project is displayed on the screen i.e.“Hello mckv”.



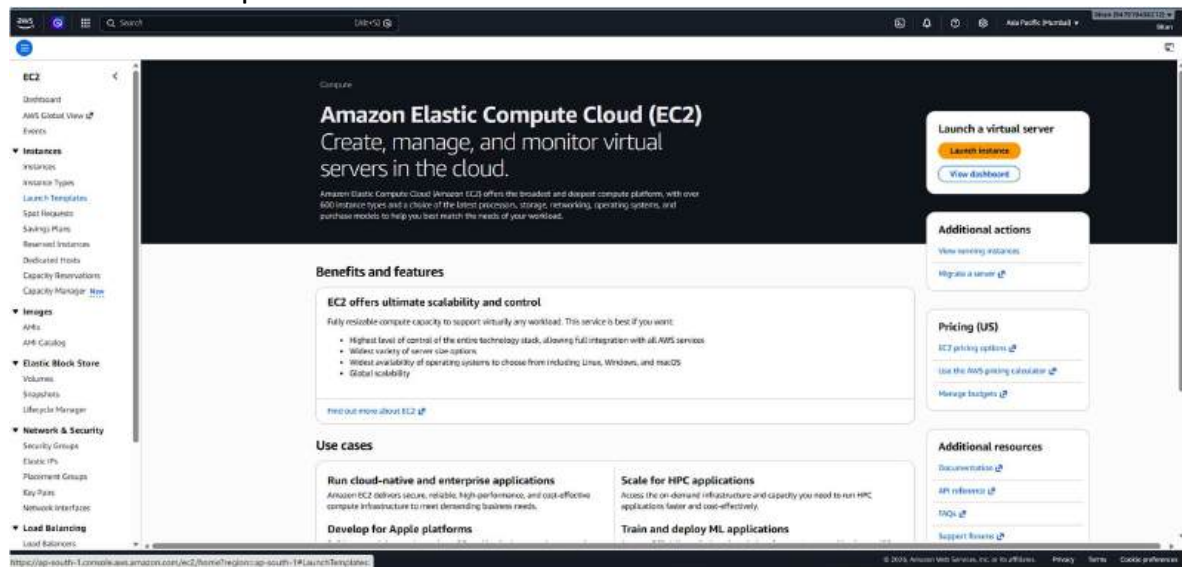
“Thus, seeing the final output, we can say our Project has been deployed from the GitHub to EC2 by creating a new security group (here ProjectSecurity) and the user data (the code which we copied)”

ASSIGNMENT 11 : Build scaling plans in AWS that balance load on different EC2 instances

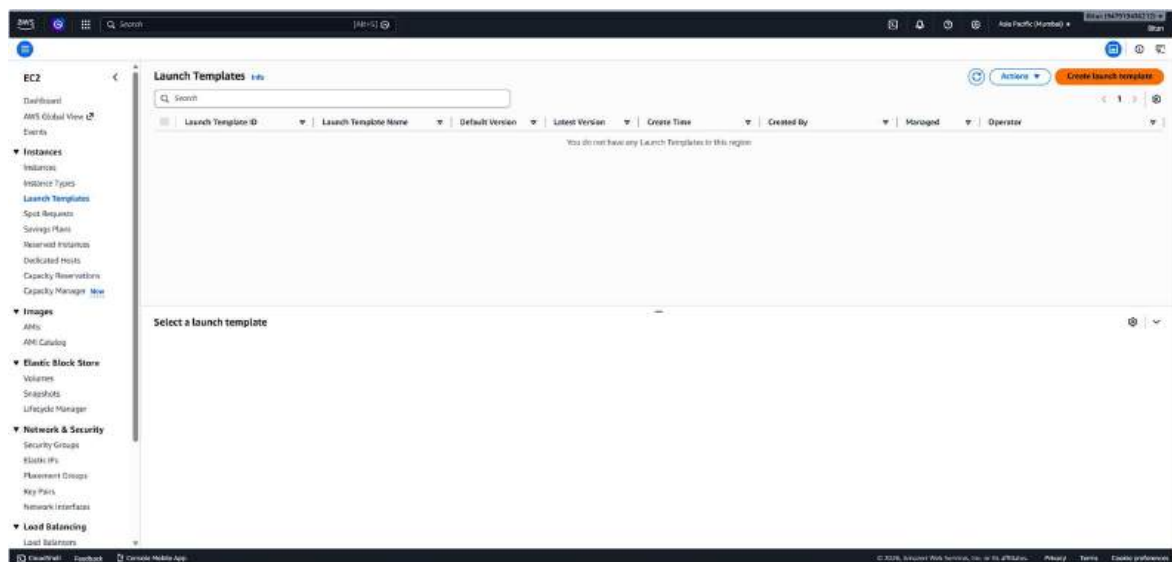
1. Log into AWS console and click on “EC2”.



2. Click on ‘Launch Templates’.



3. Create launch templates.



4. Enter necessary details. In 'Launch templet name and description' give a name and description to your launch templet and select the check box for 'Provide guidance to help me set up a template that I can use with EC2 Auto Scaling'

Create launch template
Creating a launch template allows you to create a saved instance configuration that can be reused, shared, and launched at a later time. Templates can have multiple versions.

Launch template name and description

Launch template name → required
MyTemplate

Template version description
none

Auto Scaling guidance
Provide guidance to help me set up a template that I can use with EC2 Auto Scaling

Launch template contents
Specify the details of your launch template below. Leaving a field blank will result in the field not being included in the launch template.

Application and OS Images (Amazon Machine Image) - required
An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Search our full catalog including 1000s of application and OS images

Recently Quick Start

Amazon Linux, macOS, Ubuntu, Windows, Red Hat, SUSE Linux, Debian

Ubuntu Server 20.04 LTS (HVM), SSD Volume Type

For OS Image select 'Ubuntu Server'

Application and OS Images (Amazon Machine Image) - required
An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Search our full catalog including 1000s of application and OS images

Recently Quick Start

Amazon Linux, macOS, Ubuntu, Windows, Red Hat, SUSE Linux, Debian

Ubuntu Server 20.04 LTS (HVM), SSD Volume Type

Description
Canonical, Ubuntu, 20.04, amd64 release image

Architecture
64-bit (x86)

AWS ID
ami-07d0c1f70bb9464

Publish date
2020-04-21

Username
ubuntu

Instance type
t2.micro

For instance type, select 't2.micro' or any other free tier eligible instance type. For key pair, select an existing key pair or crate a new key pair.

Instance type
t2.micro

Key pair (login)
MyKeyPair

Network settings

For security group select an existing security group where port 4000 is open for incoming traffic. Keep Storage and Resource Tag as default.

The screenshot shows the 'Create launch template' page in the AWS Management Console, specifically the 'Network settings' section. The 'Subnet' dropdown is set to 'Don't include in launch template'. The 'Availability Zone' is set to 'us-east-1a'. The 'Firewall (security group)' dropdown is set to 'Select existing security group'. The 'Security groups' dropdown is set to 'sg-02ac7d379d5446230'. The 'Advanced network configuration' section is expanded. The 'Storage (volumes)' section is also expanded, showing 'EBS Volumes' with 'Volume 1 (AMI Root) - 8 GiB, EBS, General purpose SSD (gp3)'. The 'Resource tags' section is expanded. The 'Summary' section on the right shows the 'Software image (AMI)' as 'Canonical, Ubuntu, 20.04, amd64', 'Virtual server type (instance type)' as 't3.micro', 'Firewall (security group)' as 'Morp1', and 'Storage (volumes)' as '1 volume(s) - 8 GiB'. The 'Create launch template' button is visible.

In Advanced details scroll down to the bottom and add the text to the user data. Then click on Create launch template.

The screenshot shows the 'Create launch template' page in the AWS Management Console, specifically the 'Advanced details' section. The 'Metadata key options' dropdown is set to 'Don't include in launch template'. The 'Metadata version' dropdown is set to 'V2 only (broken required)'. The 'Metadata response page limit' is set to '2'. The 'Allow tags in metadata' dropdown is set to 'Don't include in launch template'. The 'User data - optional' section is expanded, showing a text area with the following content:

```
#bin/bash
apt-get update
apt-get install -y nginx
systemctl start nginx
systemctl enable nginx
apt-get install -y git
curl -SL https://d16vradp9csm.com/ubuntu_16.x64de -C bash -
apt-get install -y mc
git clone https://github.com/Codecademy/235/MWS-Project.git
cd AWS-Project
git install
make install
```

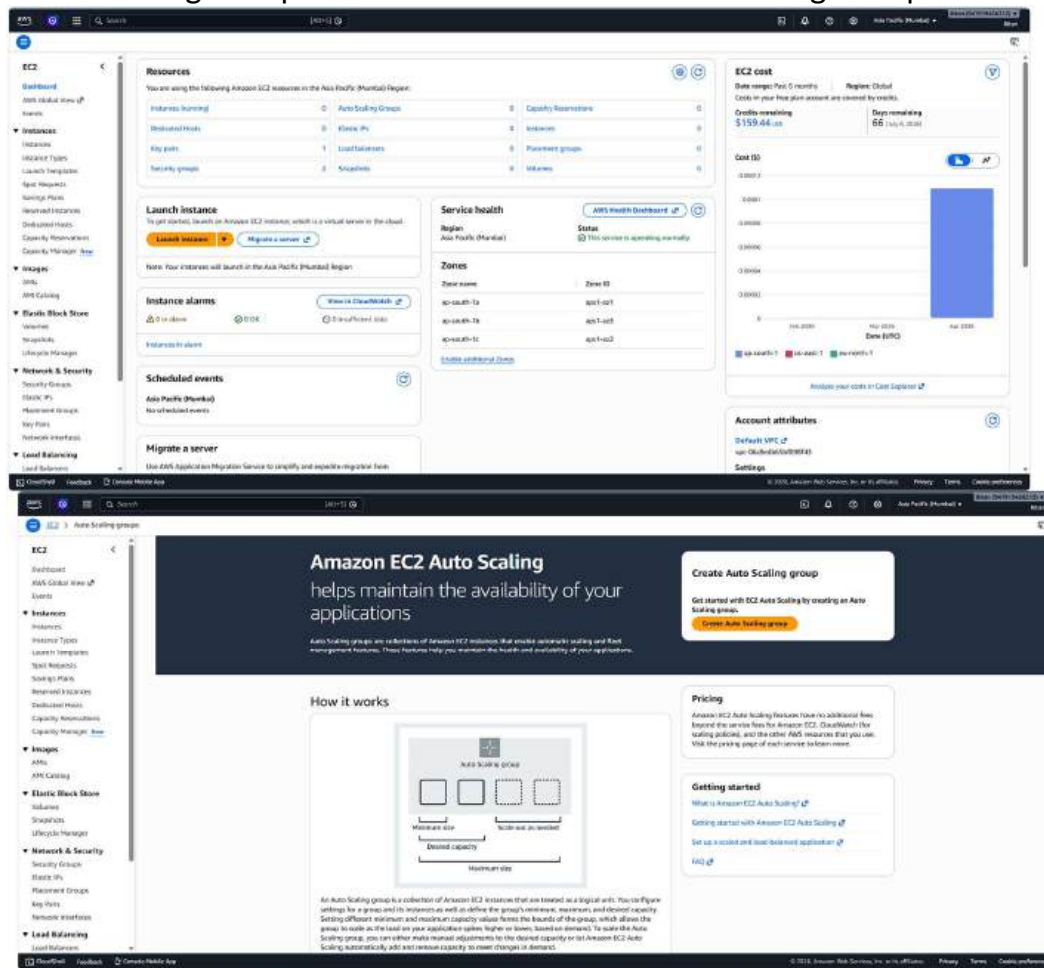
 The 'Create launch template' button is visible.

Launch Templates created successfully.

The screenshot shows the 'Launch Templates' page in the AWS Management Console. The table lists the following launch template:

Launch Template ID	Launch Template Name	Default Version	Latest Version	Create Time	Created By	Managed	Operator
lt-0005g794ldch7n0sf	Morpola	1	1	2025-05-01T05:54:43.000Z	awsiam:arn:aws:iam::547915436212:user	false	-

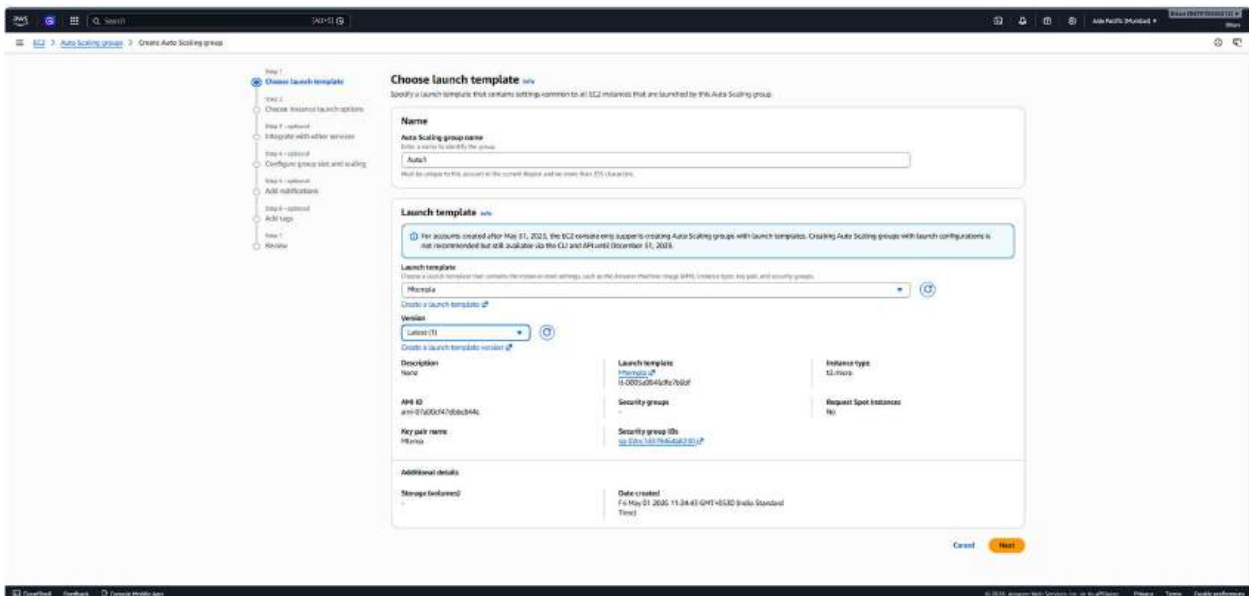
6. Now go to 'Auto Scaling Groups'. Then click on 'Create Auto Scaling Group'.



7. Add Details for the Auto Scaling Group.

Step 1: Choose launch templates- Give an name to the Auto Scaling Group and choose the Launch template you just created as the launch template.

Click on Next.



Step 2: Choose instance launch options- Choose all the ‘Availability Zones’ available. Click on Next.

This screenshot shows the 'Choose instance launch options' step in the AWS Management Console. The left sidebar contains a navigation menu with steps 1 through 7. The main content area is titled 'Choose instance launch options' and includes several sections: 'Instance type requirements' with a table showing 'Launch template' (t3.micro), 'Instance type' (t3.micro), and 'Description' (None); 'Network' with a 'VPC' dropdown set to 'vpc-00b0d090908043' and 'Availability zones and subnets' with three subnets selected; and 'Availability zone distribution' with the 'Balanced across all' radio button selected.

Step 3: Configure advanced options- For Load balancing select ‘Attach to a new load balancer’

This screenshot shows the 'Integrate with other services' step in the AWS Management Console. The left sidebar shows steps 1 through 7. The main content area is titled 'Integrate with other services - optional' and includes a 'Load balancing' section with three radio buttons: 'No load balancer', 'Attach to an existing load balancer', and 'Attach to a new load balancer' (which is selected). Below this, the 'Attach to a new load balancer' section shows fields for 'Load balancer type' (Application Load Balancer), 'Load balancer name' (Auto-1), 'Load balancer scheme' (Internal), and 'Network mapping' (Internal). The 'Availability zones and subnets' section shows three subnets selected.

For new load balancer configuration select ‘Application Load Balancer’, ‘Internet-facing’ and make HTTP port from 80 to 4000 while creating a target group.

This screenshot shows the 'Listeners and routing' step in the AWS Management Console. The left sidebar shows steps 1 through 7. The main content area is titled 'Listeners and routing' and includes a 'Listeners and routing' section with a 'Port' dropdown set to '4000' and a 'Default routing (Forward rule)' section with a 'Create a target group' button. Below this, the 'VPC Lattice integration options' section shows the 'No VPC Lattice service' radio button selected. The 'Application Recovery Controller (ARC) zone shift' section is also visible at the bottom.

Health checks tick the check box for 'Turn on Elastic Load Balancing health checks' and make the health check grace period 180 seconds. Click on Next.

This screenshot shows the 'Health checks' section of the 'Create Auto Scaling group' wizard. The 'Health checks' section is active, showing options for 'EC2 Auto Scaling' and 'Application Recovery Controller (ARC) zonal shift'. The 'EC2 Auto Scaling' section is expanded, showing the 'Turn on Elastic Load Balancing health checks' checkbox, which is checked. Below this, the 'Health check grace period' is set to 180 seconds. The 'Next' button is visible at the bottom right.

Step 4: Configure group size and scaling- In Group size set Desired capacity as 2. In Scaling set Min desired capacity as 2 and Max desired capacity as 3. In Automatic scaling choose target scaling policy and set the instance warmup as 180 second. Keeping everything else as default, Click on Next

This screenshot shows the 'Configure group size and scaling' section of the 'Create Auto Scaling group' wizard. The 'Group size' section is active, showing the 'Desired capacity' set to 2. The 'Scaling' section is also active, showing 'Min desired capacity' set to 2 and 'Max desired capacity' set to 3. The 'Automatic scaling - optional' section is expanded, showing the 'Target tracking scaling policy' checkbox, which is checked. Below this, the 'Scaling policy name' is set to 'Target Tracking Policy'. The 'Metric type' is set to 'Average CPU utilization' and the 'Target value' is set to 20. The 'Instance warmup' is set to 180 seconds. The 'Next' button is visible at the bottom right.

Skip through Step 5 and Step 6 by clicking Next.

This screenshot shows the 'Add notifications - optional' step in the AWS Auto Scaling group creation wizard. The left sidebar lists steps 1 through 7, with step 5 highlighted. The main content area has the title 'Add notifications - optional' and a description: 'Send notifications to SNS topics whenever Amazon EC2 Auto Scaling launches or terminates the EC2 instances in your Auto Scaling group.' There is an 'Add notification' button. At the bottom right, there are 'Cancel', 'Skip to review', 'Previous', and 'Next' buttons. The 'Next' button is highlighted in orange.

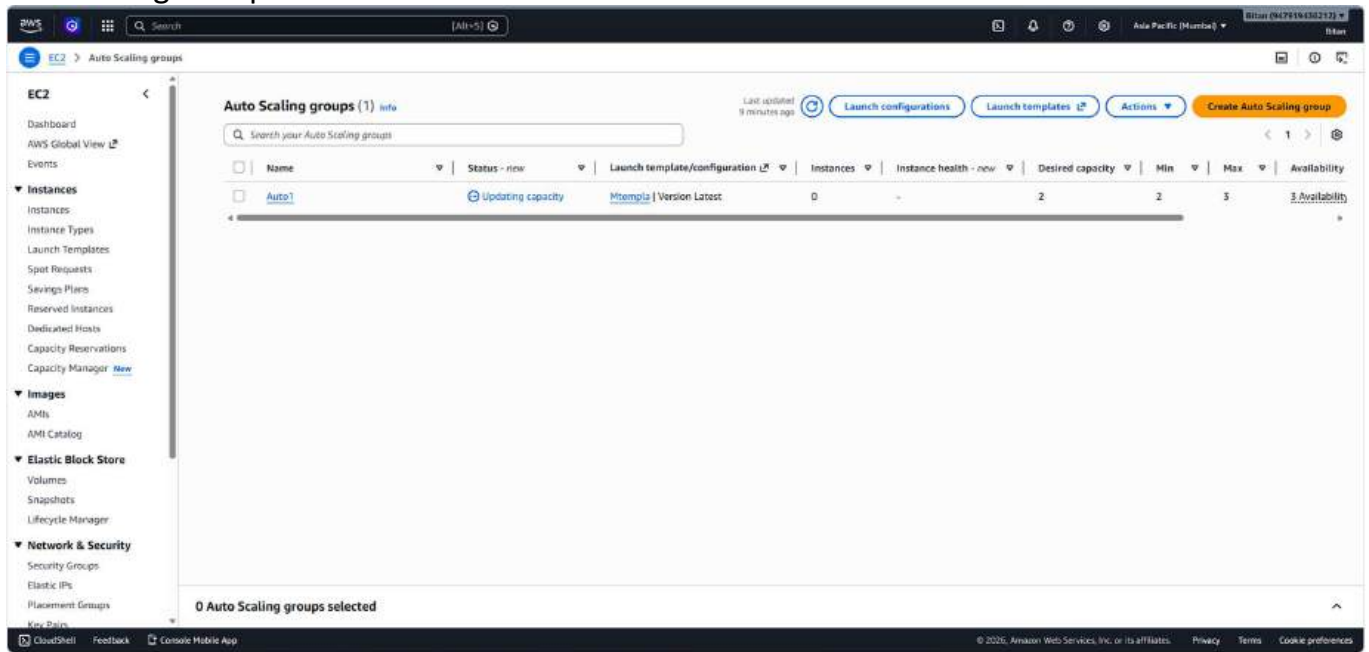
This screenshot shows the 'Add tags - optional' step in the AWS Auto Scaling group creation wizard. The left sidebar lists steps 1 through 7, with step 6 highlighted. The main content area has the title 'Add tags - optional' and a description: 'Add tags to help you search, filter, and track your Auto Scaling group across AWS. You can also choose to automatically add these tags to instances when they are launched.' There is a warning box stating: 'You can optionally choose to add tags to instances (and their attached EBS volumes) by specifying tags in your launch template. We recommend caution, however, because the tag values for instances from your launch template will be overridden if there are any duplicate keys specified for the Auto Scaling group.' Below this is a 'Tags (0)' section with 'No tags' and an 'Add tag' button. At the bottom right, there are 'Cancel', 'Previous', and 'Next' buttons. The 'Next' button is highlighted in orange.

Finally in Step 7 Click on Create Auto Scaling Group.

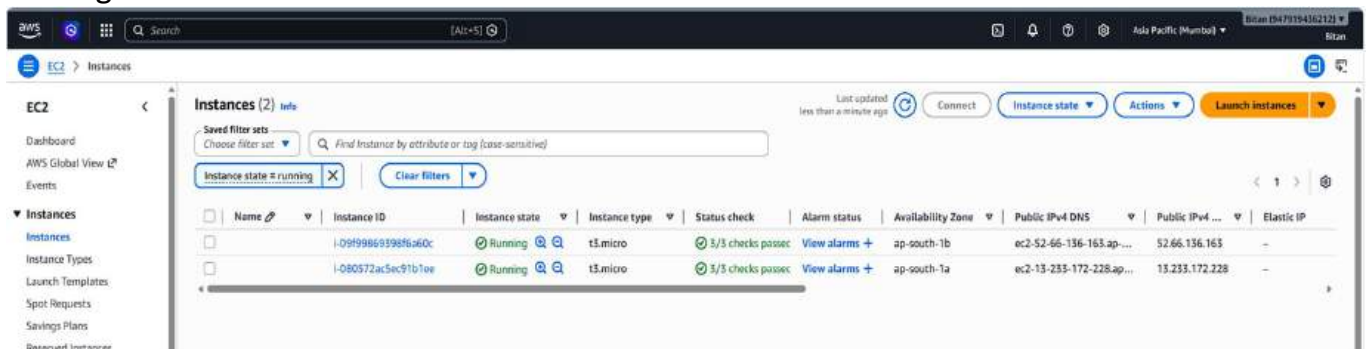
This screenshot shows the 'Review' step in the AWS Auto Scaling group creation wizard. The left sidebar lists steps 1 through 7, with step 7 highlighted. The main content area has the title 'Review' and two sections: 'Step 1: Choose launch template' and 'Step 2: Choose instance launch options'. The 'Step 1' section shows 'Group details' with 'Auto Scaling group name' as 'Auto1' and 'Launch template' as 'M3tenvyla'. The 'Step 2' section shows 'Network' details, including VPC 'vpc-06c8ed065b1f098f43', and a table of 'Availability Zones and subnets'. At the bottom right, there are 'Edit' buttons for both steps. The 'Next' button is highlighted in orange.

Availability Zone	Subnet	Subnet CIDR range
ap-s1-az1 (ap-south-1a)	subnet-0c90f1f65fd57a258	172.31.32.0/20
ap-s1-az2 (ap-south-1c)	subnet-03ba0f4991eaf17cb	172.31.16.0/20
ap-s1-az3 (ap-south-1b)	subnet-012c01263d8f2f69b	172.31.0.0/20

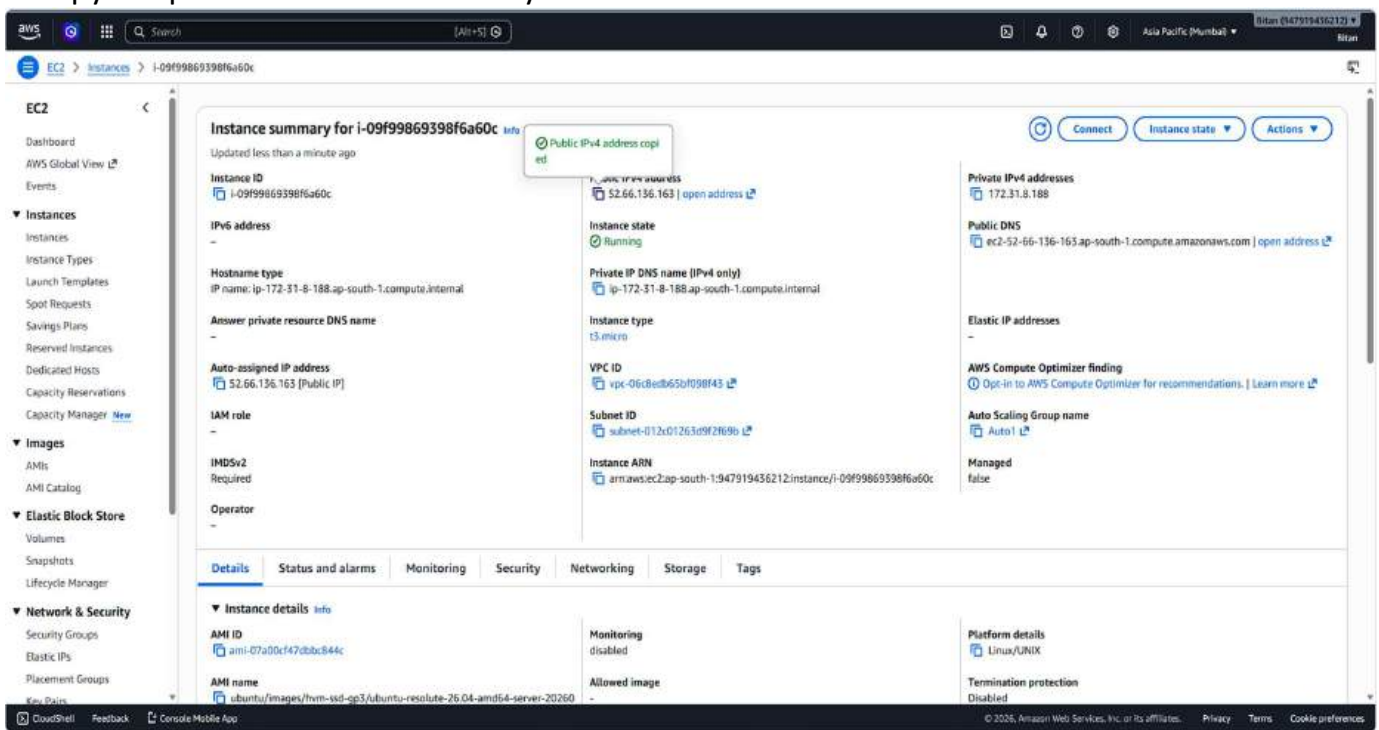
Auto Scaling Group is created.



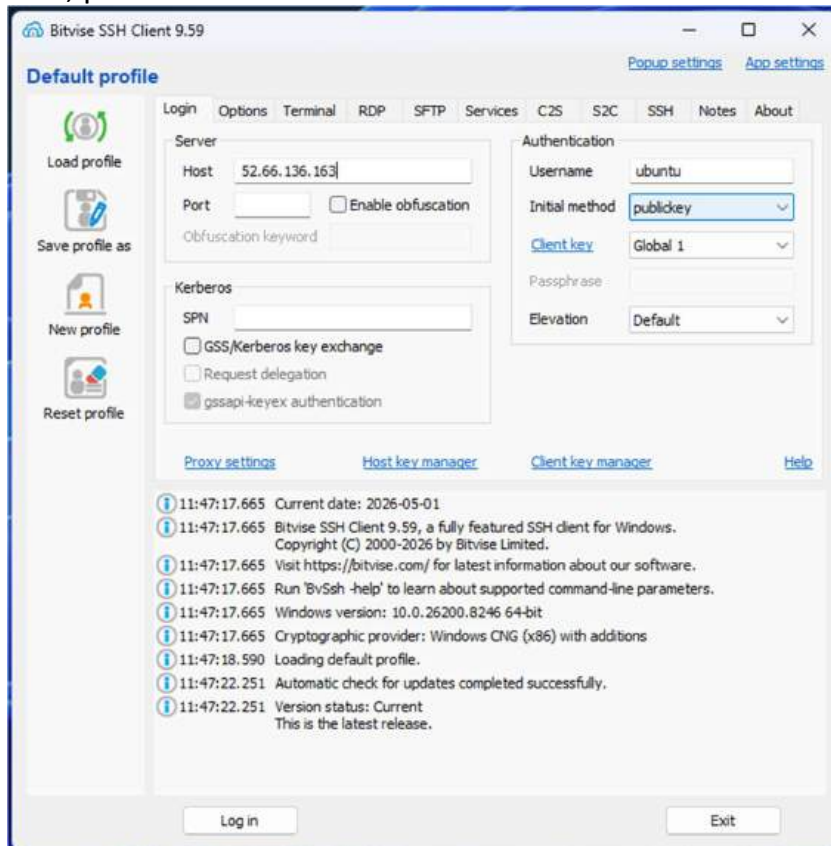
7. Going back to instances we can see two unnamed instances are created.



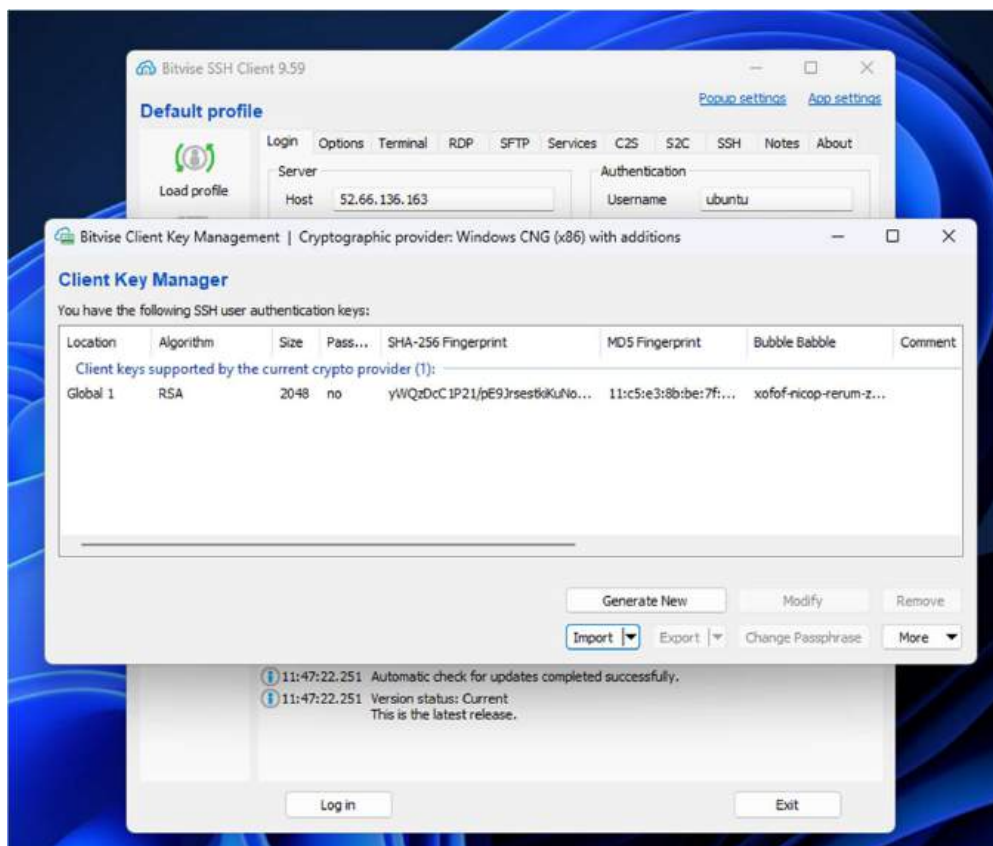
8. Copy the public IPv4 address of any one of the two instance.



9. In Bitvise SSH Client, paste the 'Public IPV4 address' and click on 'Client key Manager'.

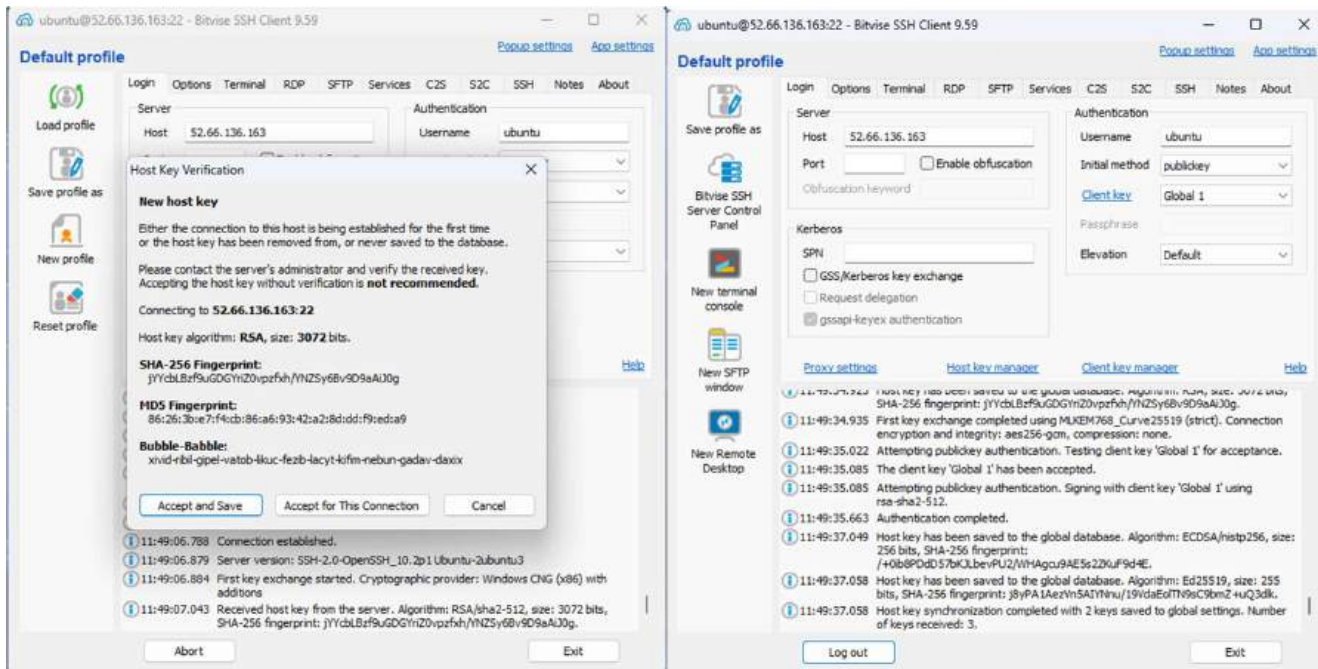


10. Under 'Client key manager', if there is any existing key then remove it and click on 'Import', select the same key that you used while creating the instance, from your file directory and then again click on 'Import' and then the key is successfully imported.



10. In 'Bitwise SSH Client', under 'Default profile' give the 'Username' as 'ubuntu' then select 'Global1' from 'Client Key Manager' then click on 'Log in' & 'Accept and save'.

If the 'Log In' button changes to 'Log Out', then you are logged in successfully.



11. Click on 'New terminal consol', to access the terminal of your instance.

Once you are inside the terminal of your instance

To create a infinitely running bash script vim infi.sh

```
ubuntu@ip-172-31-43-69:~$ sudo nano infi.sh
```

In the bash script, add the following code.

```
GNU nano 2.7.1 infi.sh
#!/bin/bash
while(true)
do
    echo "Inside loop"
done
```

To make the script executable

sudo chmod 777 infi.sh

```
ubuntu@ip-172-31-43-69:~$ sudo nano infi.sh
ubuntu@ip-172-31-43-69:~$ sudo chmod +x infi.sh
```

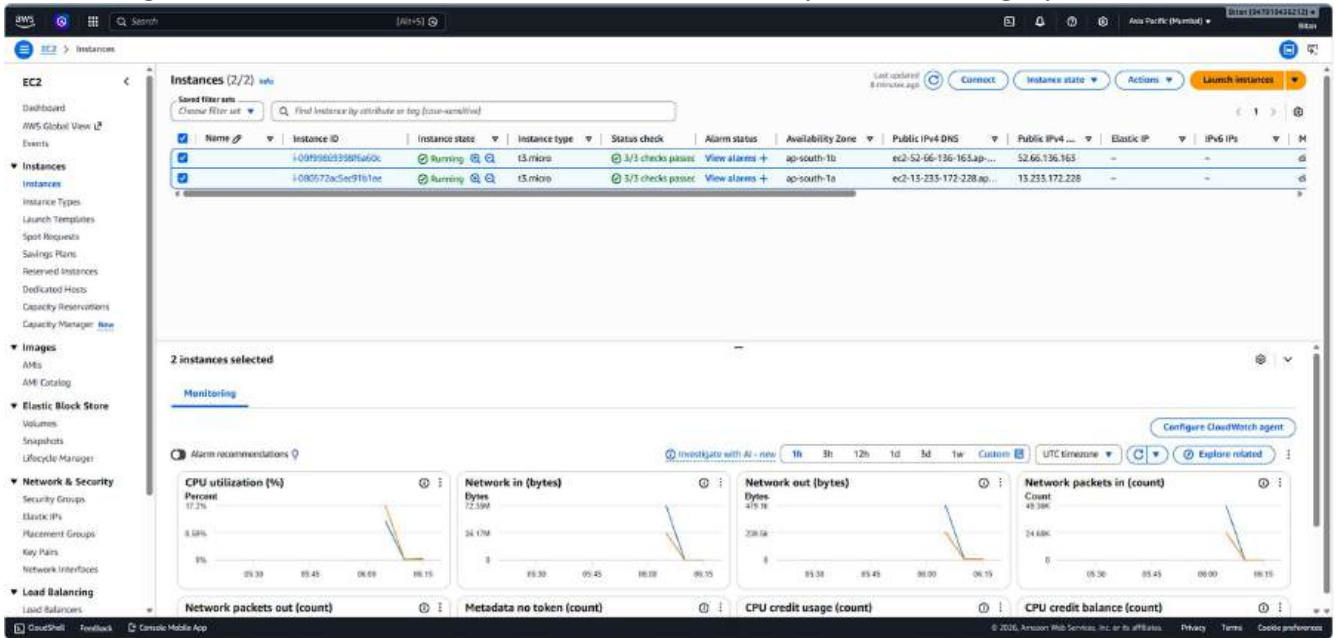
Run the script sh
infi.sh

```
ubuntu@ip-172-31-43-69:~$ sh infi.sh
```

12. An infinite loop starts to run, stressing the cpu of the instance.



13. Selecting the two instances we can take a look at the cpu utilisation graph.



14. After a while the instance reaches and surpasses it's 50% CPU utilisation threshold.



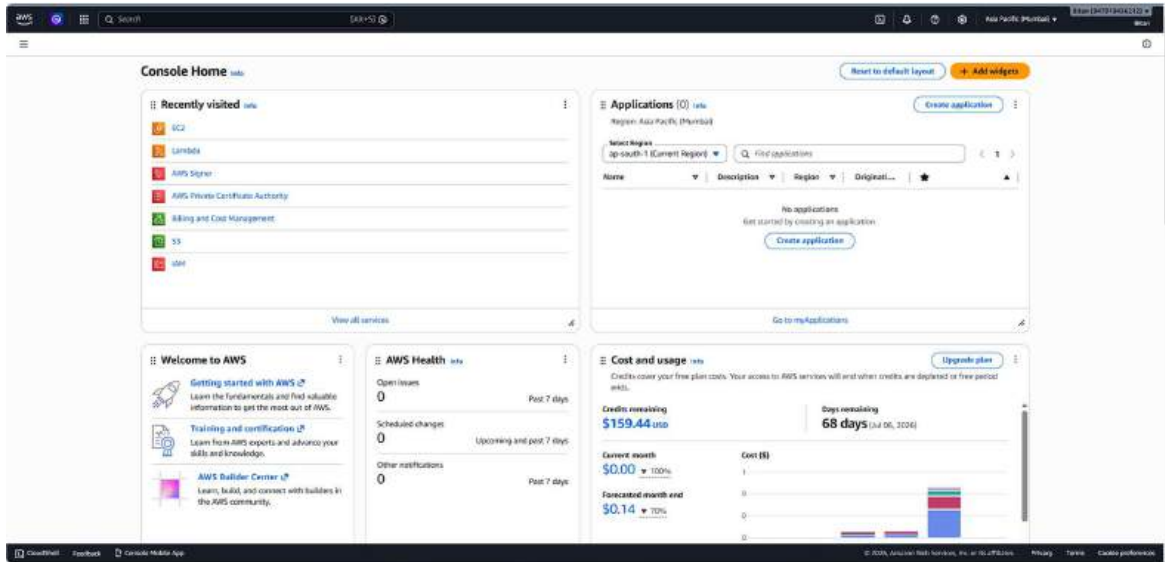
15. So to take that load off the instance, the auto scaler spins up another instance to balance the load.

The screenshot shows the AWS Management Console for the EC2 Instances page. The table lists three instances:

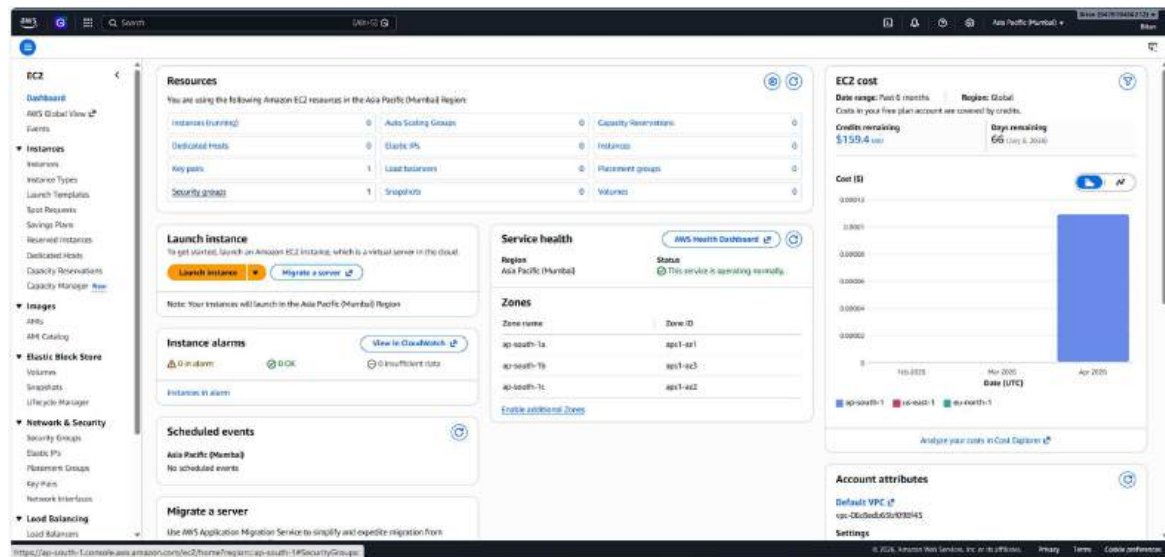
Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
i-0145d3472f94a300	i-0145d3472f94a300	Running	t3.micro	Initializing	View alarms +	ap-south-1a	ec2-3-109-124-100.ap...	3.109.124.100	-	-
i-09f996693b8f6460c	i-09f996693b8f6460c	Terminated	t3.micro	-	View alarms +	ap-south-1a	-	-	-	-
i-080572ac5ed1b1ee	i-080572ac5ed1b1ee	Running	t3.micro	3/3 checks pass	View alarms +	ap-south-1a	ec2-3-233-172-228.ap...	3.233.172.228	-	-

ASSIGNMENT 12 : Deploy and run the project in AWS without using the port.

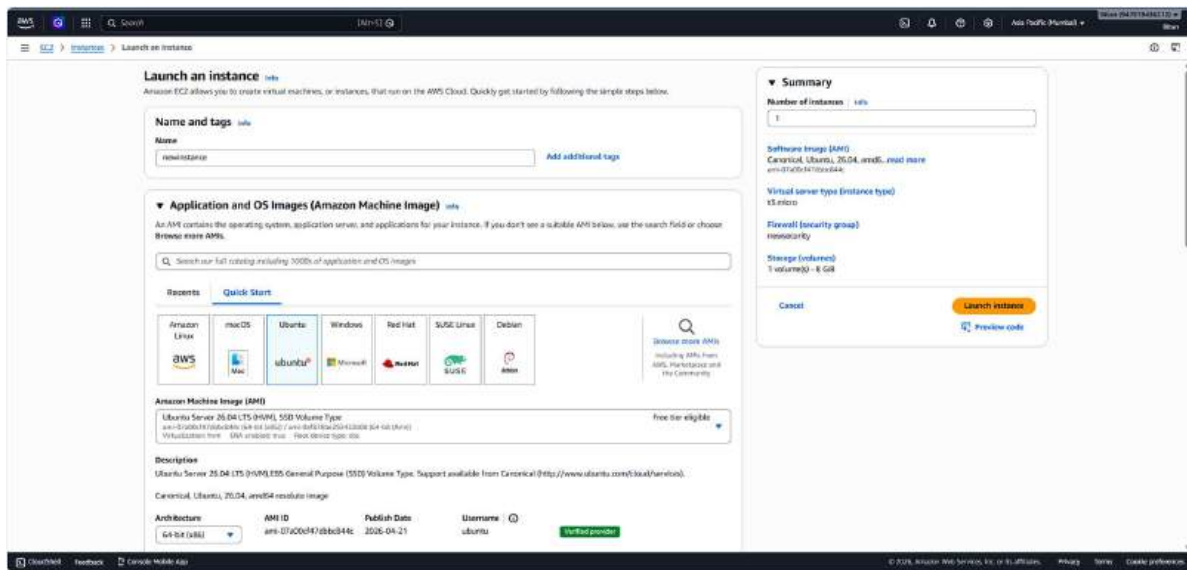
1: Go to the EC2 dashboard.



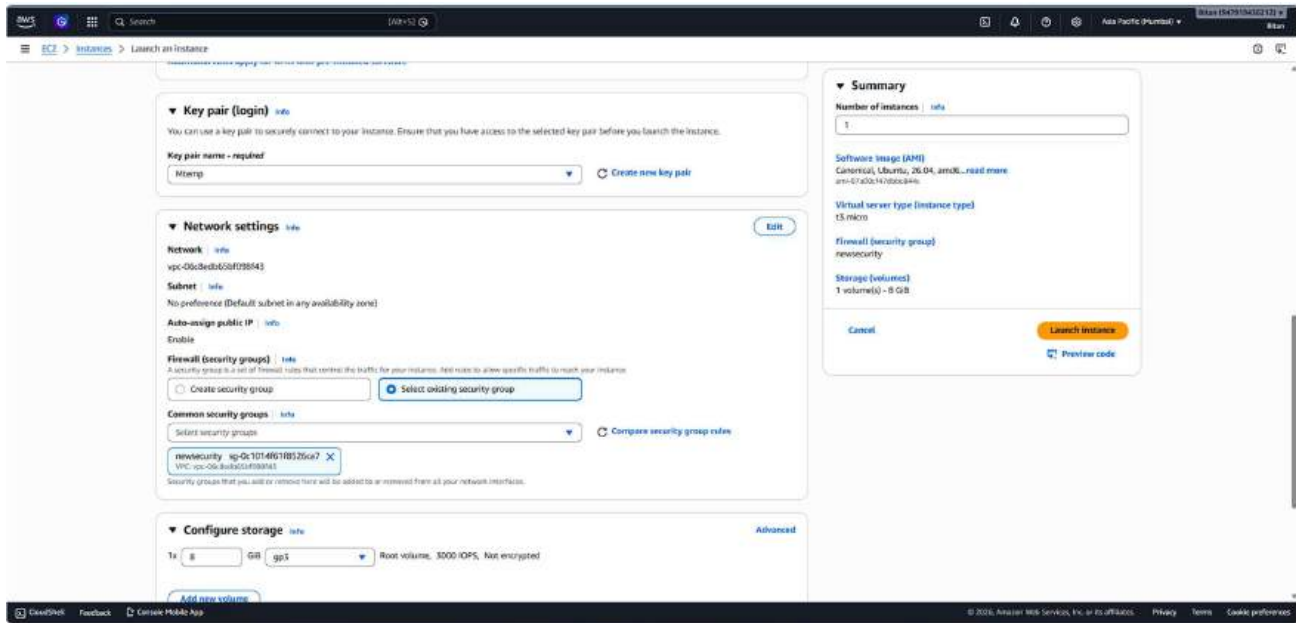
2. Click on Launch Instance.



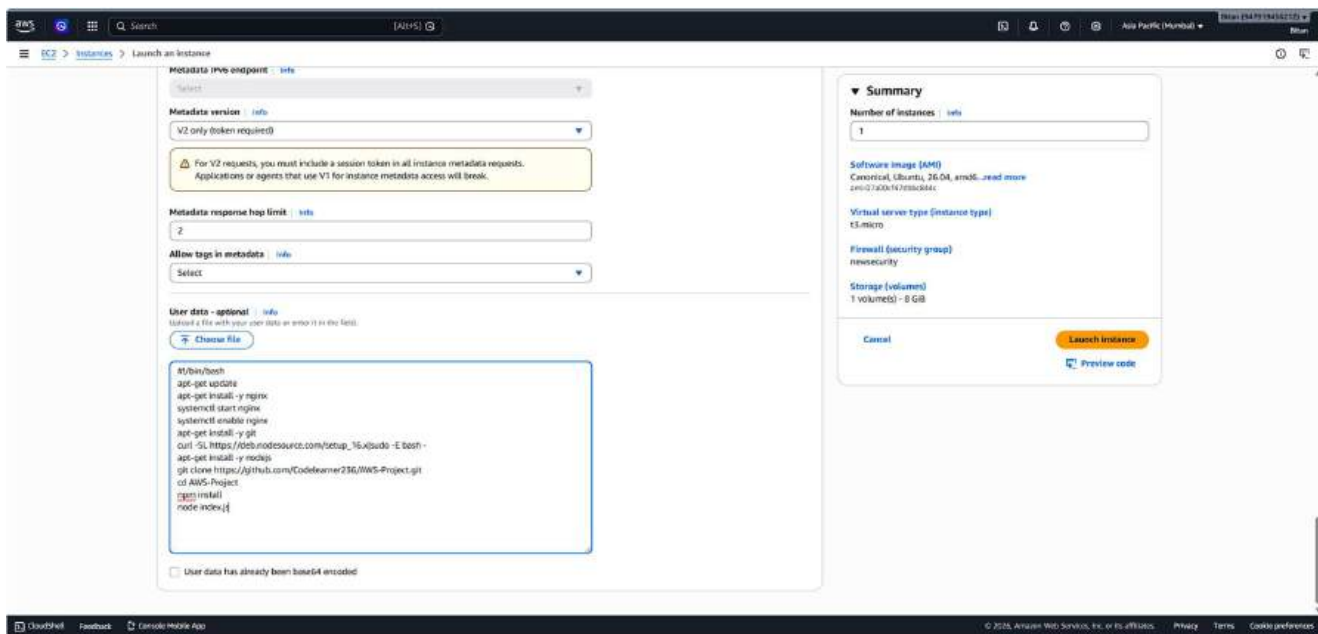
3. Give the name of the instance and select UBUNTU as the OS.



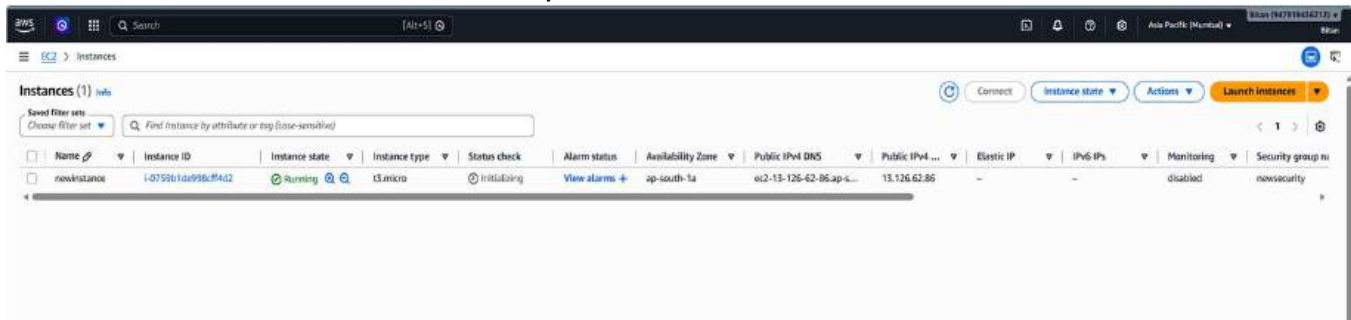
4. Select the keypair and select existing security group. (If not created then create the both).



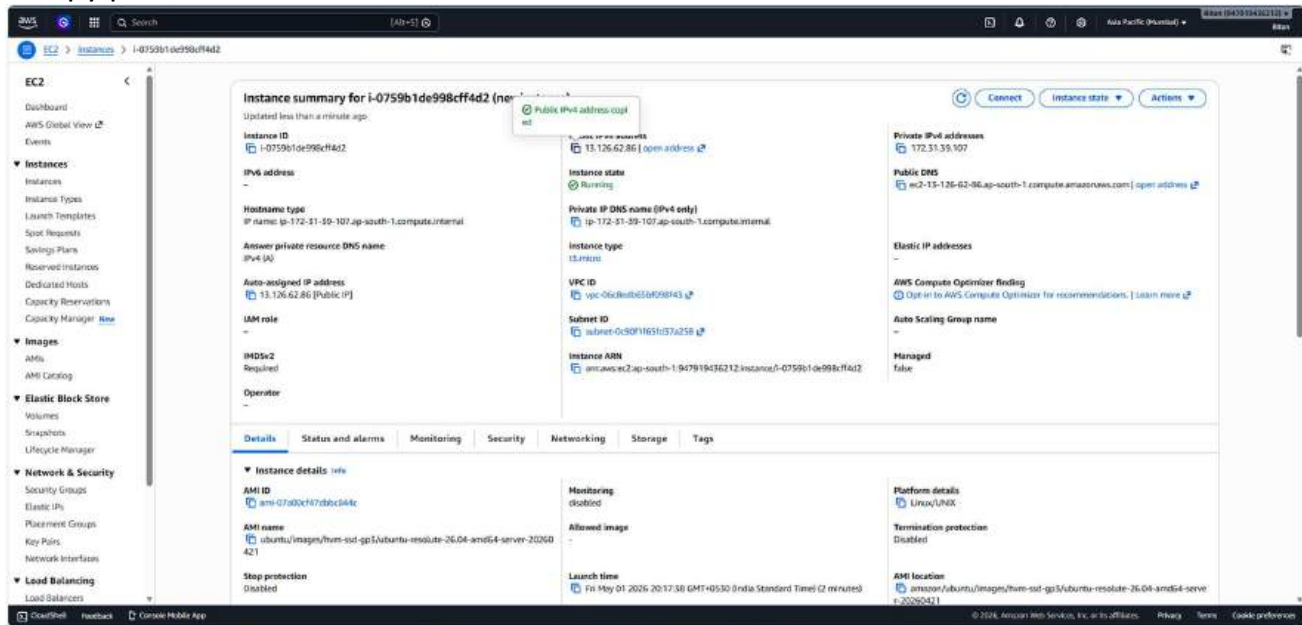
5. In the Advanced settings scroll down and write the code shown in the picture. Then click on “Launch Instance”.



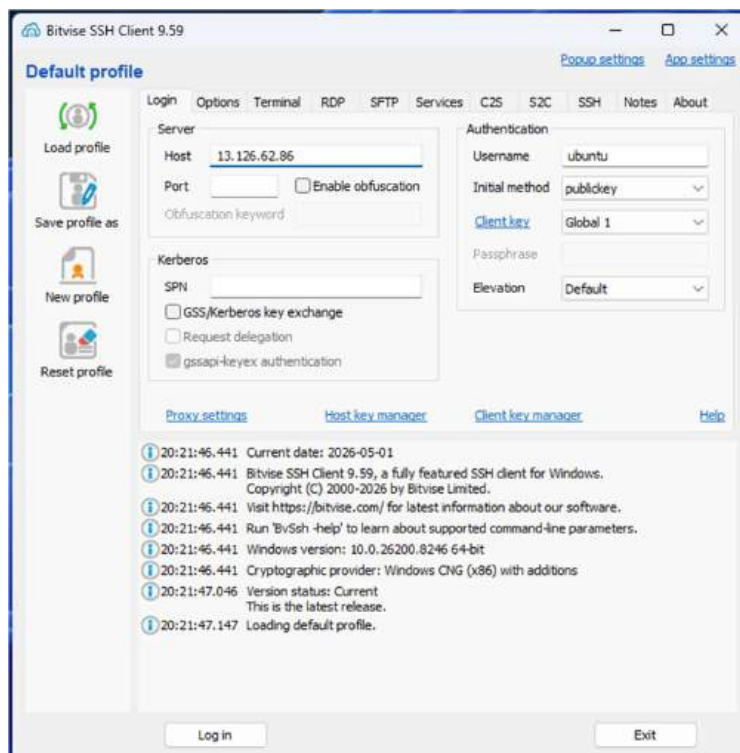
6. The instance is created successfully.



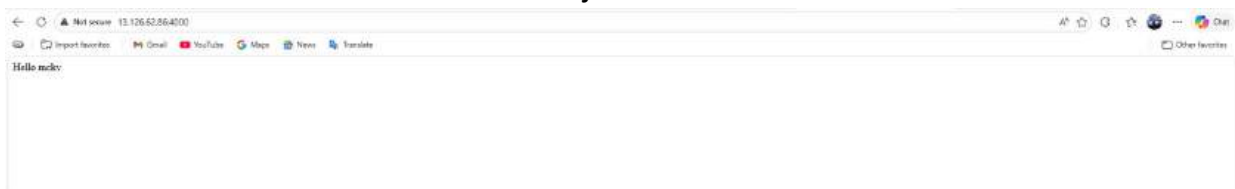
7. Copy paste the Public IPv4 address of the created instance.



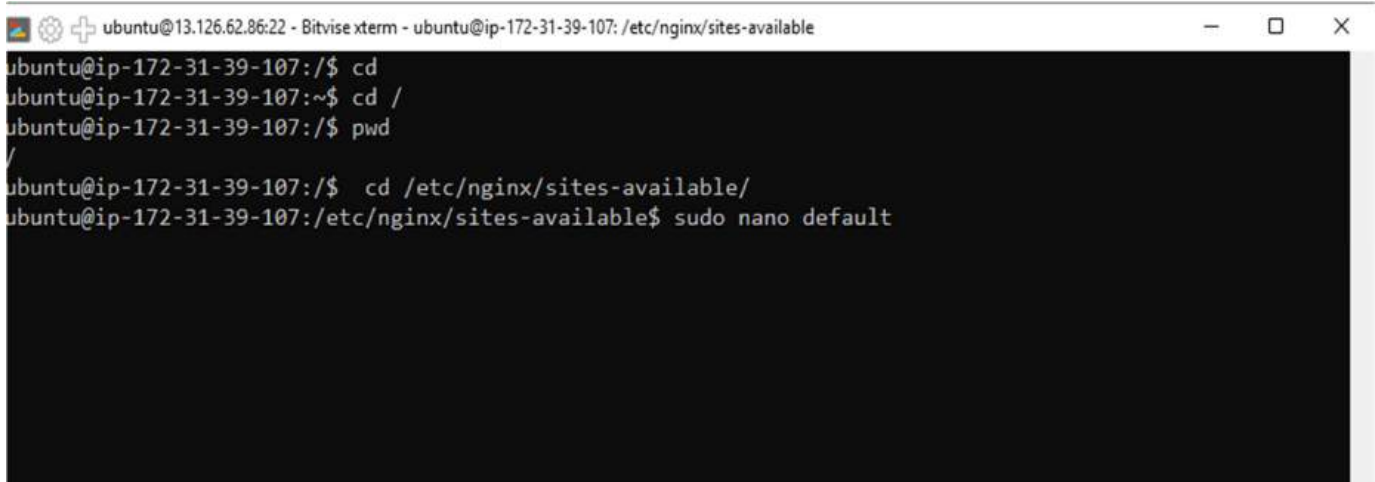
8. Open the Bitvise SSH client and paste the ipv4 address here. Under Authentication select username as “ubuntu”, initial method as “public key” , client key as “Global 1” and in client key manager import the keypair selected while creating the instance. Then click on “Log in” then Accept and Save.



9. Paste the copied ipv4 address on the new tab. Add 4000 (the port number) at the last of the address to see the content of the index.js file.



10. Go to the new terminal window from Bitvise SSH client and write the commands shown in the picture.



```
ubuntu@13.126.62.86:22 - Bitvise xterm - ubuntu@ip-172-31-39-107: /etc/nginx/sites-available
ubuntu@ip-172-31-39-107:/$ cd
ubuntu@ip-172-31-39-107:~$ cd /
ubuntu@ip-172-31-39-107:/$ pwd
/
ubuntu@ip-172-31-39-107:/$ cd /etc/nginx/sites-available/
ubuntu@ip-172-31-39-107:/etc/nginx/sites-available$ sudo nano default
```

11. Uncomment the code shown in the picture.

```
#     location / {
#         # First attempt to serve request as file, then
#         # as directory, then fall back to displaying a 404.
#         try_files $uri $uri/ =404;
#     }

#     # pass PHP scripts to FastCGI server
#
```

12. Add the following code instead. Then Ctrl + X ↵ Y ↵ Enter. You will come back to bash terminal.

```
root /var/www/html;

# Add index.php to the list if you are using PHP
index index.html index.htm index.nginx-debian.html;

server_name _;
location / {
    proxy_pass http://localhost:4000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
    proxy_cache_bypass $http_upgrade;
}
```

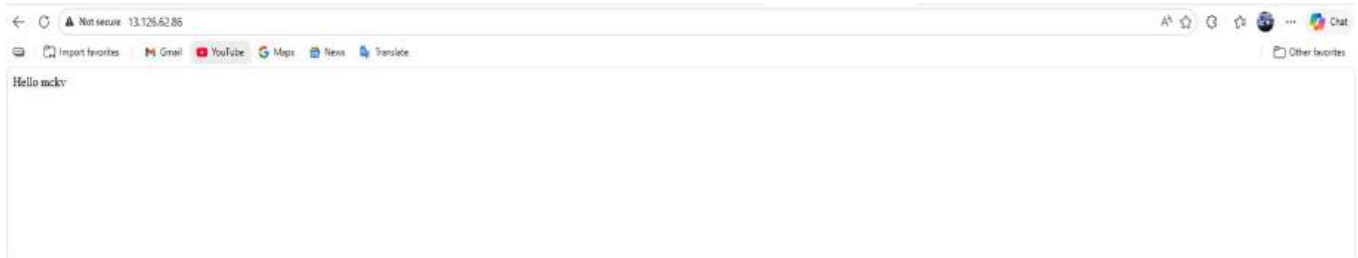
13. Write the code : `sudo systemctl restart nginx`

Go back to the created instance in AWS and refresh it.

```
ubuntu@ip-172-31-39-107:/etc/nginx/sites-available$ sudo systemctl restart nginx
```

14. Copy paste the ipv4 address again in the new tab without adding the port number 4000 to it.

You will get your desired output.

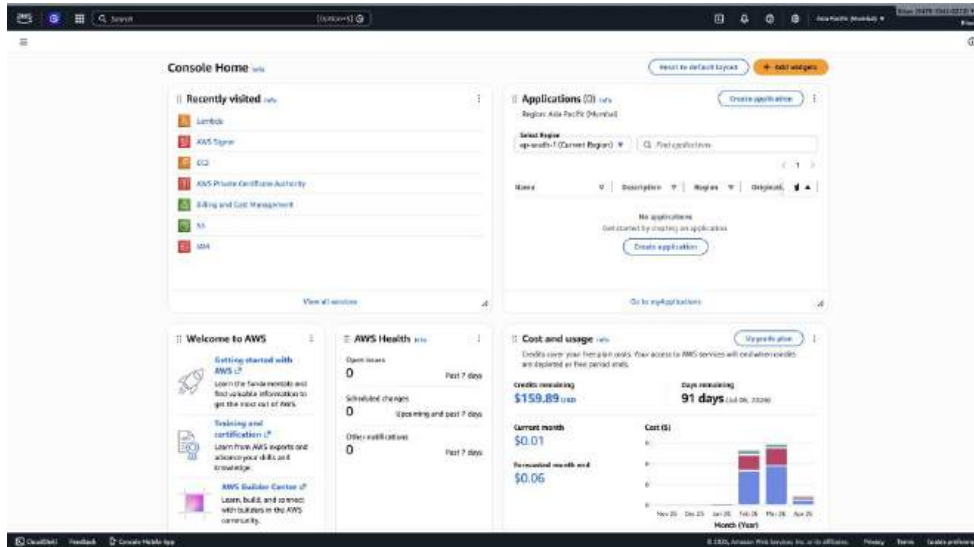


Seeing the output we can conclude that our project is successfully deployed and run in AWS without the help of port number.

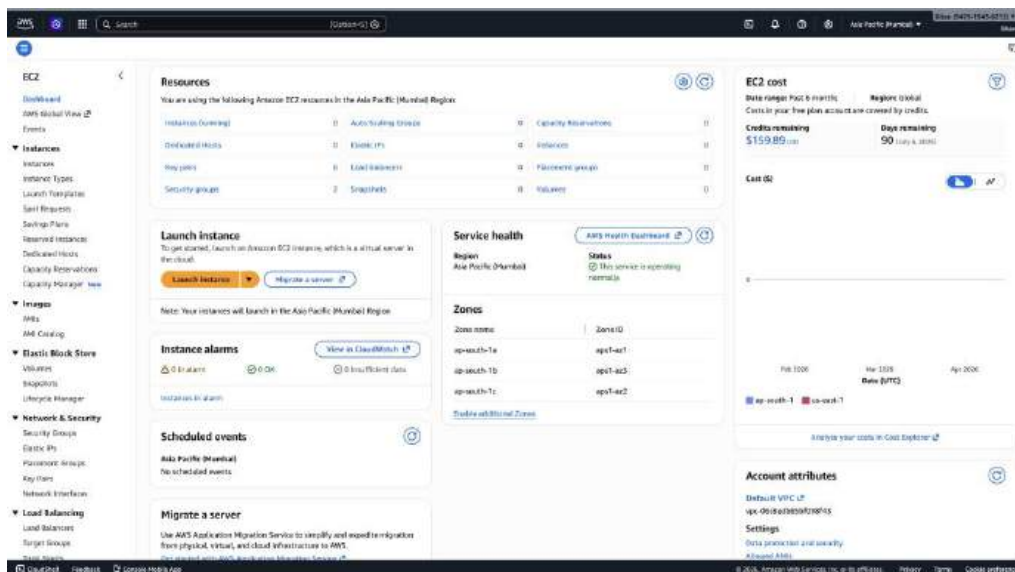
Assignment No. : 14

Problem Statement: Create an Elastic IP for an Instance.

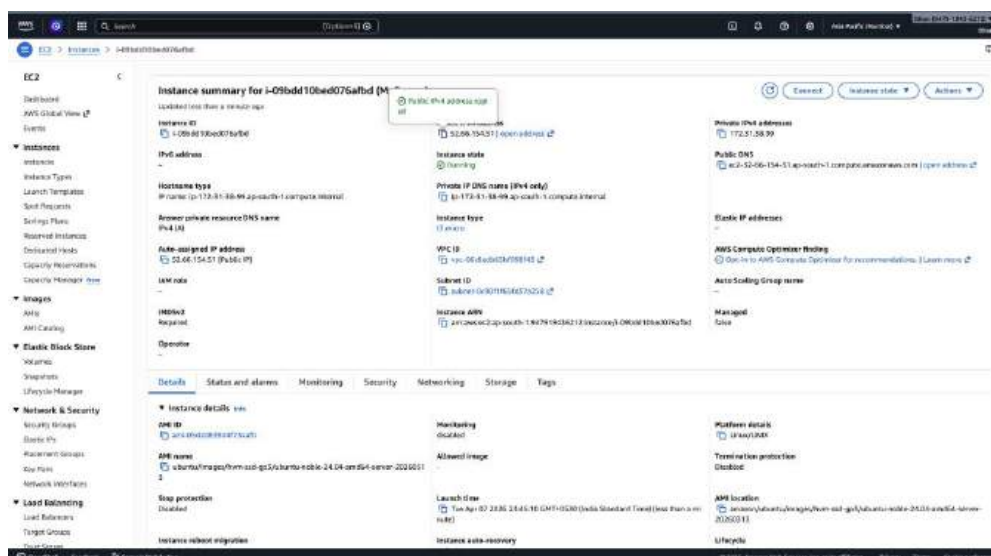
STEP 1-> Go to “EC2” from the AWS home screen.



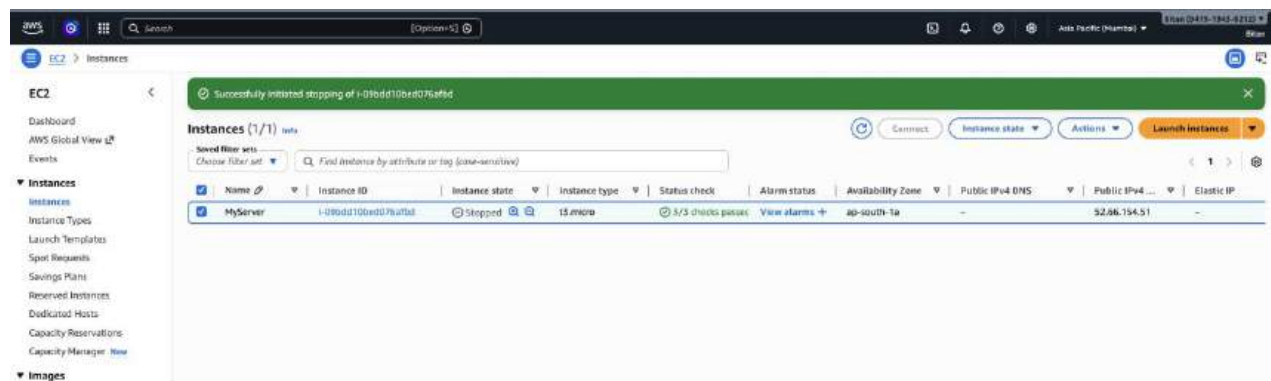
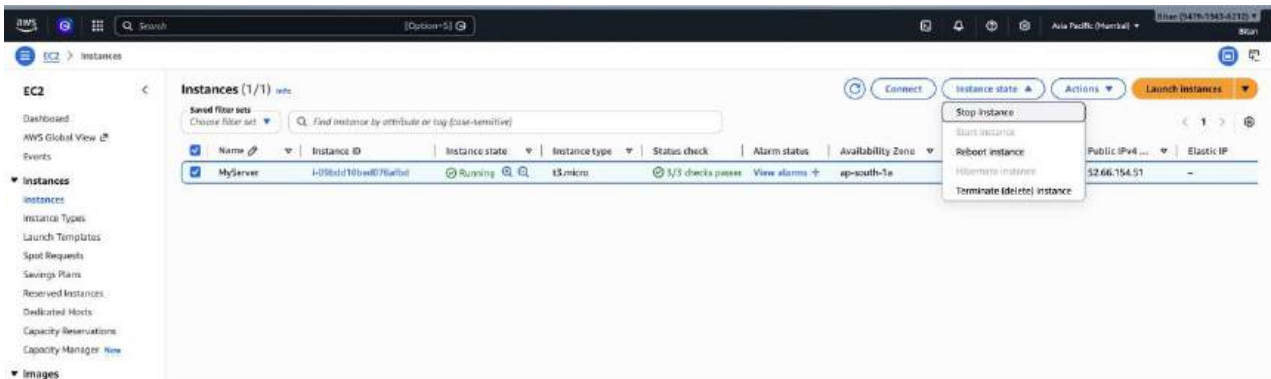
STEP 2-> From the left side menu, go to “Instances”.



STEP 3-> Select an instance or Launch a new one and save its Public IPv4 address in a text file for future comparison.



STEP 4-> With the instance selected, open the Instance State dropdown menu and select “Stop Instance” option to temporarily put the instance in the sleep state. Now again start this instance.

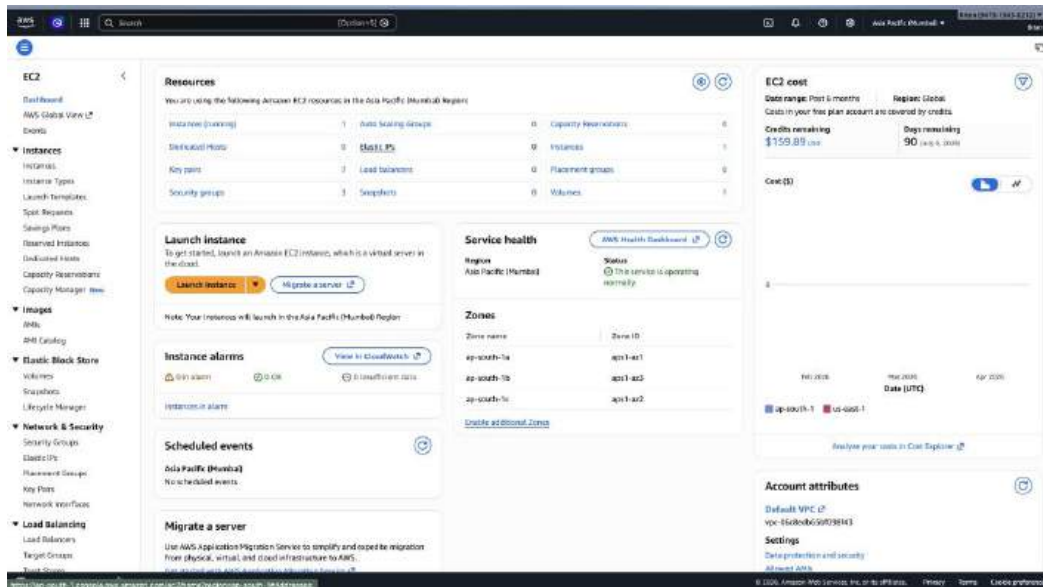


STEP 5-> Now again save the Public IPv4 address in the text file and compare it with the previous IP.

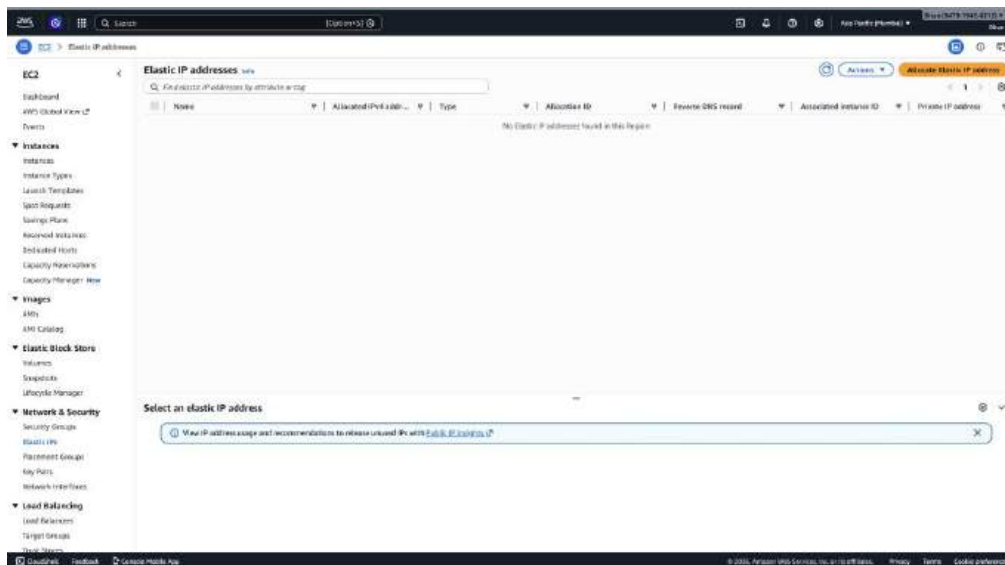


It is evident that the IP address changes every time the instance is launched. To avoid it Elastic IP has to be assigned.

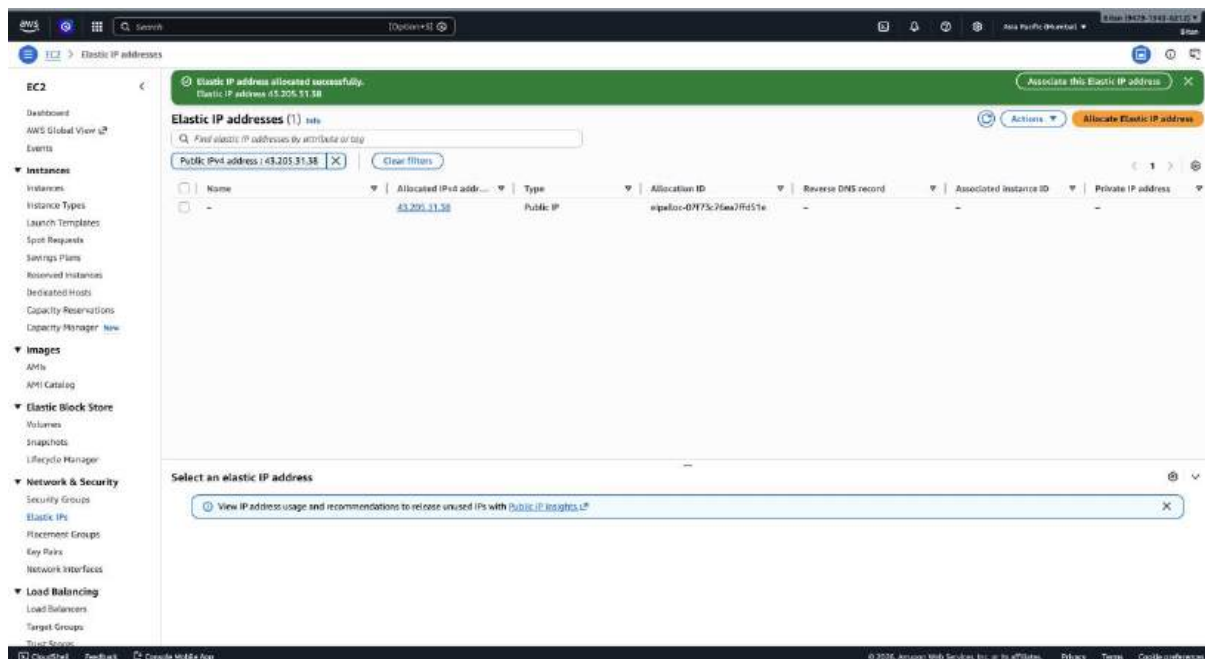
STEP 6-> From the EC2 Dashboard, go to the “Elastic Ips” option.



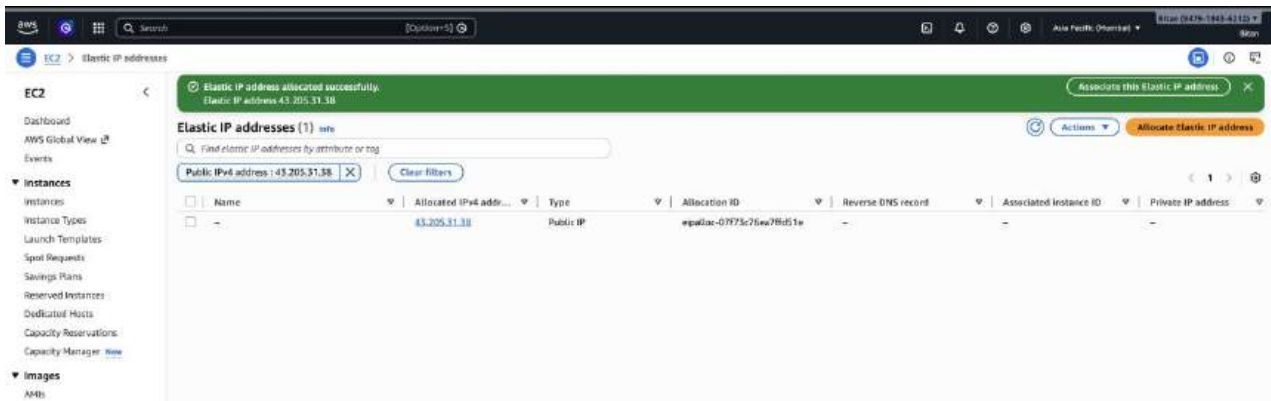
STEP 7-> Click on “Allocate Elastic IP” button on the top right corner of the screen.



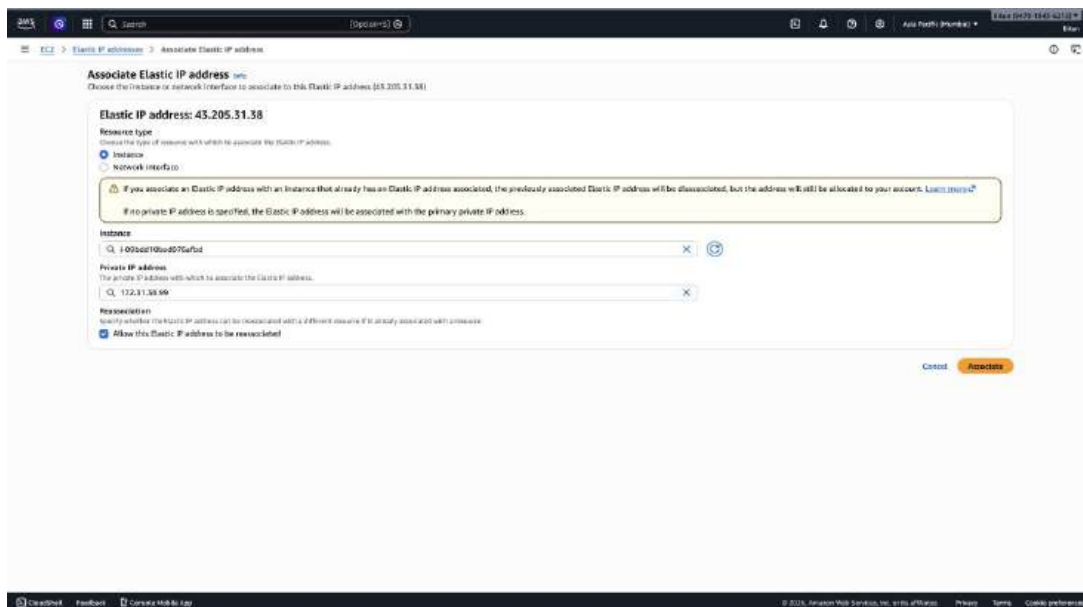
STEP 8-> With “Amazon’s pool of IPv4 Addresses” selected under Public IPv4 Address pool, click on “Allocate”.



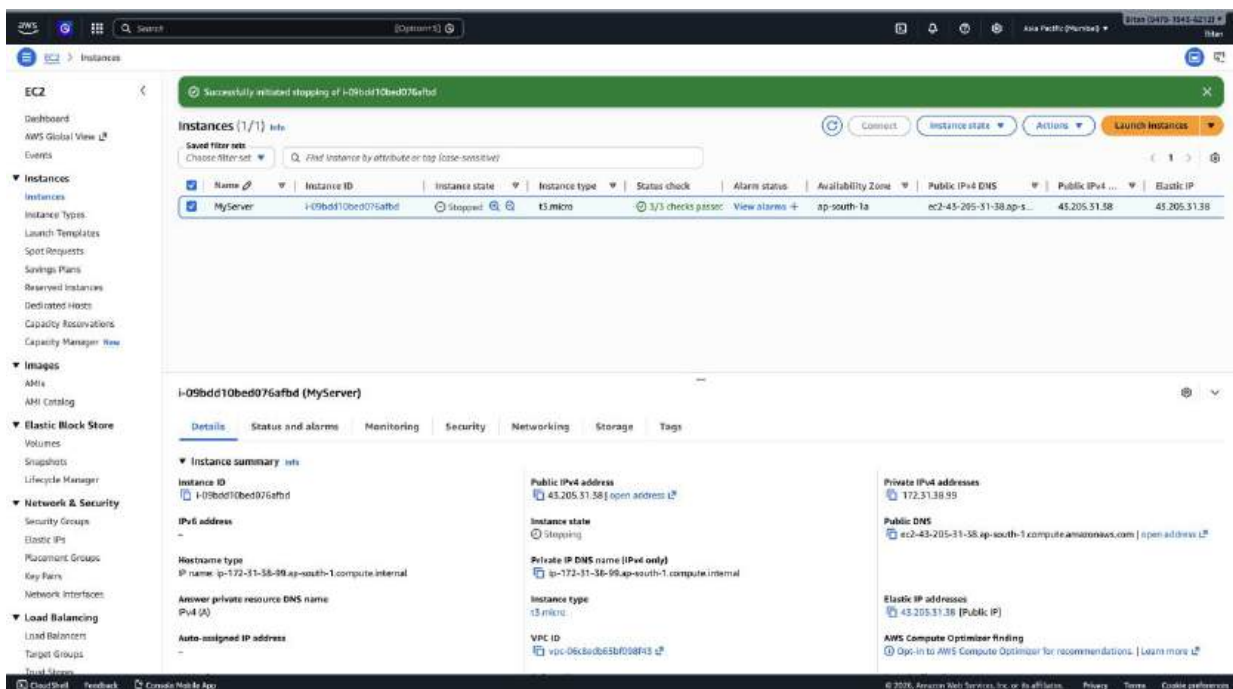
STEP 9-> The Elastic IP has been created. Then Click on “Associate Elastic IP Address” showing in the green box.



STEP 10-> With resource type selected as “Instance”, select the name of the instance & its private IP address. Check the bottom checkbox to reassociate the Elastic IP. Then click on “Associate”.



STEP 11-> Copy the IPv4 address before selecting “Stop Instance” and again start this instance and copy the IPv4 address.



Instances (1/1) info

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP
MyServer	i-09bdd10bed076afbd	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1a	ec2-43-205-31-38.ap-s...	43.205.31.38	43.205.31.38

i-09bdd10bed076afbd (MyServer)

Instance summary info

Instance ID: i-09bdd10bed076afbd

IPv4 address: -

Hostname type: IP name: ip-172-31-38-99.ap-south-1.compute.internal

Answer private resource DNS name (IPv4 (A)): -

Auto-assigned IP address: -

Public (IPv4) address: 43.205.31.38 | [open address](#)

Instance state: Running

Private IP DNS name (IPv4 only): ip-172-31-38-99.ap-south-1.compute.internal

Instance type: t3.micro

VPC ID: vpc-06c80b65u098f43

Private IPv4 addresses: 172.31.38.99

Public DNS: ec2-43-205-31-38.ap-south-1.compute.amazonaws.com | [open address](#)

Elastic IP addresses: 43.205.31.38 (Public IP)

AWS Compute Optimizer finding: Opt-in to AWS Compute Optimizer for recommendations. | [Learn more](#)

STEP 12-> No changes are observed.

```

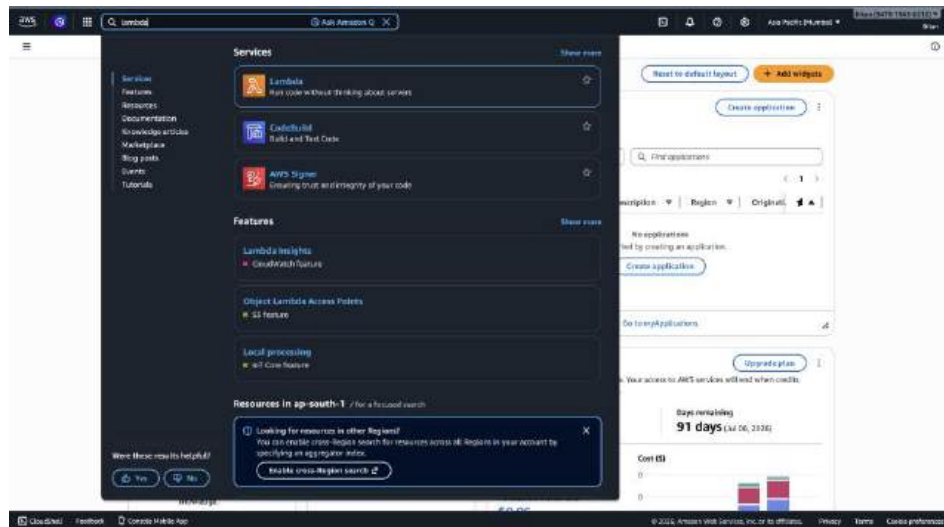
Before assigning the Elastic IP
52.66.154.51 - IPv4 address before
65.2.31.221 - IPv4 address after

After assigning the Elastic IP
43.205.31.38 - IPv4 address before
43.205.31.38 - IPv4 address after
  
```

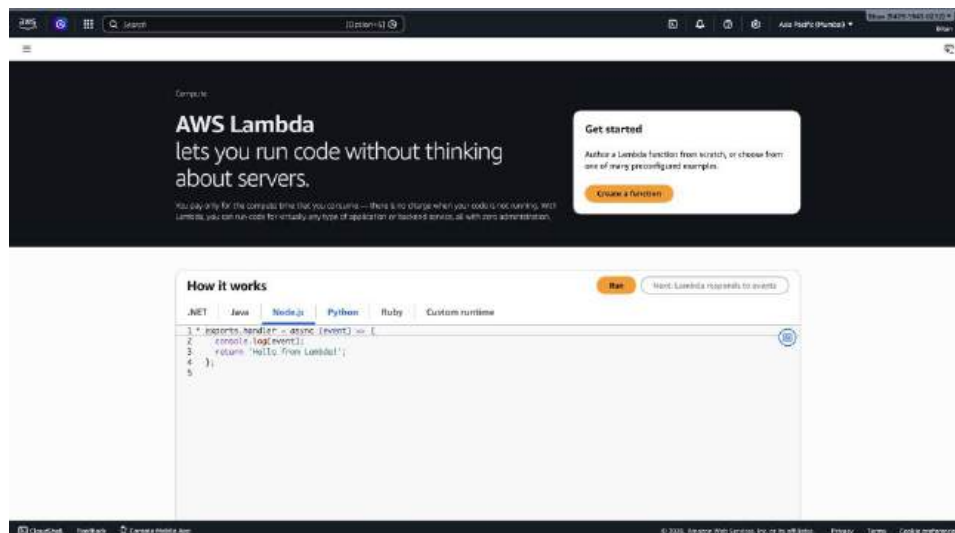
Assignment No.: 15

Problem Statement: Create a serverless computing service.

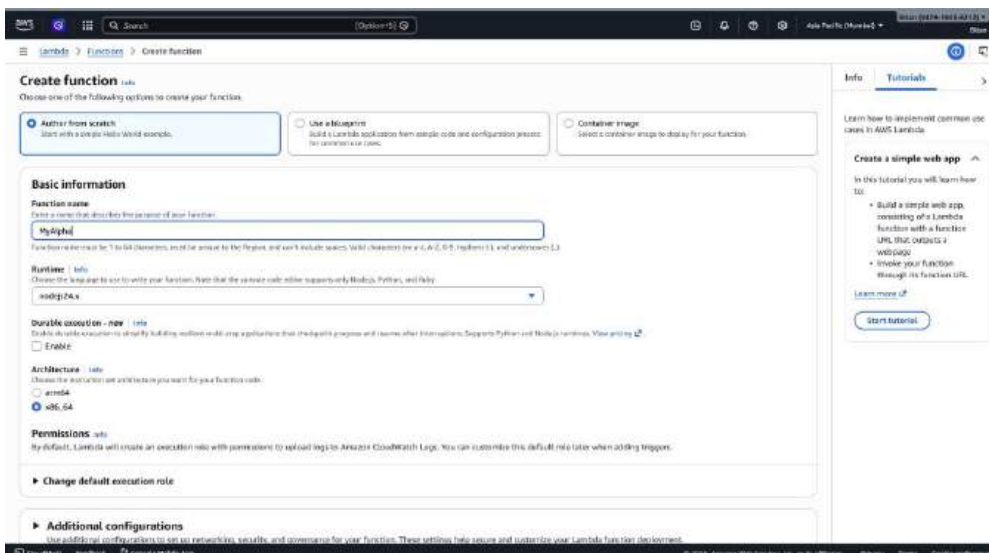
STEP 1-> Search for Lambda and click on it.



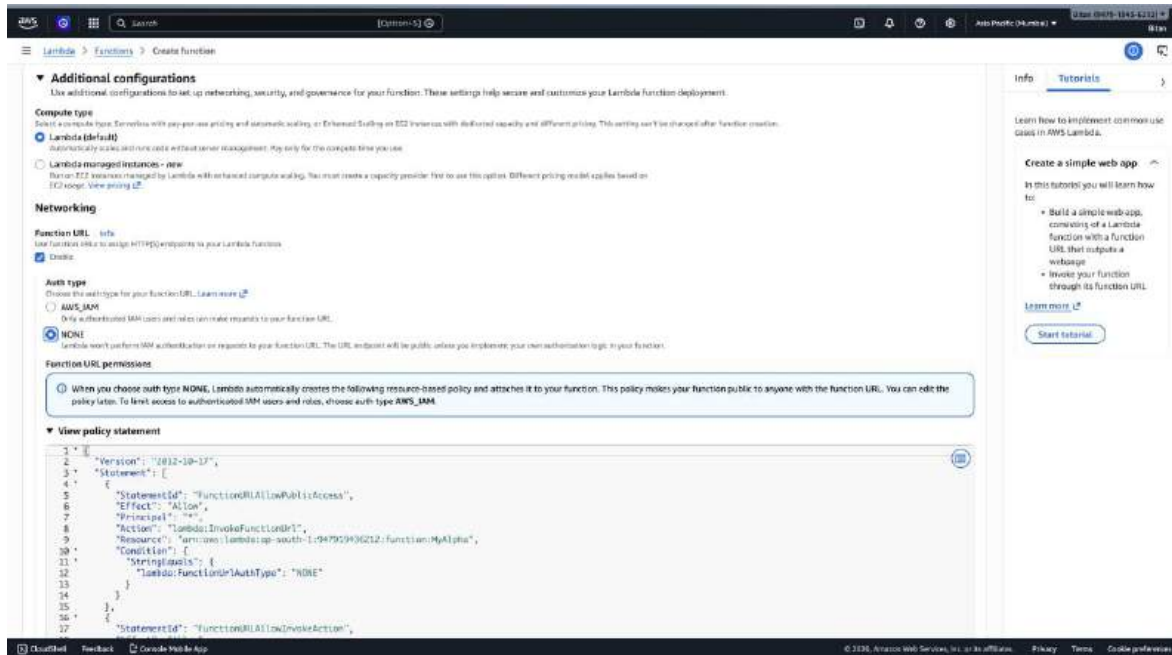
STEP 2-> Click on Create a function button.



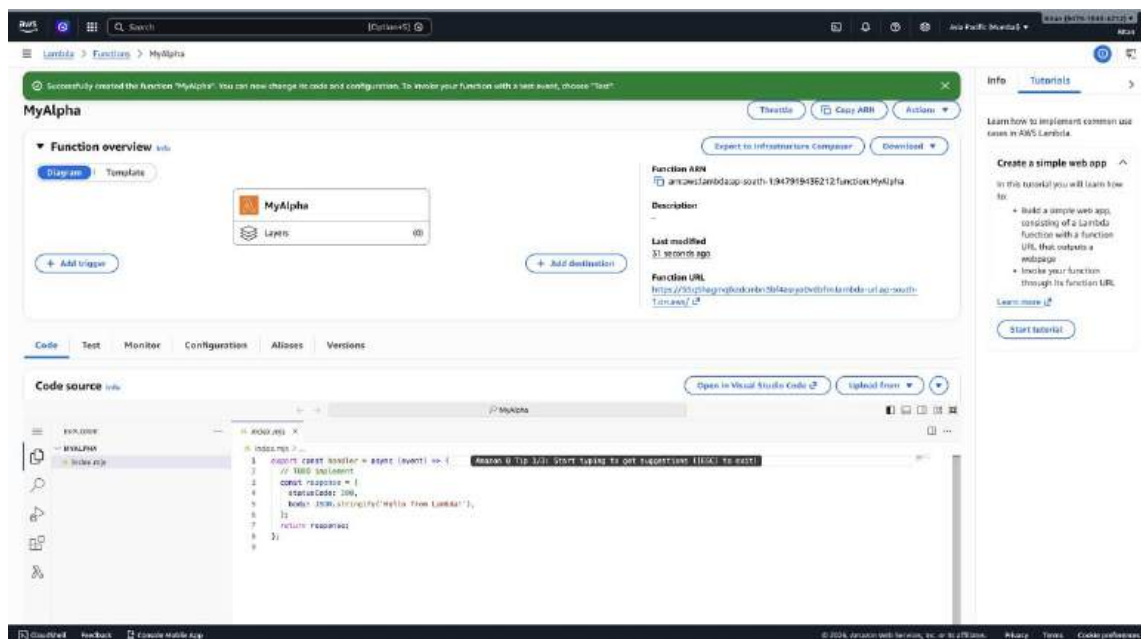
STEP 3-> Give a name to the function here "MyAlpha" and under runtime select "nodejs24.x".



STEP 4-> Under the Advanced settings, check the Enable Function URL checkbox & under Auth type, select NONE. Then click on Create Function.



STEP 5-> The function has been created. Now copy the function URL and paste it on a new tab.



STEP 6-> The following output should be obtained indicating we have created a serverless computing service.

